

Simulated Annealing

Simulated Quantum Annealing

Quantum Annealing



[https://drive.google.com/drive/folders/
1X_rWyyuS3y2MSze6uvX8IJ2gRbgfJ_y6?usp=share_link](https://drive.google.com/drive/folders/1X_rWyyuS3y2MSze6uvX8IJ2gRbgfJ_y6?usp=share_link)

TSUN-YUAN KU
Research Assistant

Table of contents

Chapter 1: Defining the Problem

- What problem are we trying to solve?
- Why can't classical algorithms solve it?

Chapter 2: Introduction to Simulated Annealing (SA)

- What is Simulated Annealing?
- From Heat Treatment to Optimization
- Mathematical Models
- Acceptance Probability Formula

Chapter 3: Quantum Concepts and Model Preliminaries

- Laying the Foundation: Essential Quantum Concepts
- Superposition
- Quantum Tunneling
- Hamiltonian & QUBO
- Choosing the Representation: Ising vs. QUBO
- What if I need the variables to be (-1, 1)?

Chapter 4: QUBO Modeling

- QUBO Modeling
- QUBO to QPU
- QUBO Models and Penalty Techniques

Chapter 5: Simulated Quantum Annealing (SQA)

- Starting from Classical Annealing
- SQA Core Model: Imaginary Time Slicing + PIMC (Path Integral Monte Carlo)
- Can SQA be executed without using Imaginary Time?

Chapter 6: NCHC Resources (Taiwania 3)

- What is Taiwania 3?
- Specifically, how do we use Taiwania 3?
- Will existing programs need adjustments for Supercomputer (HPC) environments?
- Research Demo

Chapter 7: D-Wave DEMO

- DEMO 1: NL Solver Basics - Solution (NL = Nonlinear)
- DEMO 2: NL Solver - Constraints - Solution
- DEMO 3: "Set" Variable - Knapsack Problem
- DEMO 4: "List" Variable - Basic Routing Problem

Chapter 8: Custom QUBO Application DEMO

- DEMO 1: From TSP to VRP (Traveling Salesperson Problem to Vehicle Routing Problem)

Chapter 9: Real-World DEMO

- DEMO 1: Traffic Flow Optimization

Q&A & References

Chapter 1



Problem Definition

What problem are we trying to solve?

NP-hard Problems in Combinatorial Optimization

Goal: To find $\min_{x \in X} f(x)$

$x \in \{0,1\}^n$: Binary decision variables

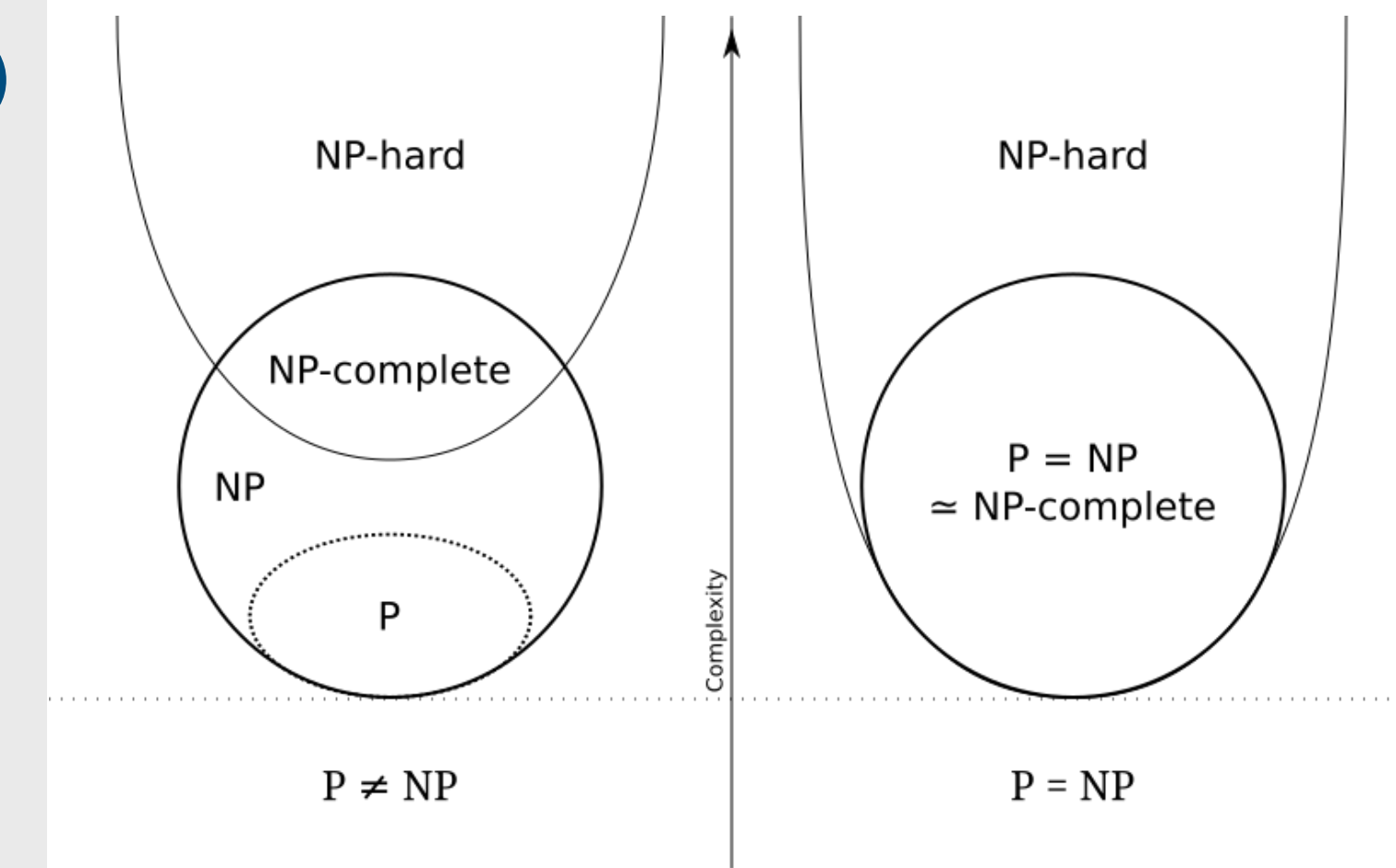
$X \subset \{0,1\}^n$: The set of all feasible combinatorial solutions (or Search space of feasible solutions)

$f(x)$: **Objective function**, evaluating the **cost** or **energy** of a specific solution.

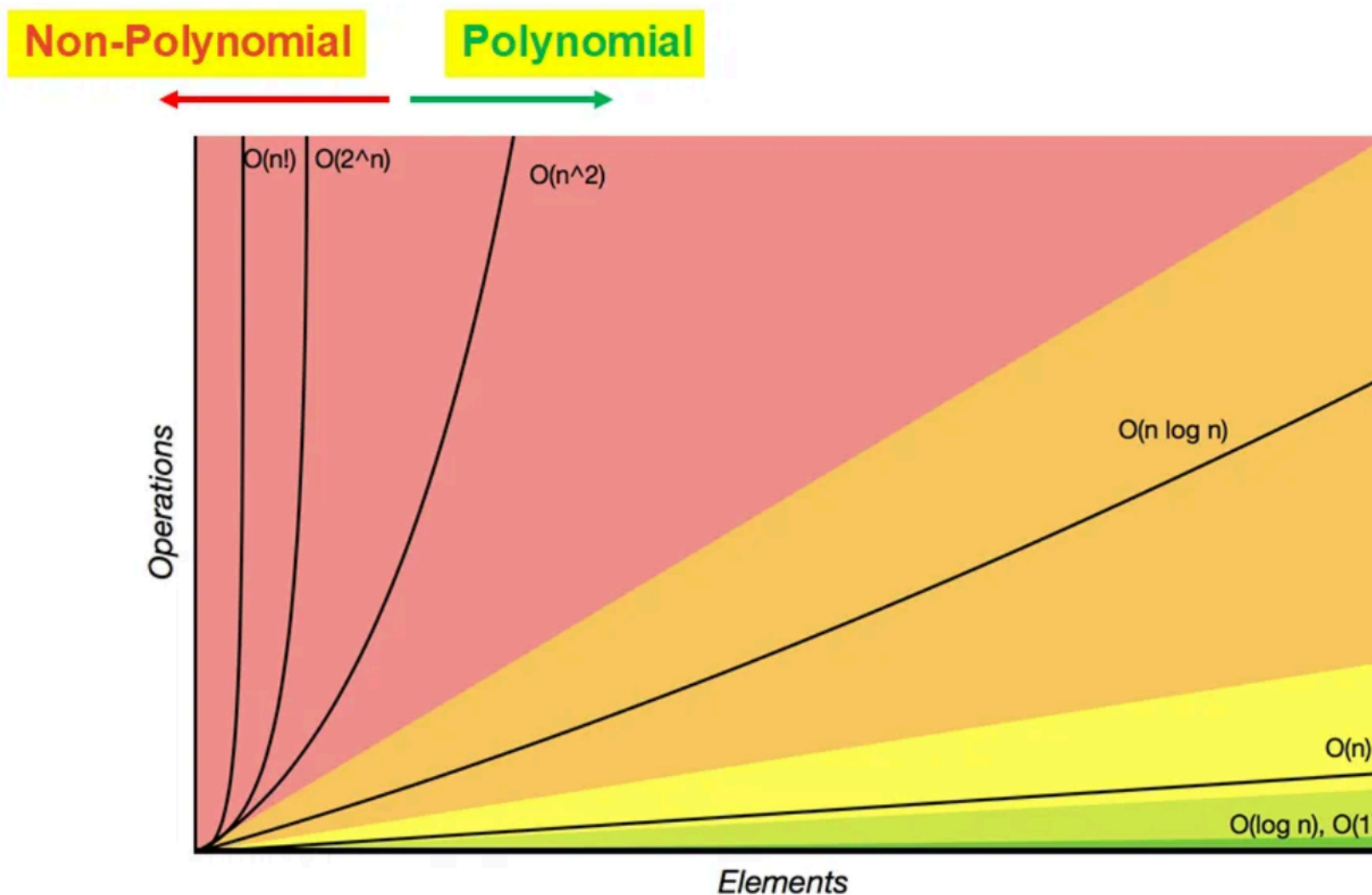
Traveler Salesman Problem (TSP)

Vehicle Routing Problem (VRP)

0/1 Knapsack Problem



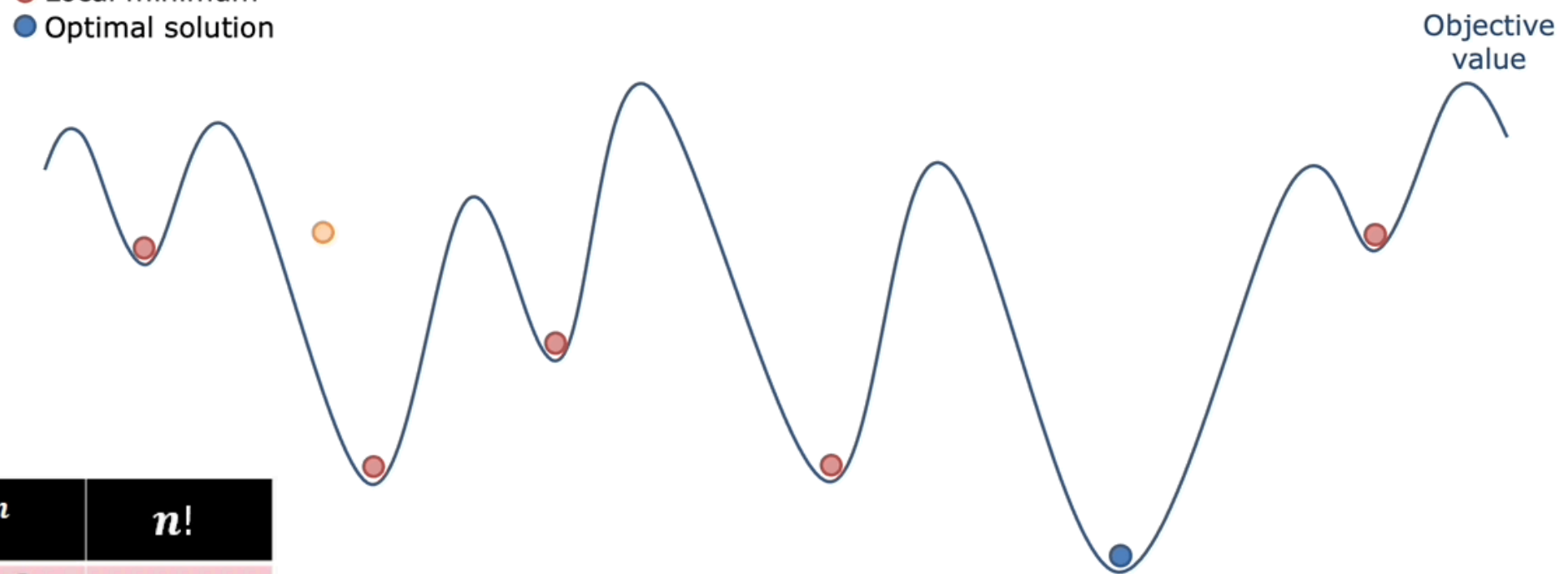
Why can't classical algorithms solve it?



- 解空間爆炸性成長
- Local minima trap

	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
10	3.32	10	33.22	10^2	10^3	$\sim 10^3$	3628800
10^2	6.64	10^2	664.39	10^4	10^6	$\sim 10^{30}$	$\sim 10^{158}$
10^3	9.97	10^3	9965.78	10^6	10^9	$\sim 10^{300}$	X
10^4	13.29	10^4	132877.12	10^8	10^{12}	$\sim 10^{3000}$	X

Local minima
 ● Current solution
 ● Local minimum
 ● Optimal solution



Polynomial Non-Polynomial

Chapter 2

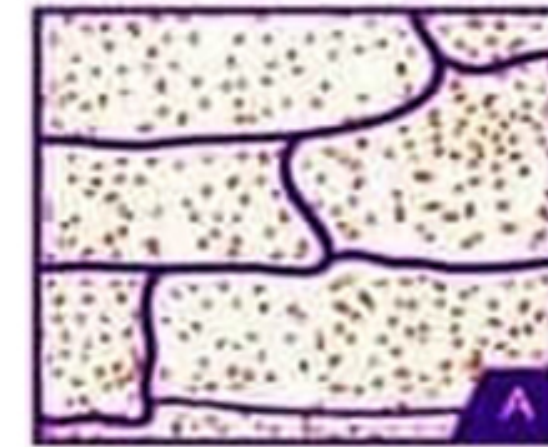
Introduction to Simulated Annealing

What is Simulated Annealing (SA)?

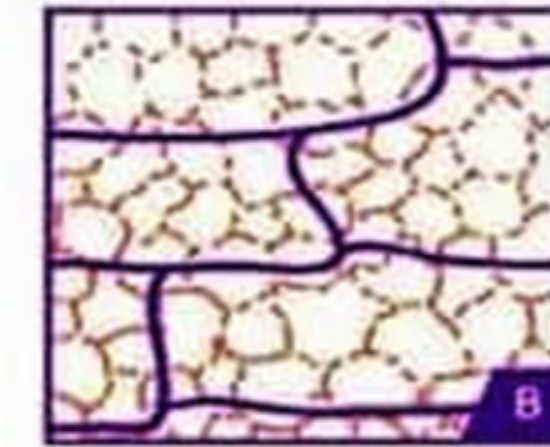
Simulated Annealing is a heuristic algorithm inspired by the physical process of annealing in metallurgy.

It allows for the acceptance of inferior (worse) solutions, providing a mechanism to escape local minima.

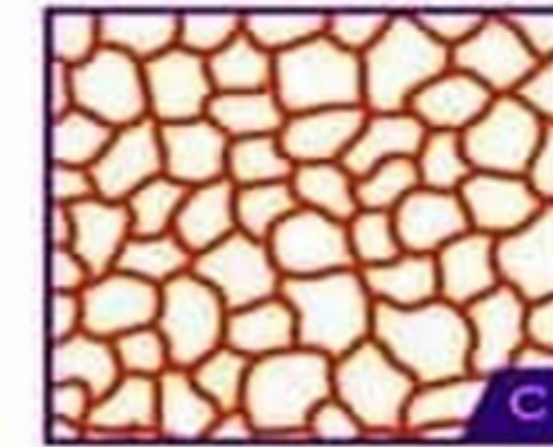
Annealing is the heat treatment process that softens a metal that has been hardened nearby cold working.



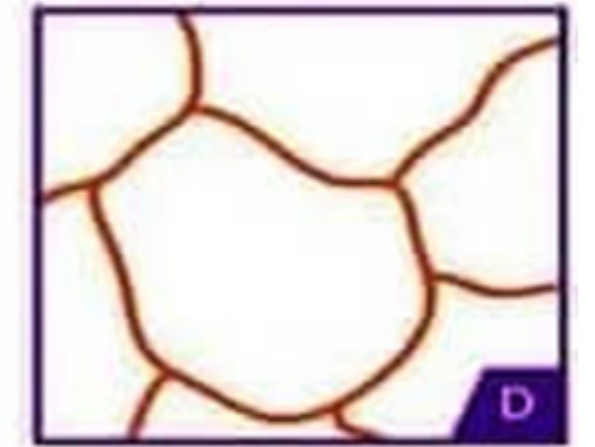
initial cold sfate



Heating; hiah sfress
areas dlssipato



Recrystallization
form

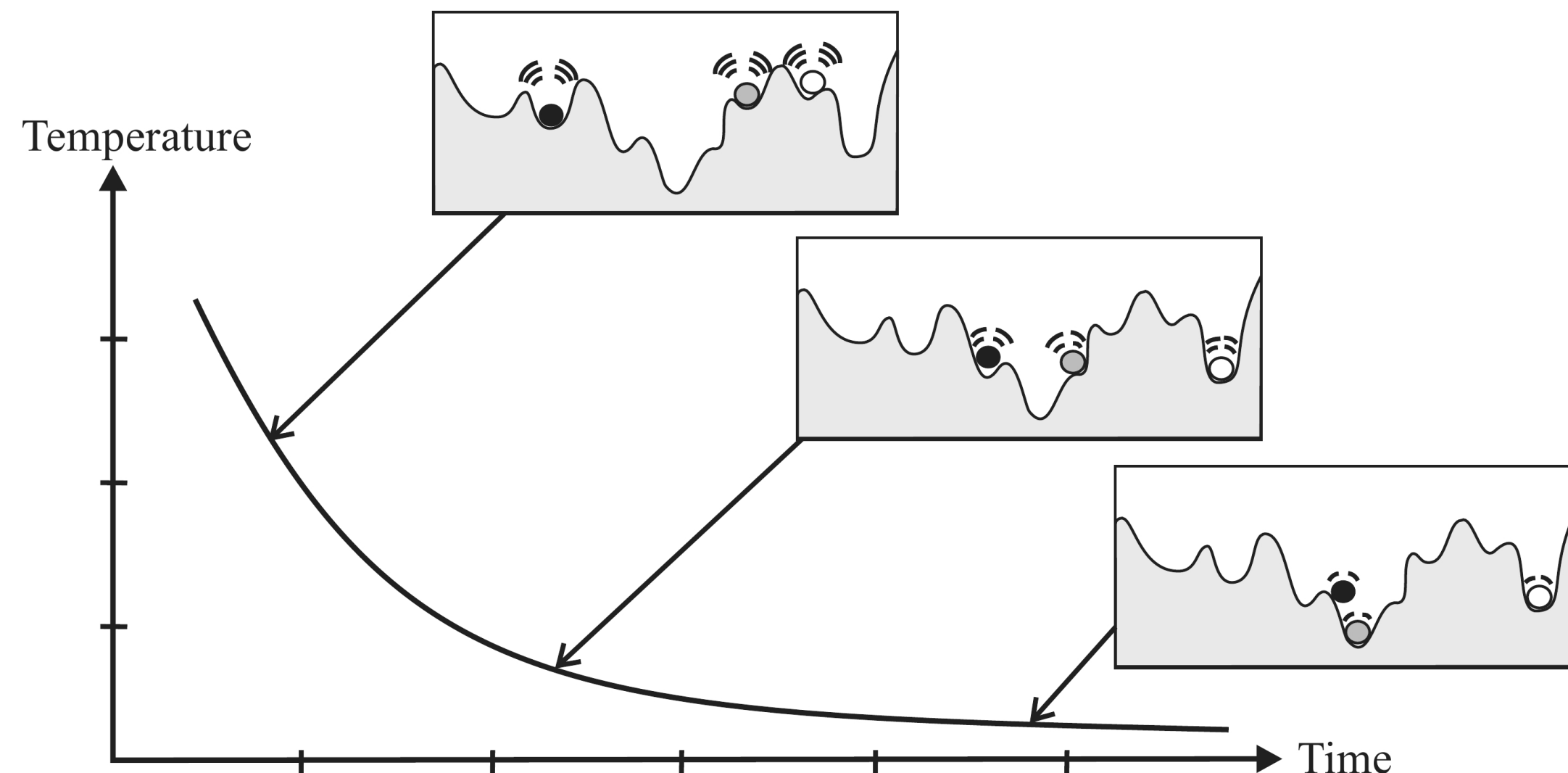


Recrystallization
form

From Heat Treatment to Optimization

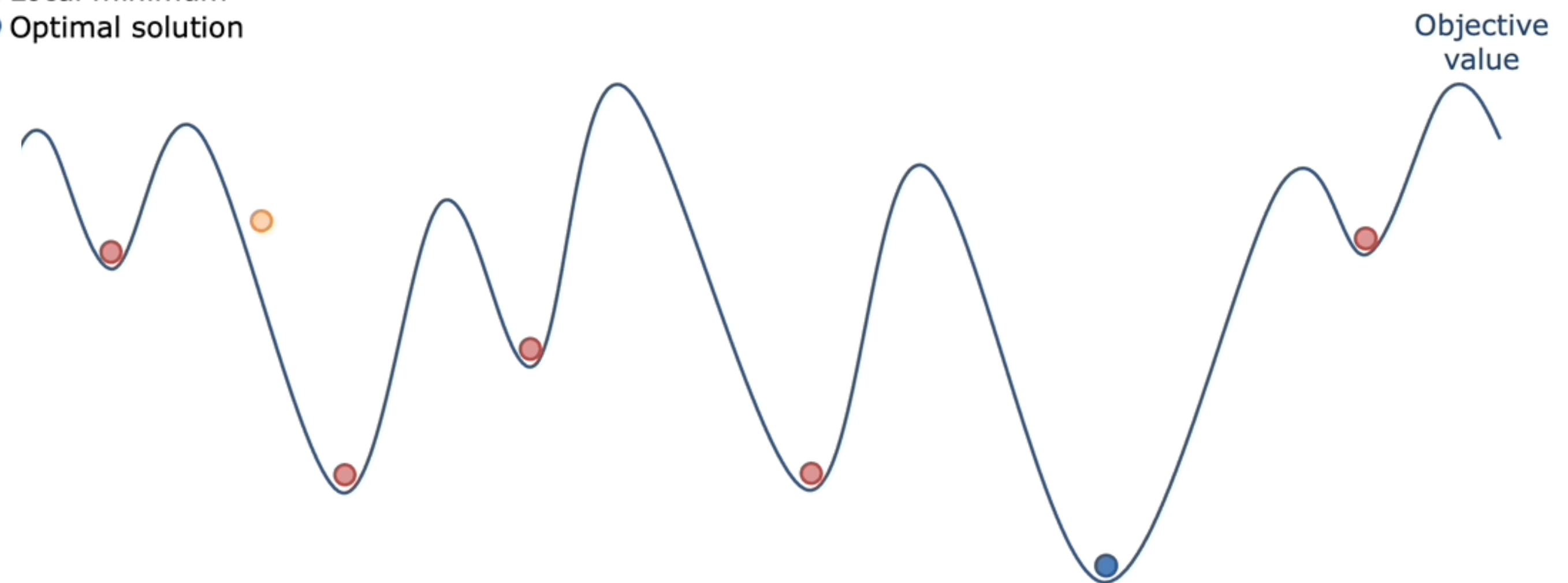
High Temperature → Large-scale exploration and stochastic jumps.

Low Temperature → Fine-tuned exploitation and localized search.



Escape local minima

- Current solution
- Local minimum
- Optimal solution



- **Escape mechanism: Thermal fluctuation**

Mathematical Model

Hint: Various cooling schedules exist, each with distinct functional advantages.

T_0 : Initial Temperature

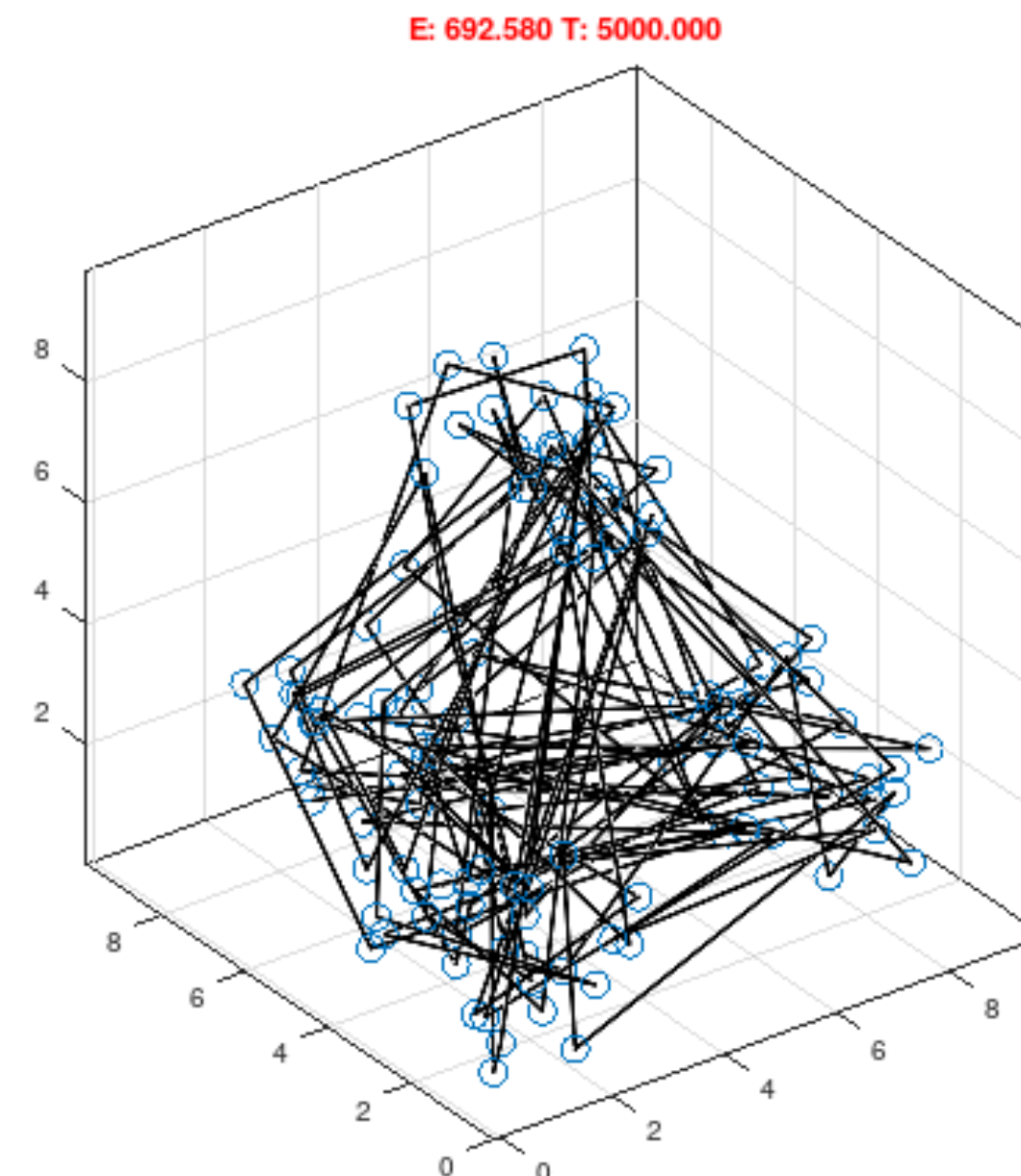
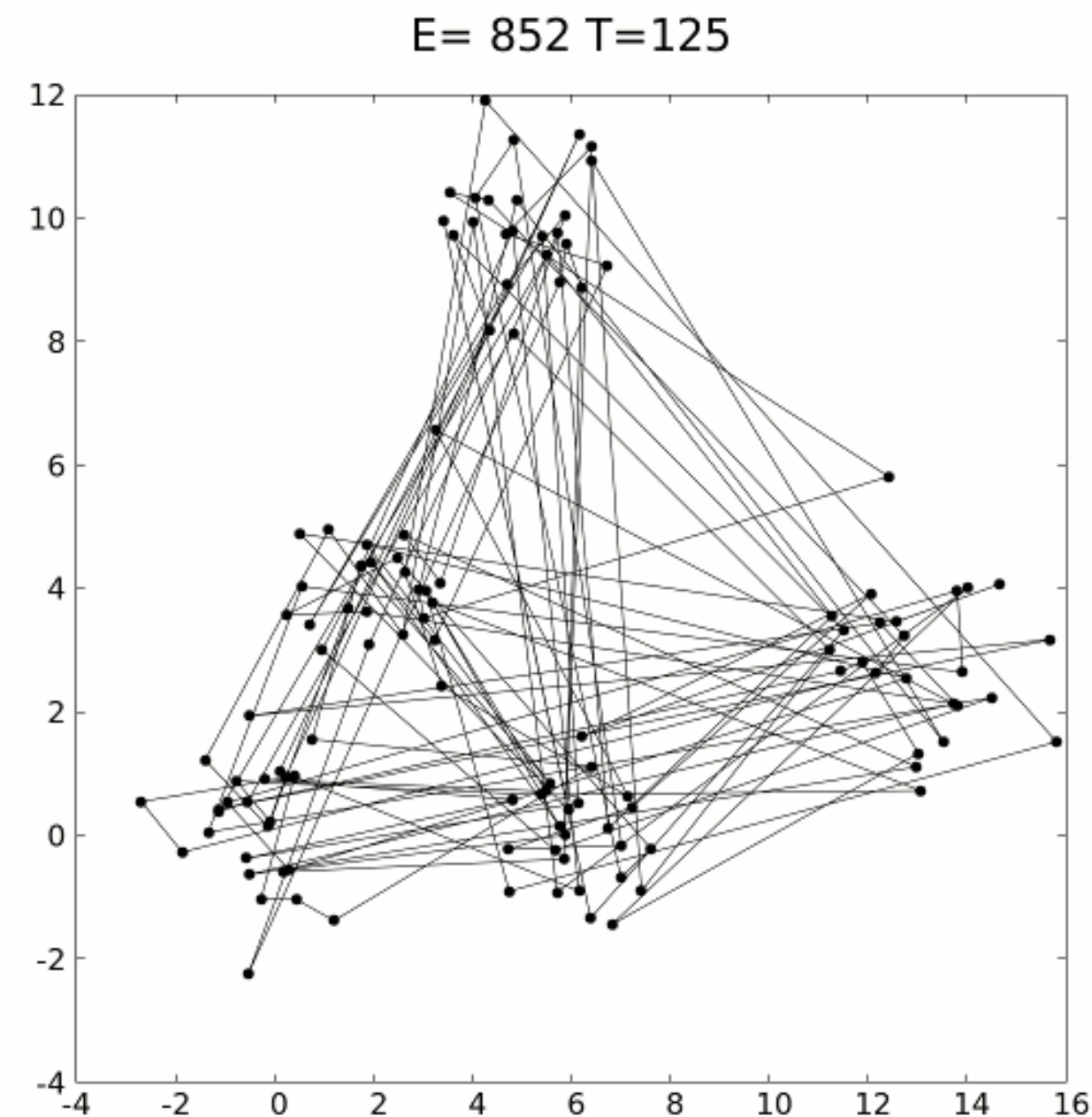
T_{min} : Minimum Temperature

α : Cooling Rate e.g. 98%

N_{inner} : Number of Iterations per Temperature Level

s, t : Starting and ending points of the path

- **Neighboring Solution Generation: How to transition between states? This depends on the algorithm's "Local Perturbation" or "Local Move" logic.**
- **Still trapped in local minima? Possible**

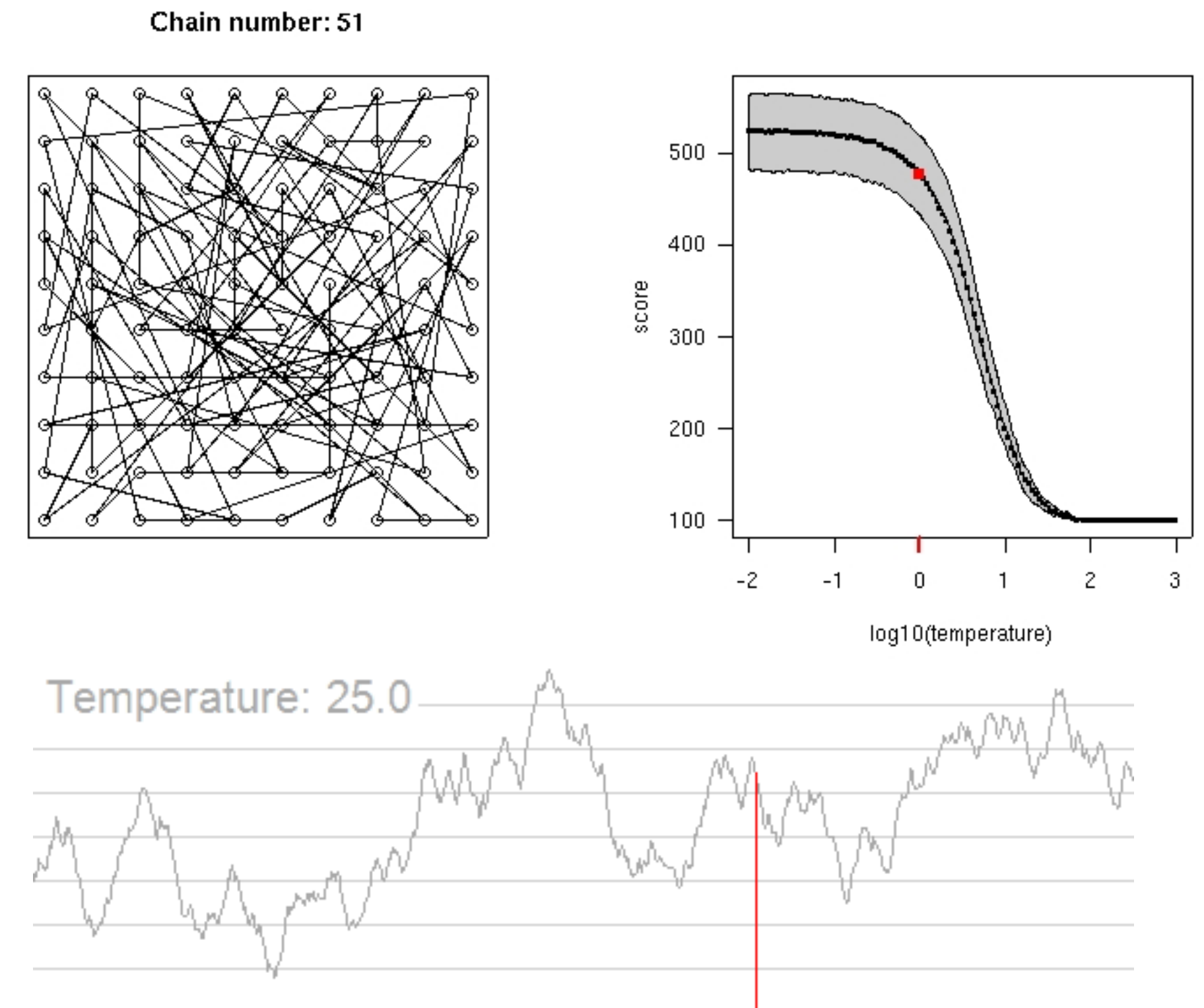


Acceptance Probability Formula

Metropolis Acceptance Criterion

$$P(x \rightarrow x') = \begin{cases} 1, & \text{if } \Delta < 0 \\ \exp\left(-\frac{\Delta}{T}\right), & \text{if } \Delta \geq 0 \end{cases} \quad \Delta = x' - x$$

if delta < 0 or `random.random()` < `math.exp(-delta / t)`:
current_path = **neighbor**



Chapter 3



Quantum Concepts and Model Preliminaries

Laying the Foundation: Essential Quantum Concepts

Superposition	→	To understand the qubit	Explores all possible states simultaneously.
Quantum Tunneling	→	To traverse energy barriers	Escapes local minima to find the global
Hamiltonian & QUBO	→	To define system energy	Maps problems to energy landscapes (Ground state = Optimal solution).

Superposition

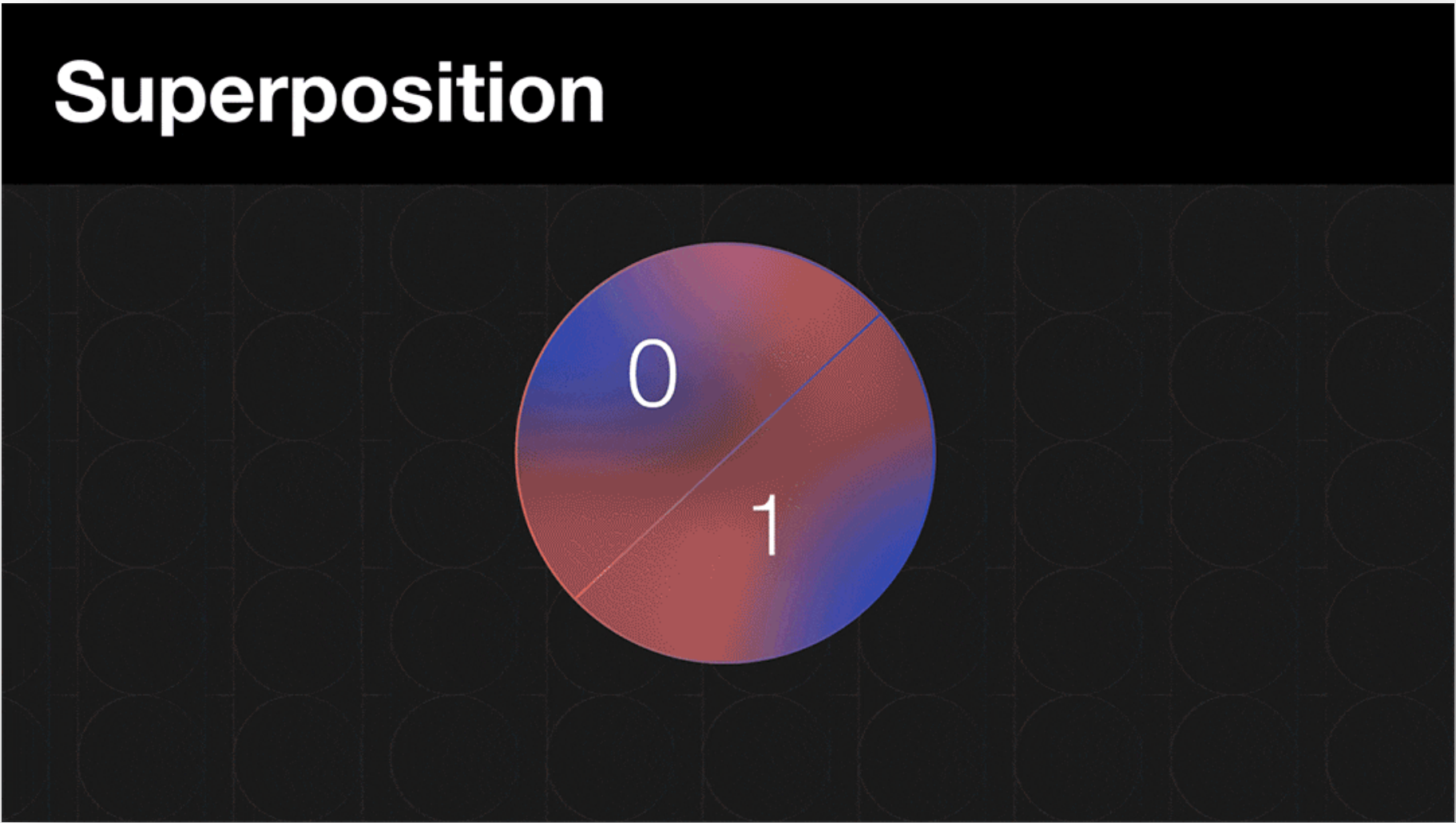
A qubit can exist in a superposition of 0 and 1 simultaneously. Superposition means that before measurement, the qubit occupies both $|0\rangle$ and $|1\rangle$ states with certain probabilities.

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle \quad \alpha, \beta \in \mathbb{C}, |\alpha|^2 + |\beta|^2 = 1$$

$|0\rangle$ 、 $|1\rangle$: The fundamental units basis states

α 、 β : Complex probability amplitudes.

$|\alpha|^2$ 、 $|\beta|^2$: The **probability** of the qubit resulting in $|0\rangle$ or $|1\rangle$ after measurement.



✗ **Classical:** Processing n variables requires storing 2^n combinations individually. **Quantum:** A set of n qubits represents 2^n combinations simultaneously.

○ **Classical:** Exploring all combinations of n variables requires storing 2^n states. **Quantum:** n qubits in superposition enable parallel evolution of information across 2^n combinations, although measurement ultimately yields a single result.

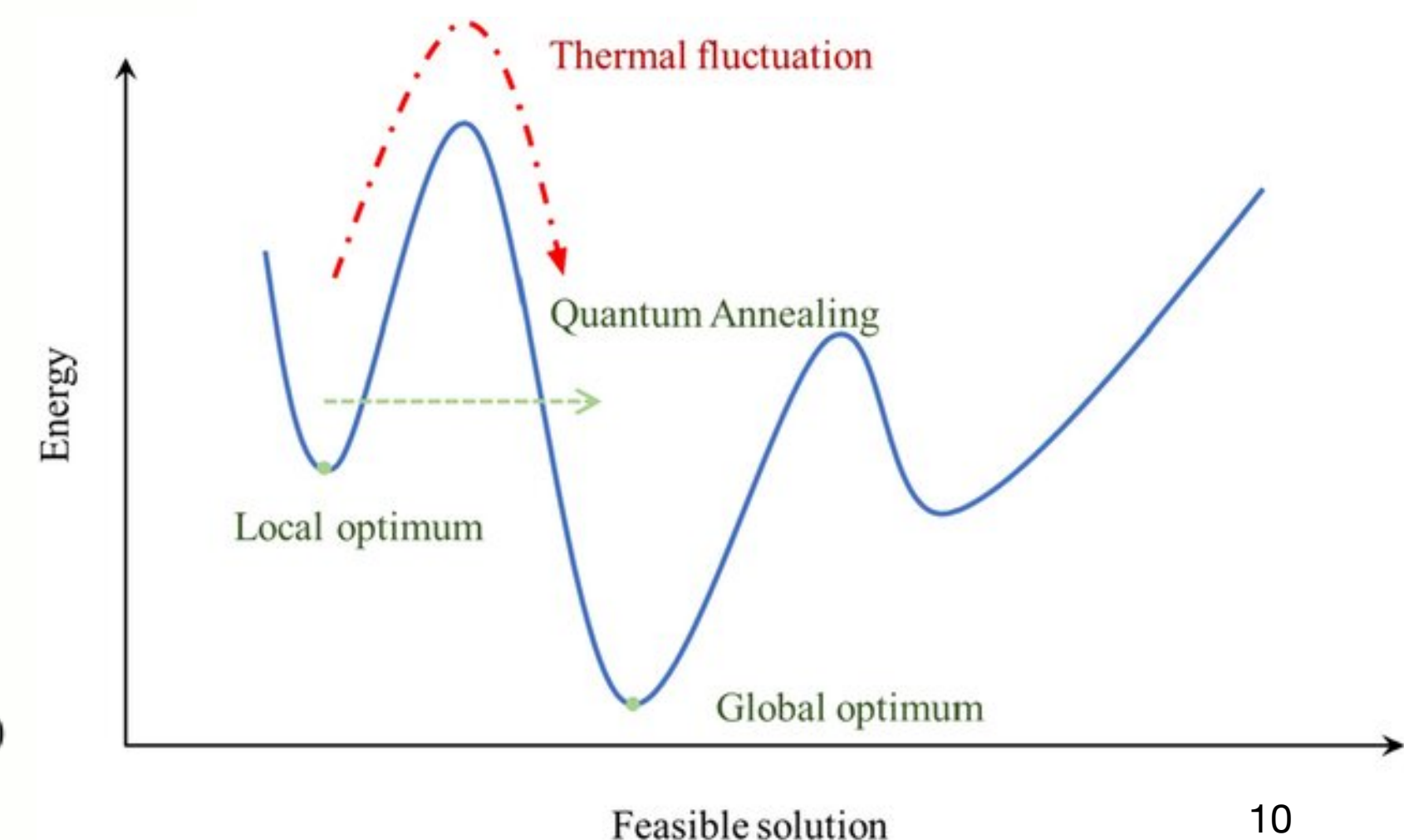
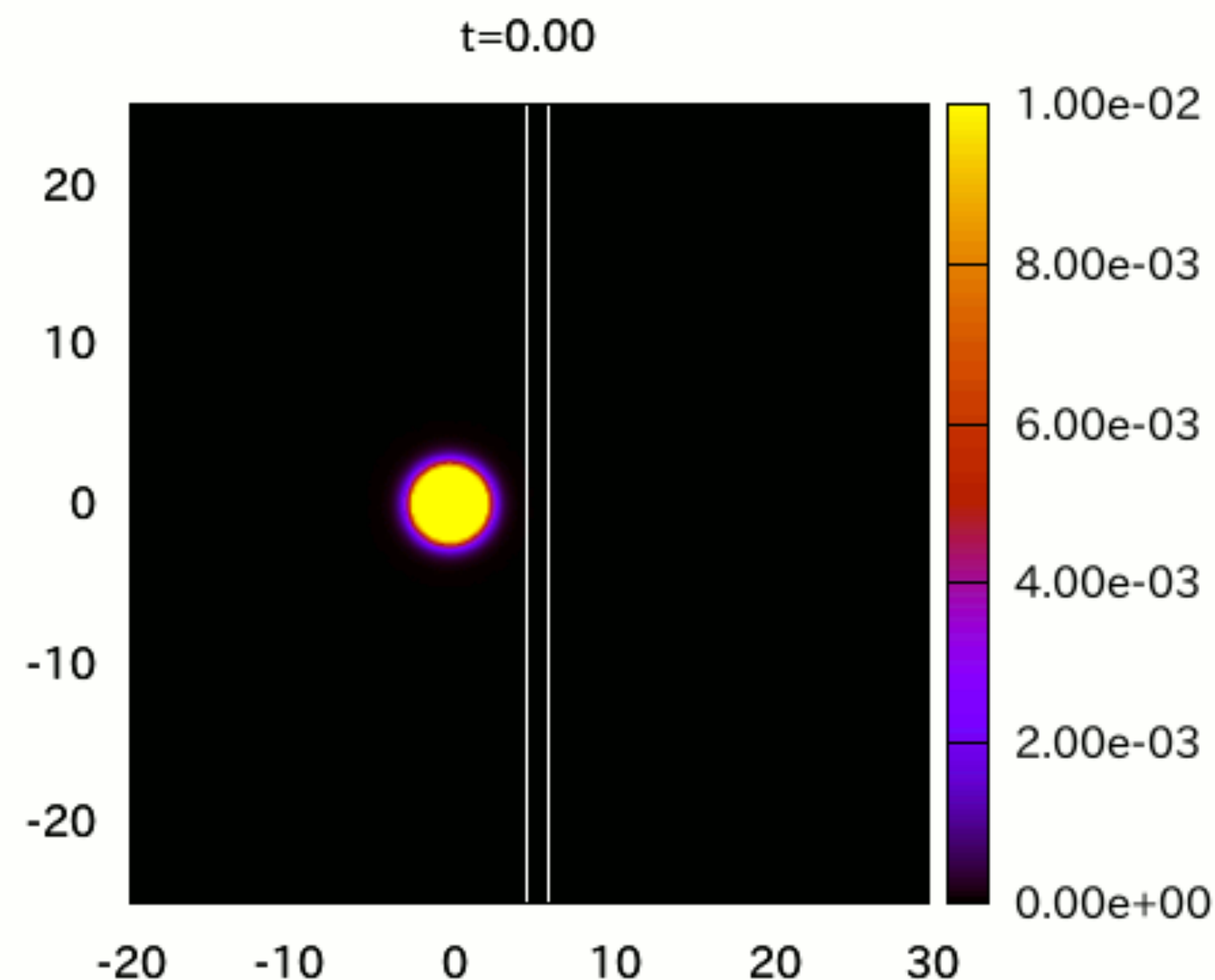
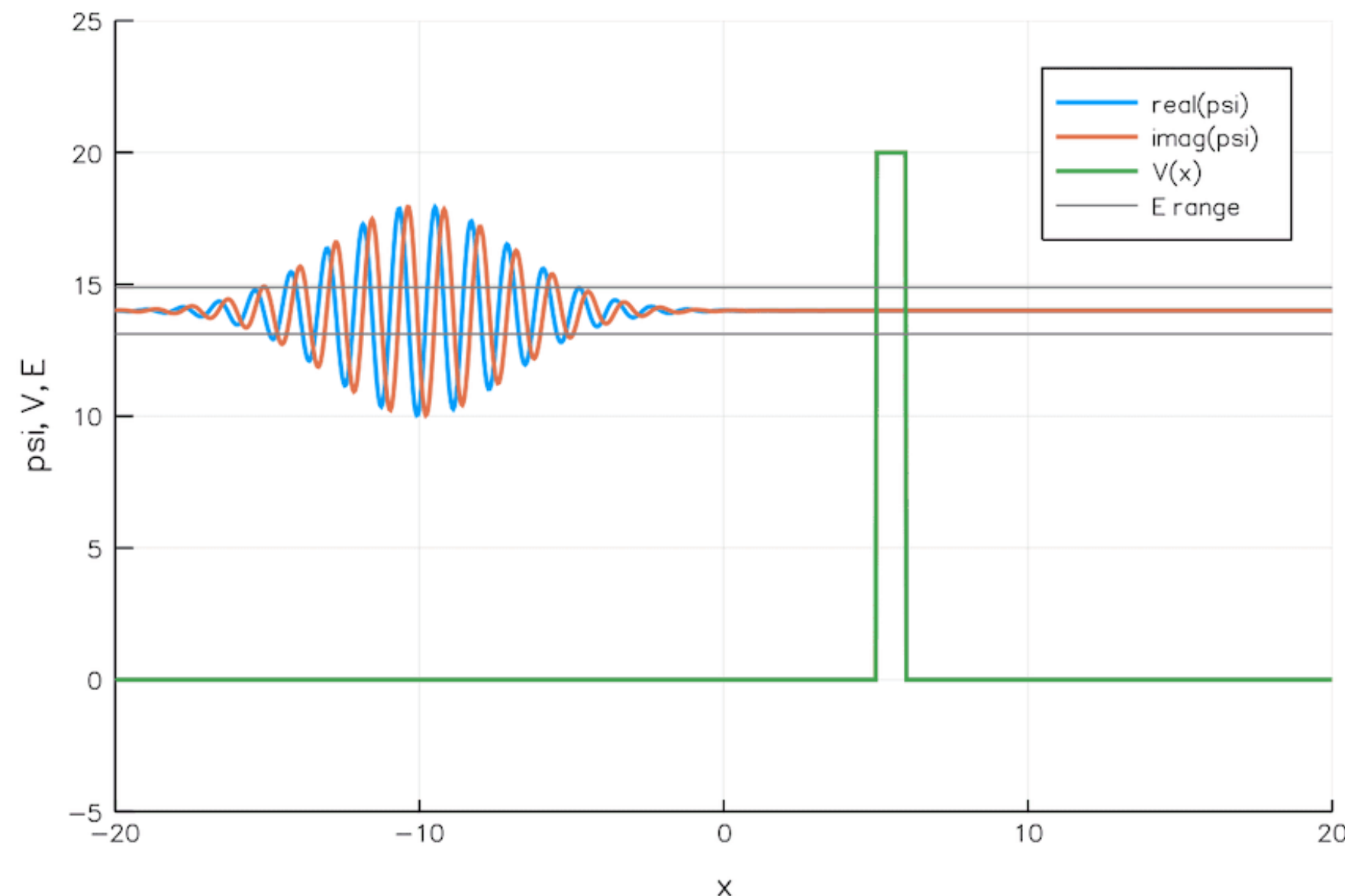
Quantum Tunneling

In classical physics, a particle hitting an **energy barrier** (potential barrier) higher than its own energy will bounce back. In quantum mechanics, it has a non-zero probability of "**tunneling through**."

Particle energy: E , Barrier height: V_0 , Barrier width: a , Particle mass: m , Reduced Planck constant: $\hbar$

$$P \propto \exp \left(-2a \sqrt{\frac{2m(V_0 - E)}{\hbar^2}} \right)$$

- **Probability Decreases:** A thicker barrier (a) or a larger energy gap ($V_0 - E$) reduces the tunneling probability.
- **Non-Zero Probability:** Critically, the probability is **never zero**!



Hamiltonian & QUBO

Quadratic Unconstrained Binary Optimization

In **Quantum Mechanics**, the **Hamiltonian** (H) is the core operator governing a quantum system's energy structure and time evolution. In **Quantum Annealing**, we deliberately construct an **energy landscape** (H_P) to guide the system toward its **ground state** — the optimal solution to the problem.

Quantum Mechanics

$$H = T + V \quad H = \text{red square} + \text{blue square} \leftrightarrow \text{green square}$$

T **kinetic energy**: Related to "particle mobility."

V **potential energy**: Represents field constraints.

$$H_P = \sum h_i \sigma_i^z + \sum J_{ij} \sigma_i^z \sigma_j^z \quad (\text{Ising Form})$$

$\sigma_i^z \in \{-1, +1\}$: Pauli-Z operator (Logical qubit state).

h_i : Individual variable bias.

J_{ij} : Interaction term between variables.

$$\min_{x \in \{0,1\}^n} \sum Q_{ii} x_i + \sum Q_{ij} x_i x_j \quad (\text{QUBO Form})$$

$x_i \in \{0,1\}$: Binary decision variables.

Q : Matrix representing weights and relationships.

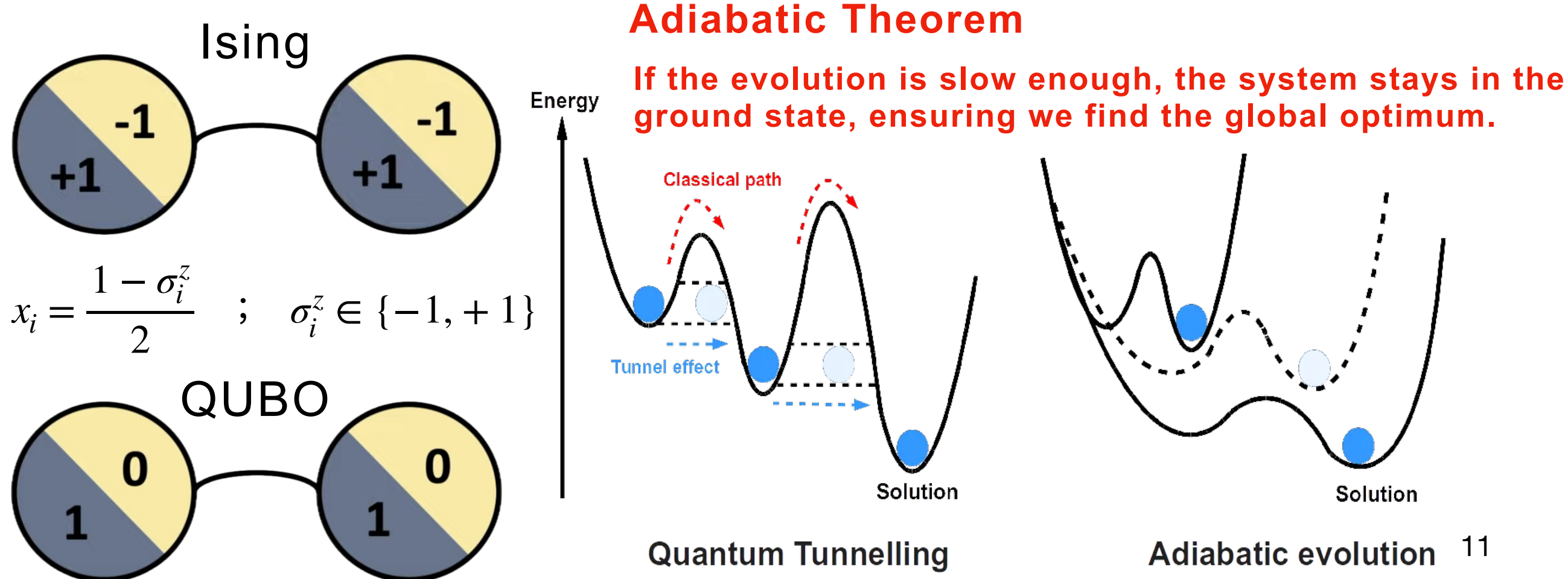
Quantum Annealing (Time-varying H)

$$H(t) = A(t)H_0 + B(t)H_P \quad H_0 = - \sum_i \sigma_i^x$$

H_0 **Initial Hamiltonian**: Similar to T ; provides the "wave-like" exploration power.

H_P **Problem Hamiltonian**: The final target potential. It corresponds to the **QUBO cost function** we designed.

Linear Annealing Example: $H(t) = \left(1 - \frac{t}{T}\right) H_0 + \left(\frac{t}{T}\right) H_P$, t : time



Choosing the Representation: Ising vs. QUBO

Ising

Pros:

- Suitable for observing **energy evolution** and **physical meaning**, or for users who want direct control over quantum coupling.
- Directly specifies the bias energy (h_i) and coupling strength (J_{ij}) for each qubit, providing more intuitive control over the energy landscape.

Cons:

- Lower modeling intuitiveness.
- Less suitable for direct formulation from 0/1 conditions (requires variable conversion).

QUBO

Pros:

- Suitable for users starting from **application modeling**, particularly combinatorial optimization problems.
- Clearer structure for modeling logic, constraints, and penalty terms.

Cons:

- Lacks intuitive control over the energy physics.
- **QUBO** → **Ising** may generate redundant constants or coefficient scaling issues.

$$\text{QUBO to Ising : } Q_{ij}x_ix_j = Q_{ij} \cdot \frac{(1-s_i)(1-s_j)}{4} = \frac{Q_{ij}}{4} \left(1 - s_i - s_j + s_is_j \right)$$



@nsqwertyful · 4 年前

This is a fantastic video. Thanks. Does Ising conversion offer any benefit and is it a mandatory step?



1



@dwavequantum · 4 年前 (已編輯)

Some problems are easier to formulate as QUBO, and others are easier to formulate as Ising - this is really up to the user. You can choose to write your programs and submit problems in either form. The system natively runs Ising problems, so all problems submitted to the system will eventually be converted to Ising. If you choose to write your programs using the QUBO representation, the Ocean SDK will do this conversion for you.



1



What If I Need the Variables to be (-1, 1)?

In many models, (-1, 1) better represents variable characteristics.

This can be implemented using `vartype='SPIN'`, which requires no manual conversion.

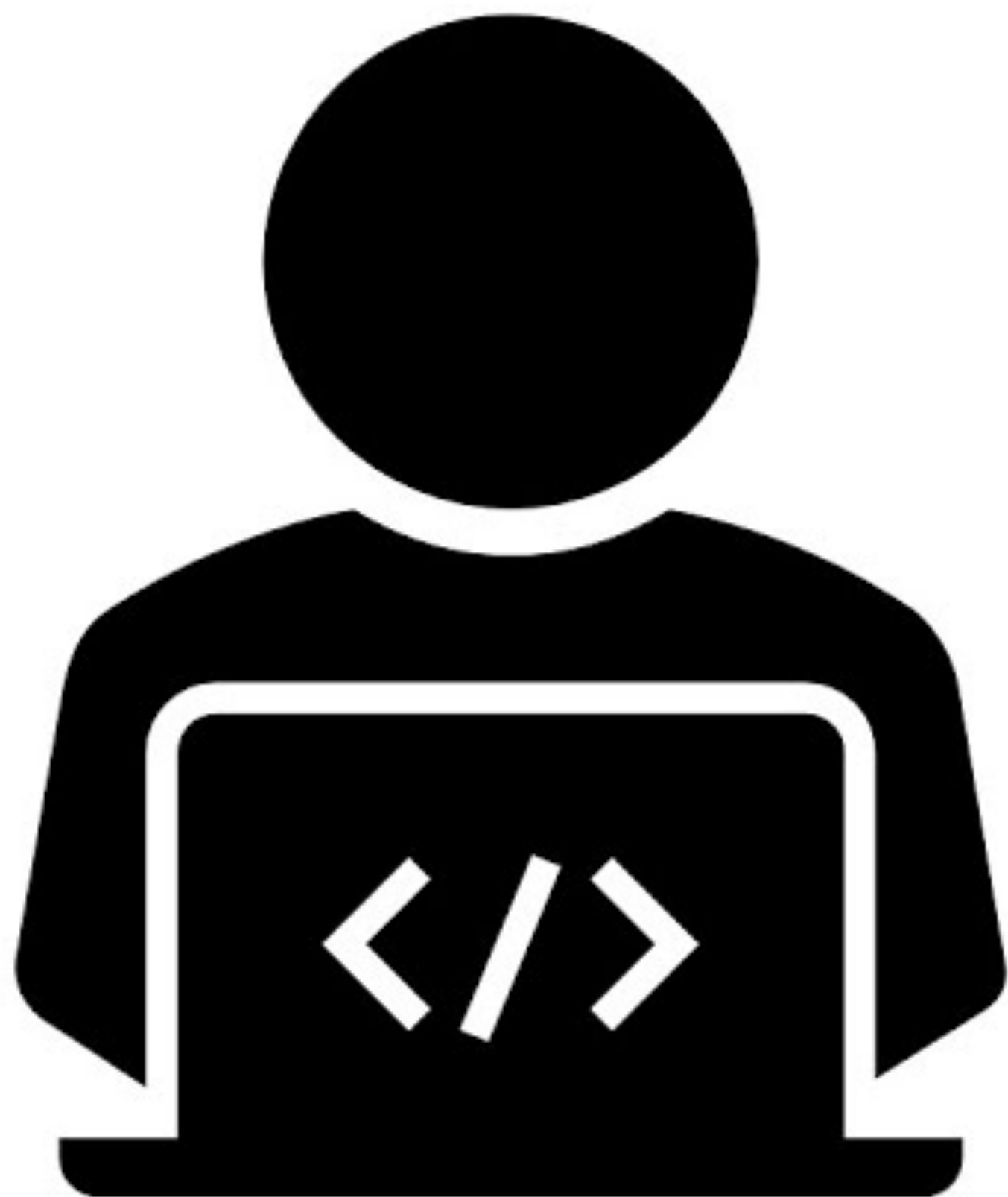
Internally, it will automatically convert to the corresponding **Ising Hamiltonian** for solving, and the final solution will then be mapped back to the original variable space.

```
QUBO > (-1, 1)to(0, 1).py > ...
1  import pyqubo
2  import neal
3
4  # 建立三個變數，名稱為 'x[0]', 'x[1]', 'x[2]'，使用 SPIN 形式 (值為 -1 或 +1)
5  x = pyqubo.Array.create('x', shape=(3,), vartype='SPIN') # 'SPIN' 表示變數取值為 {-1, +1}
6
7  # 定義目標函數：希望讓 (x[0] + 5 * x[1] + 6 * x[2]) 越接近 0 越好
8  objective_function = (1 * x[0] + 5 * x[1] + 6 * x[2]) ** 2
9
10 # 將目標函數編譯為 pyqubo 模型
11 model = objective_function.compile()
12
13 # 將模型轉換為 BQM (Binary Quadratic Model)
14 bqm = model.to_bqm()
15
16 print("我們可以將 bqm 轉換為 Ising 或 QUBO 輸出")
17 print(bqm.to_ising()) # 這會顯示 Ising 模型的 h、J 係數與常數項
18
19 # 使用模擬退火進行求解
20 sa = neal.SimulatedAnnealingSampler()
21 sampleset = sa.sample(bqm, num_reads=10)
22
23 # 解碼樣本為原始變數名稱格式
24 samples = model.decode_sampleset(sampleset)
25
26 # 找出能量最小 (最佳) 解
27 best_sample = min(samples, key=lambda s: s.energy)
28
29 print("求解時，pyqubo 內部已經將 Ising 模型轉換為 QUBO 的 0 或 1，所以輸出結果為 0 或 1")
30 print(best_sample.sample)
```

Chapter 4



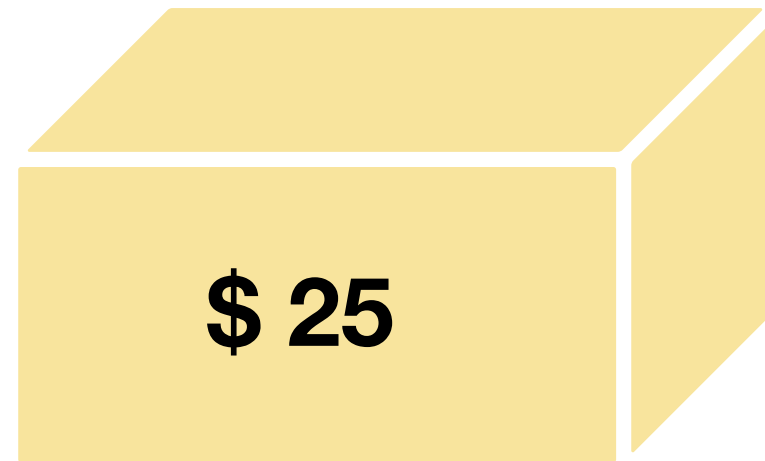
QUBO 建模



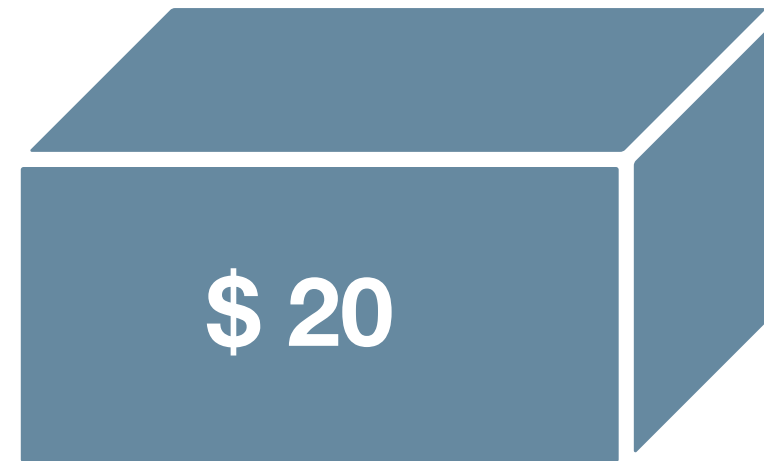
From QUBO to QPU

QUBO Modeling

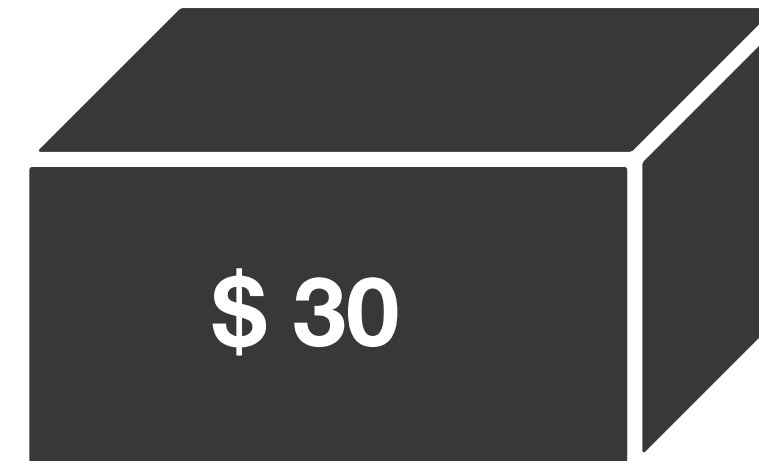
$x_i = 1$: BUY
 $x_i = 0$: NOT



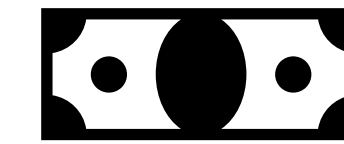
x_1



x_2



x_3



\$ 50

How to buy the maximum number of boxes within a budget?

Object function : $\min = 50 - 25x_1 - 20x_2 - 30x_3$

If no boxes are bought: $f(x) = 50$

If all boxes are bought: $f(x) = -25$

$$\min = (50 - 25x_1 - 20x_2 - 30x_3)^2$$

$$-1875x_1 - 1600x_2 - 2100x_3 + 1000x_1x_2 + 1500x_1x_3 + 1200x_2x_3$$

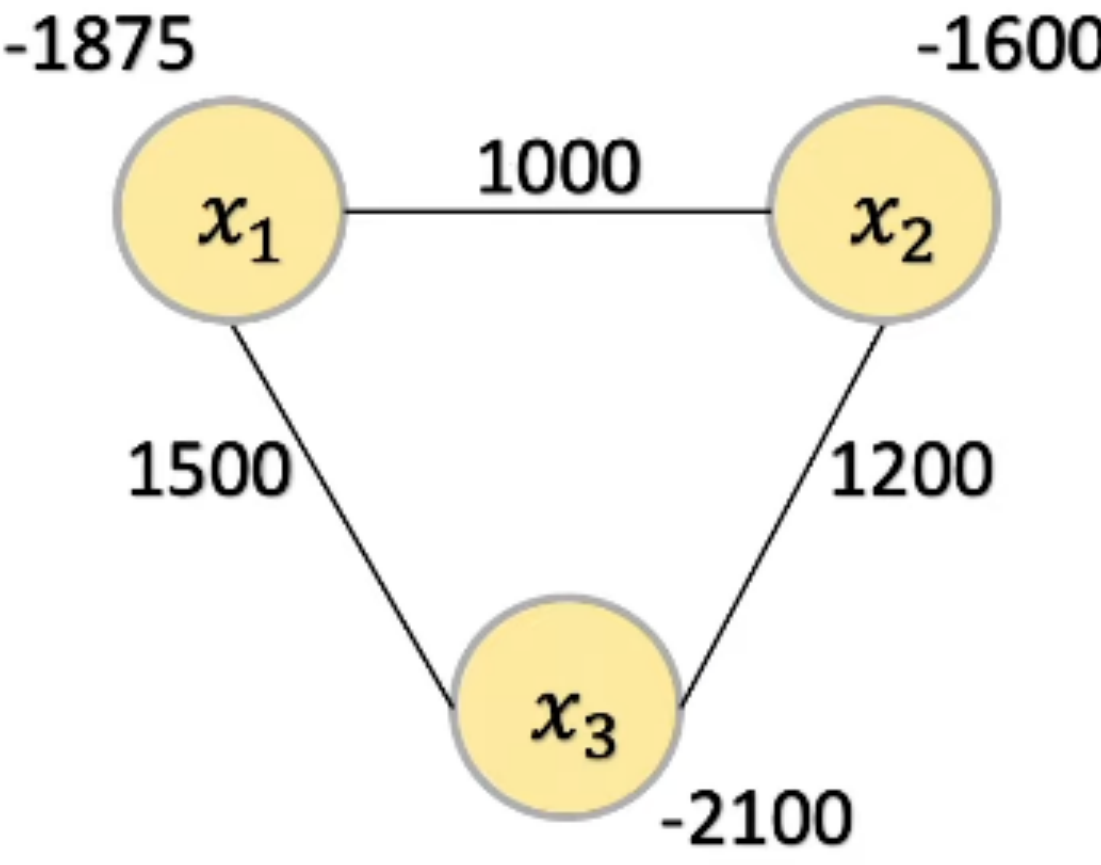
$$\min_{x \in \{0,1\}^n} \sum Q_{ii}x_i + \sum Q_{ij}x_ix_j \Leftrightarrow \min_{\mathbf{x} \in \{0,1\}^n} \mathbf{x}^T Q \mathbf{x}$$

$$\min_{\mathbf{x} \in \{0,1\}^3} [x_1 \ x_2 \ x_3] \begin{bmatrix} -1875 & 1000 & 1500 \\ 0 & -1600 & 1200 \\ 0 & 0 & -2100 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

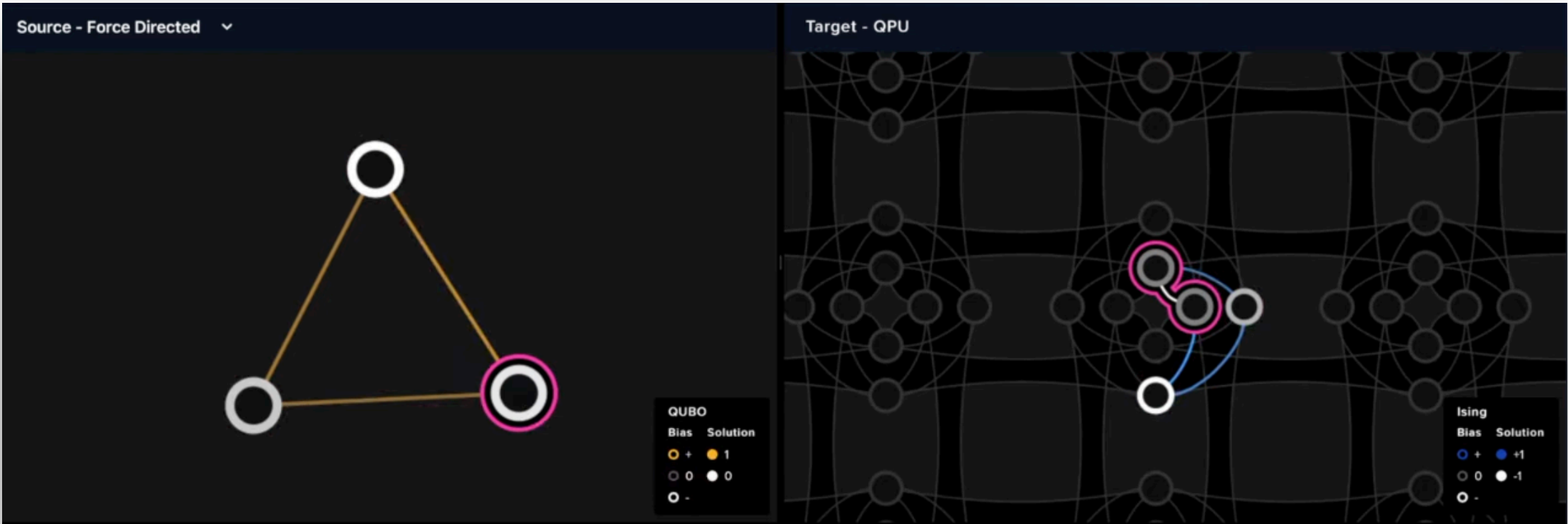
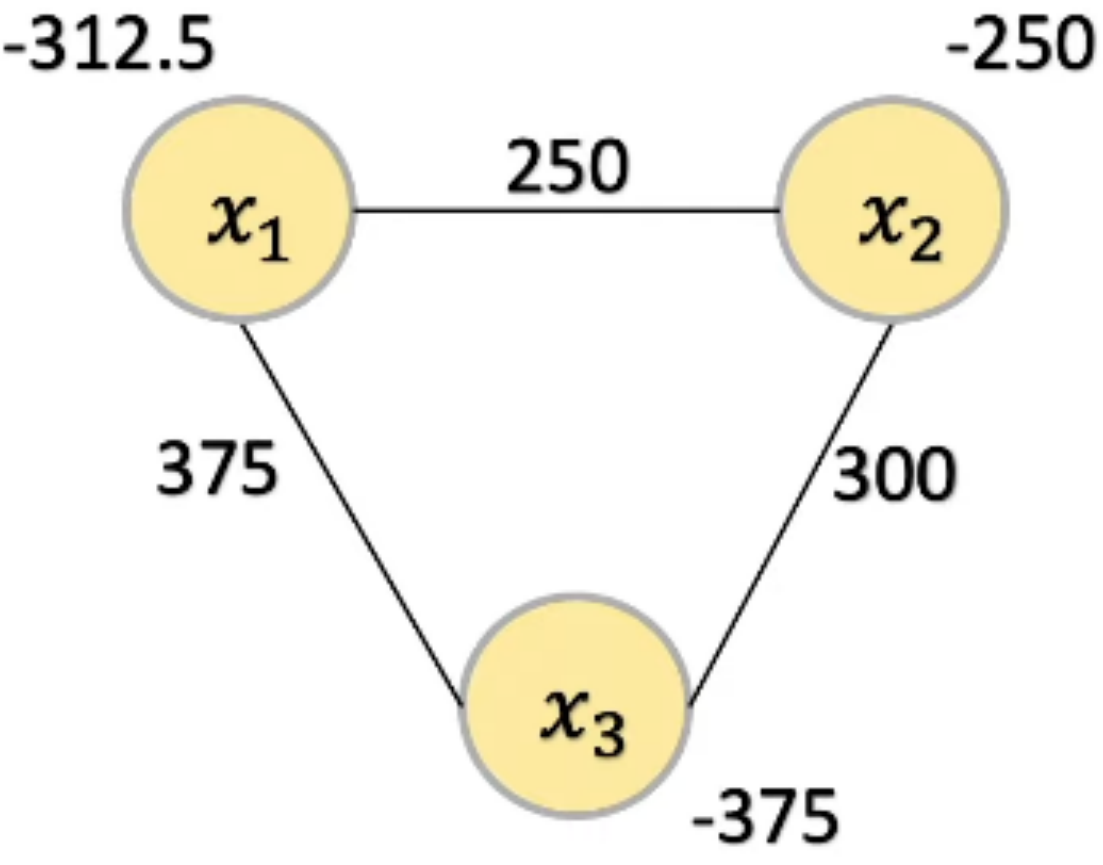
$$Q = \begin{bmatrix} -1875 & 1000 & 1500 \\ 0 & -1600 & 1200 \\ 0 & 0 & -2100 \end{bmatrix}$$

QUBO to QPU

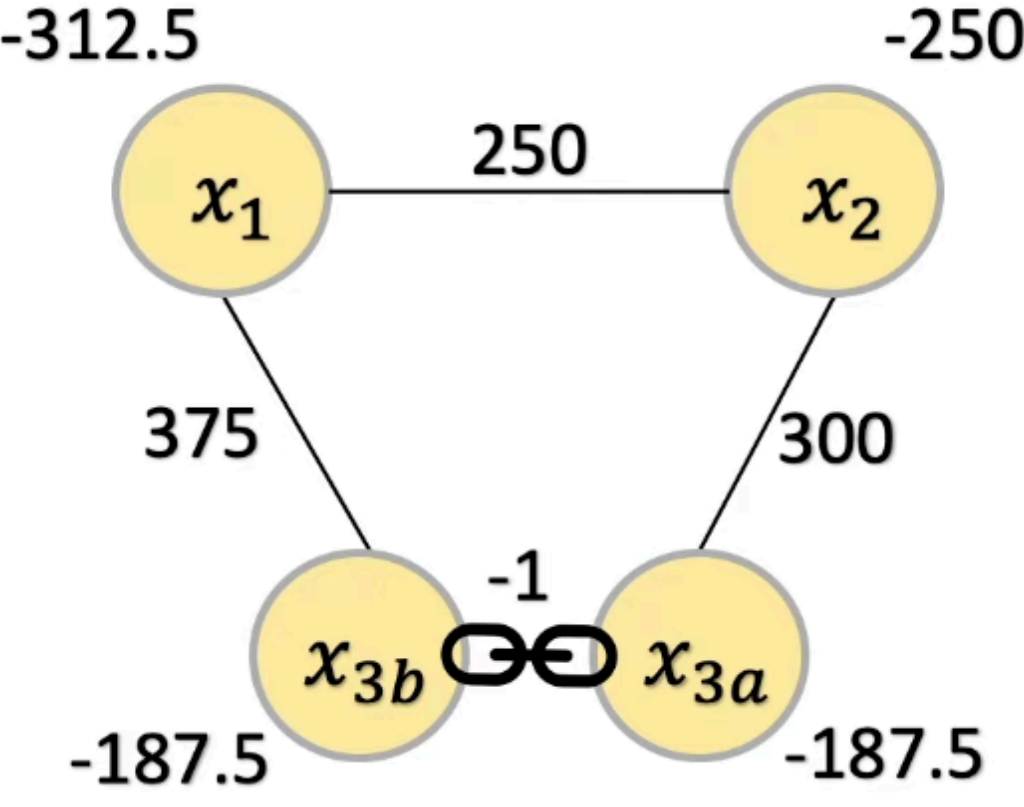
QUBO



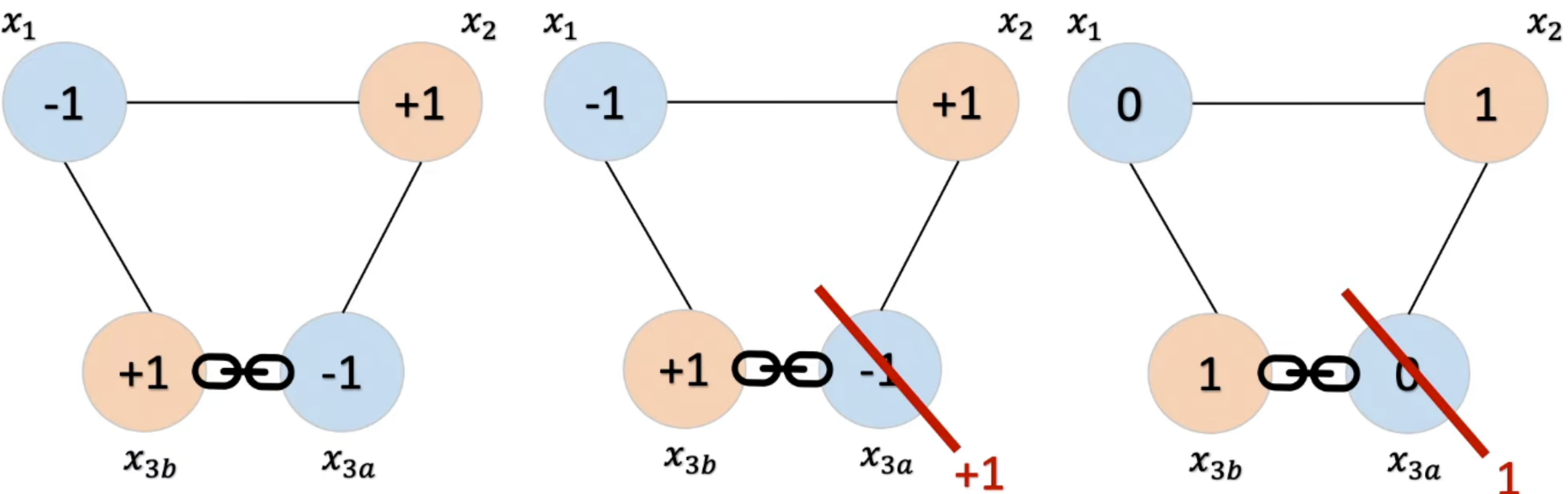
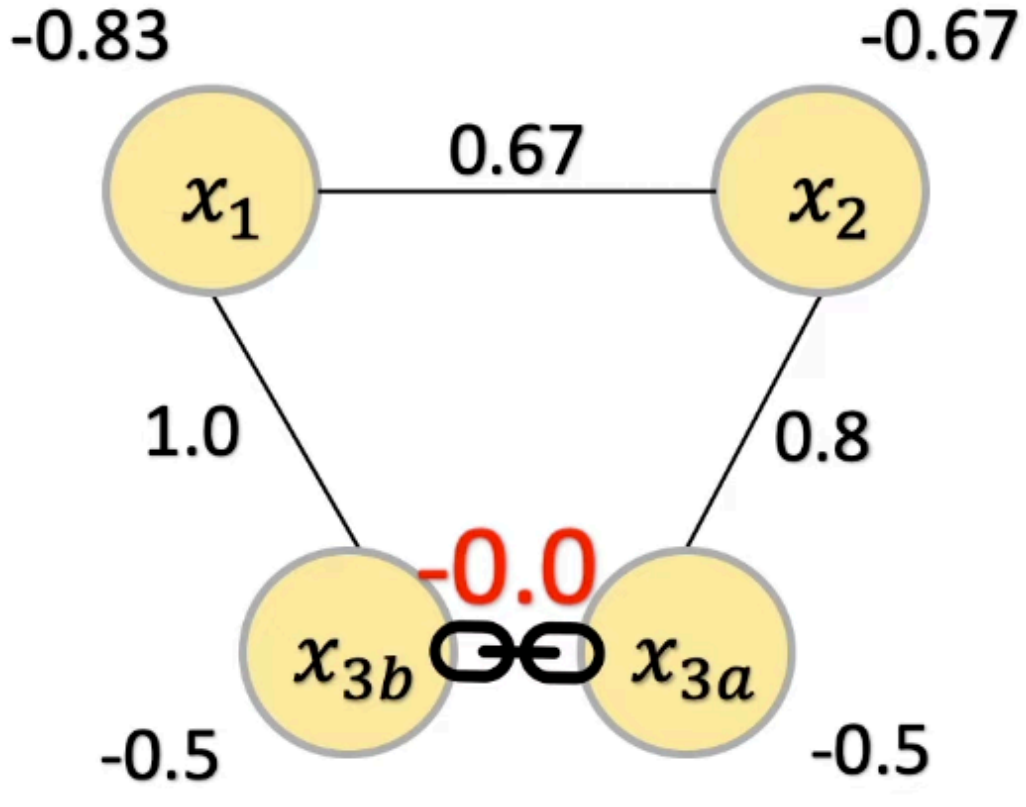
Ising



Embedded Ising



Scaled and Embedded Ising



Qubit weights : -2 to +2 , Edge weights : -1 to 1

$x_{3a} \neq x_{3b} \rightarrow$ chain break $\rightarrow$ adjust chain strength

QUBO Models and Penalty Techniques

No Penalty (Objective Term Only)

$$\min_{\mathbf{x} \in \{0,1\}^3} [x_1 \ x_2 \ x_3] \begin{bmatrix} -1875 & 1000 & 1500 \\ 0 & -1600 & 1200 \\ 0 & 0 & -2100 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} \quad x_i \in \{0,1\} \Rightarrow x_i^2 = x_i$$

$$\rightarrow \min (-1875 x_1^2 + 1000 x_1 x_2 + 1500 x_1 x_3 - 1600 x_2^2 + 1200 x_2 x_3 - 2100 x_3^2)$$

$$\rightarrow \min (-1875 x_1 + 1000 x_1 x_2 + 1500 x_1 x_3 - 1600 x_2 + 1200 x_2 x_3 - 2100 x_3)$$

Linear Terms Interaction Terms Interaction Terms Linear Terms Interaction Terms Linear Terms

$$= \min_{\mathbf{x} \in \{0,1\}^n} \sum_i \textcolor{red}{Q_{ii} x_i} + \sum_{i < j} \textcolor{blue}{Q_{ij} x_i x_j}$$

Constraint Type	QUBO Penalty Expression
Pick One: $\sum x_i = 1$	$P \cdot (\sum x_i - 1)^2$
At Most One: $\sum x_i \leq 1$	$P \cdot \sum_{i < j} x_i x_j$
Equality: $x_i = x_j$	$P \cdot (x_i - x_j)^2$
Opposite: $x_i = 1 - x_j$	$P \cdot (x_i + x_j - 1)^2$
Capacity: $\sum w_i x_i \leq C$	$P \cdot (\max(0, \sum w_i x_i - C))^2$

Penalty Techniques

$$H_P = H_A + H_B$$

H_A : Penalty term

H_B : Objective term

Penalty Example: "At Most One Box"

$$x_1 + x_2 + x_3 \leq 1 \rightarrow H_A = P \cdot (x_1 x_2 + x_1 x_3 + x_2 x_3), \text{ penalty strength } P = 5000$$

$$\rightarrow \min H_P = H_B + P(x_1 x_2 + x_1 x_3 + x_2 x_3)$$

$$\rightarrow H_P = (-1875)x_1 + (-1600)x_2 + (-2100)x_3 + (1000 + 5000)x_1 x_2 + (1500 + 5000)x_1 x_3 + (1200 + 5000)x_2 x_3$$

$$\rightarrow \textcolor{red}{H_P = -1875x_1 - 1600x_2 - 2100x_3 + 6000x_1x_2 + 6500x_1x_3 + 6200x_2x_3}$$

Chapter 5

Simulated Quantum Annealing

Starting from Classical Annealing

Simulated Annealing (SA) mimics the physical cooling process, finding the minimum energy solution through stochastic jumps.

Simulated Quantum Annealing (SQA) incorporates the concept of "Quantum Tunneling" to traverse energy barriers.

Real-world QA: Requires physical implementation on **quantum computers**, such as D-Wave.

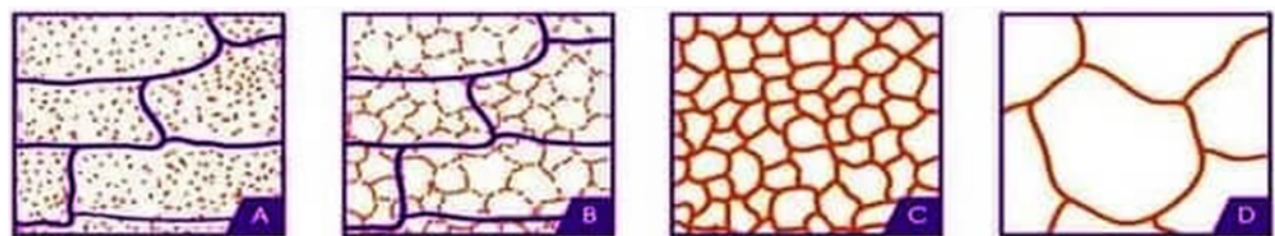
SQA Definition: SQA is an algorithm that simulates the behavior of Quantum Annealing on **classical computers**.

Key Technology: Path-Integral Monte Carlo (PIMC).

Primary Goal: To solve Quadratic Unconstrained Binary Optimization (QUBO) problems.

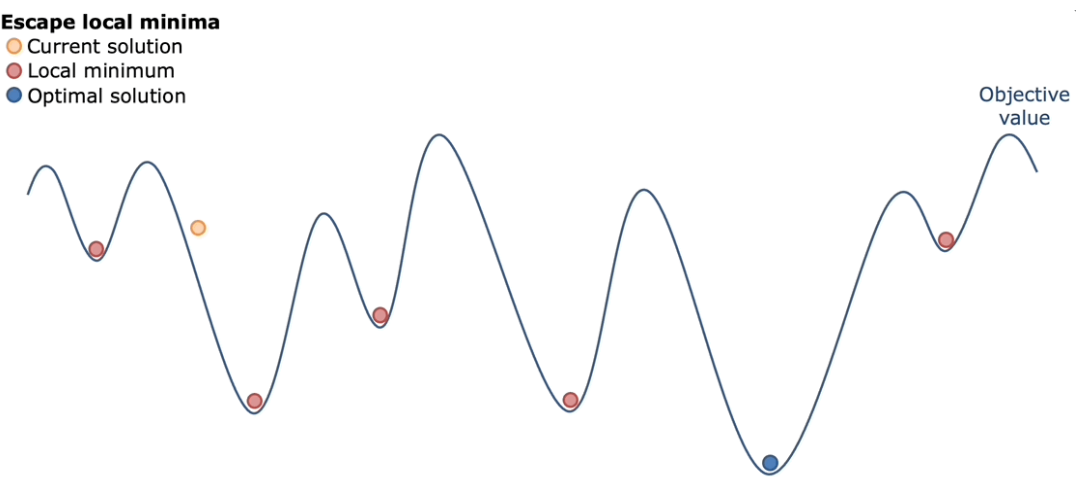
Physics

Annealing



Traditional algorithm

Classical
Computer

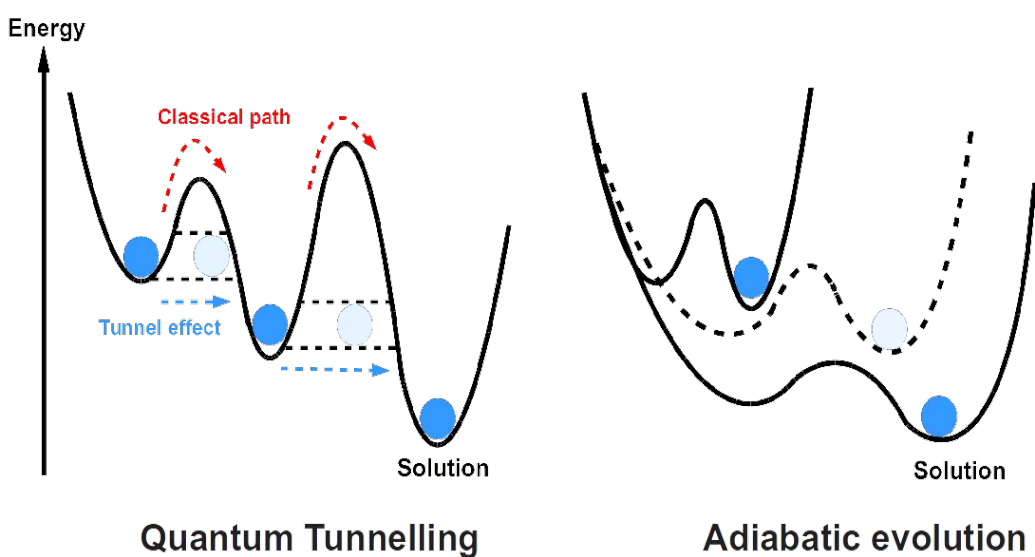


Simulated
Annealing

Quantum Computing

Quantum
Computer

Quantum Annealing

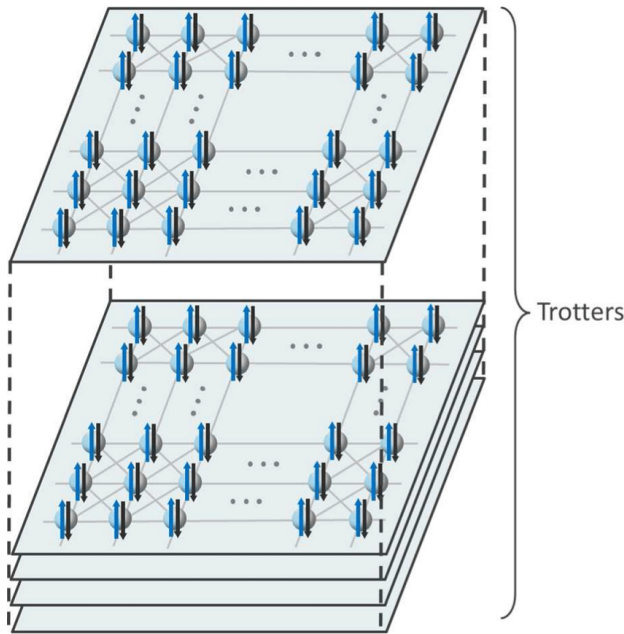


Quantum-Inspired Heuristics

Classical
Computer

High computational
resource requirements

Simulated Quantum Annealing



SQA Core Model: Time Slicing + PIMC

Classical Bit → Quantum Bit (Qubit)

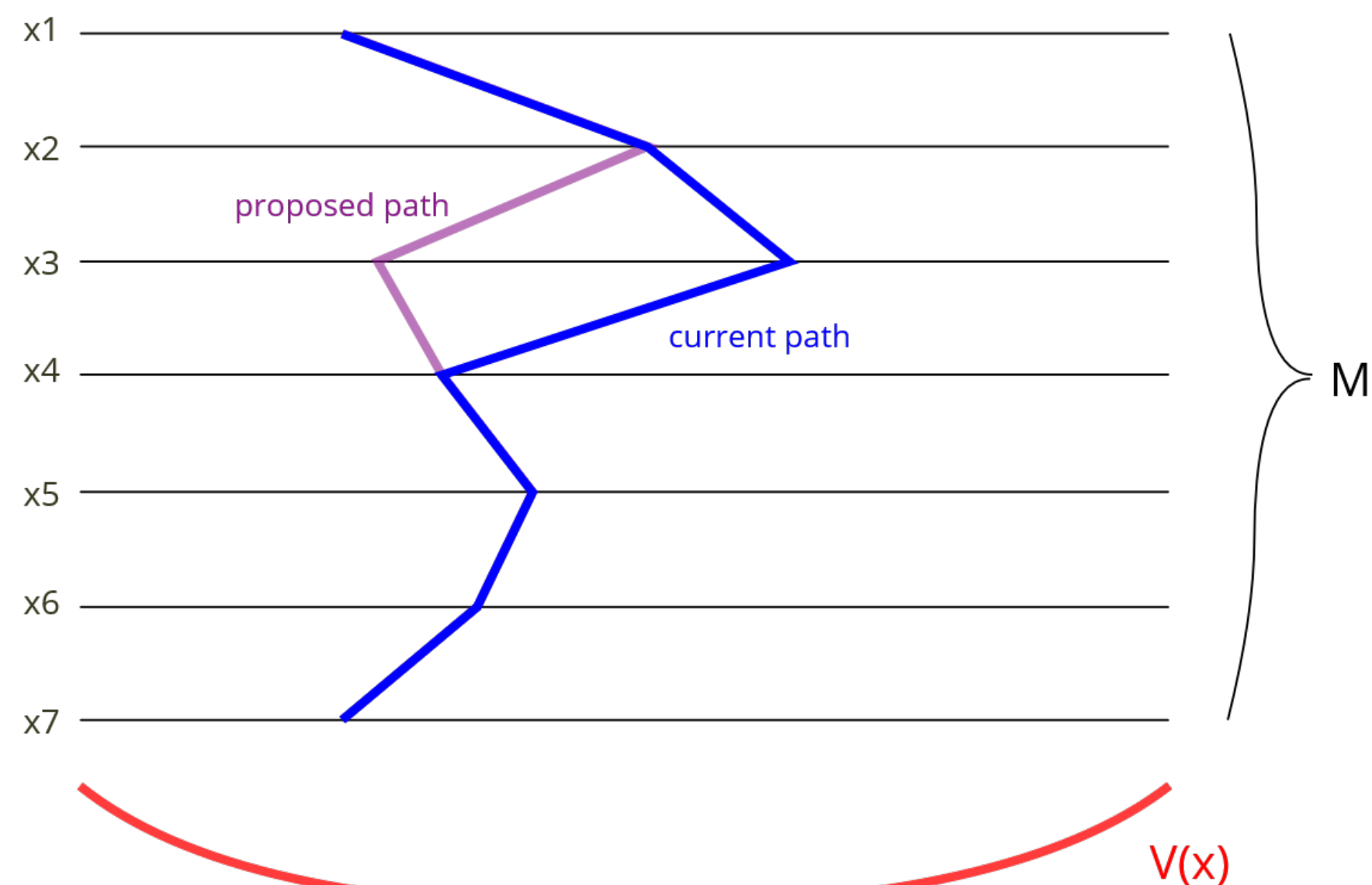
In a **classical computer**, a variable (bit) can only take one value at a time (e.g. $x_i = \pm 1$), meaning we can only observe one state at a time.

In a **quantum system**, a qubit does not just exist in a single state; it can oscillate across multiple states simultaneously (superposition).

Q: How do we simulate this "unstable jumping state" within a "classical computer"? ? **A: The Answer: Time Slices.**

Path Integral Monte Carlo (PIMC)

$$H_{\text{eff}} = \sum_{m=1}^M \left[\frac{(x_{m-1} - x_m)^2}{4\lambda\beta^2/M} + \frac{1}{M} V(x_m) \right]$$



M : Number of **Time Slices** (Imaginary time segments)

$V(x)$: External potential energy curve

Each x_m corresponds to an **imaginary time**

$\tau_m = m \times (\beta / M)$

Acceptance Probability: $\exp[-\beta (H'_{\text{eff}} - H_{\text{eff}})]$

Spring Term: Represents kinetic energy; describes the continuous coupling between time slices.

Potential term : Reflects the influence of the external potential field on the particle.

Current path : The particle's current "**Imaginary Time Path.**"

Proposed path : A newly generated random path for trial updates.

VS. Quantum Annealing: $H(t) = A(t)H_0 + B(t)H_P$

SQA Core Model: Time Slicing + PIMC

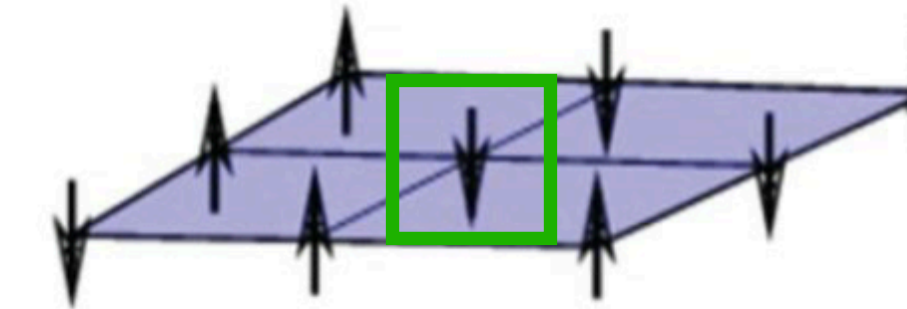
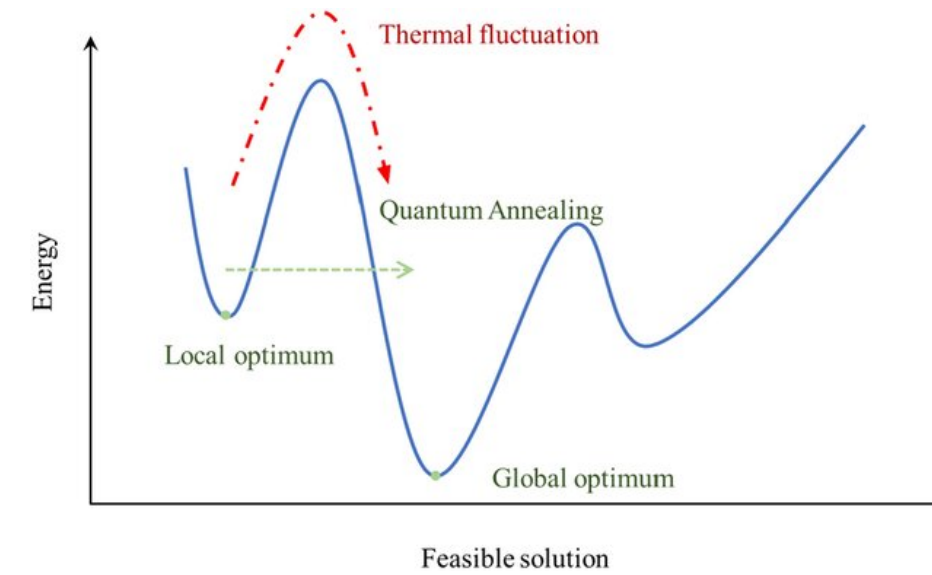
A Classical SA (Simulated Annealing)

Spin lattice structures are arranged on a plane. (2D lattice)

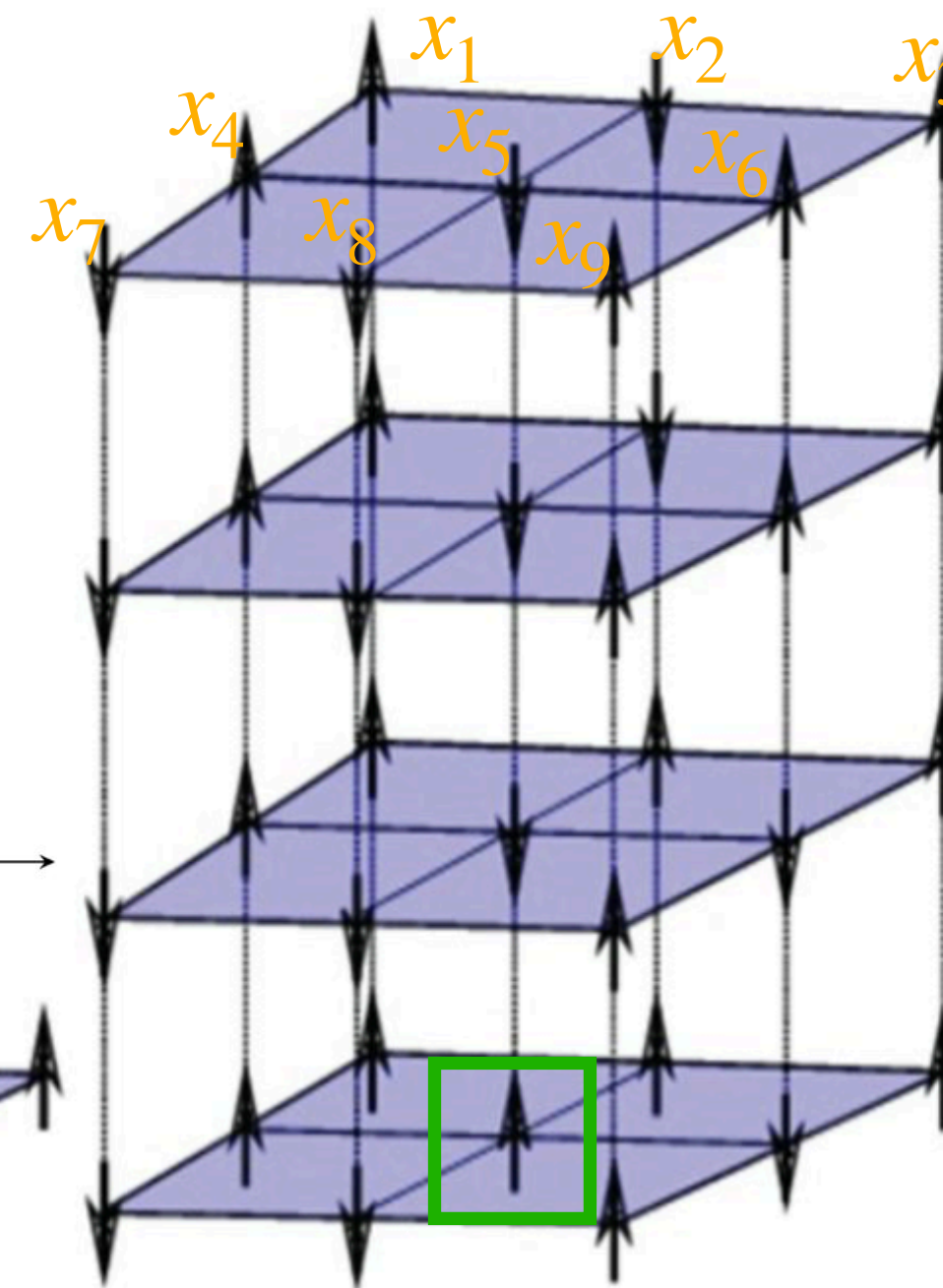
Each node represents a variable $x_i \in \{-1, +1\}$

Simulated Annealing considers only a "single configuration" for energy and approaches the minimum value by gradually lowering the temperature

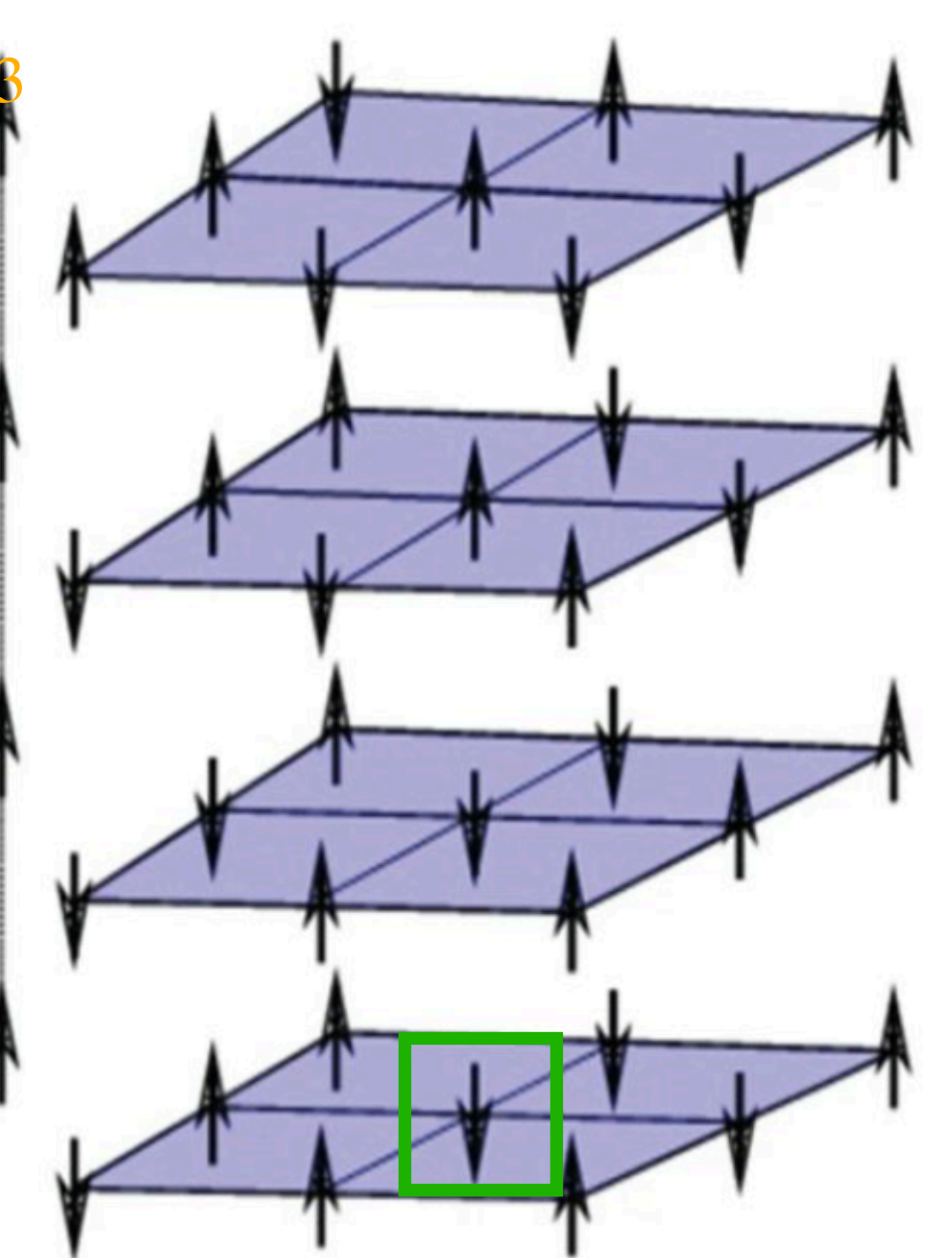
It does not use time slices and does not simulate quantum behavior.



A



B



C

B 所有切片都是在相同真實時間下的解群
真實時間指的是退火參數如 $A(t)$

B Discrete-Time SQA (DT-SQA)

Note: This refers to the **SQA** we have been discussing; it is specifically labeled as **Discrete-Time** because not all forms of SQA utilize **time slices**.

Using the SA plane lattice as a foundation, M slices are stacked in the "Imaginary Time Direction."

Each variable x_i is expanded into $x_i^1, x_i^2, \dots, x_i^M$

Vertical lines between slices represent the **imaginary time coupling term**, simulating the tunneling effect caused by a transverse field.

VS. Quantum Annealing: $H(t) = A(t)H_0 + B(t)H_P$

C Parallel SA on M replicas (Multi-replica Classical Simulation)

Each replica consists of an independent 2D SA lattice as described in Section A.

There is no imaginary time coupling; each replica operates as a standalone calculation.

Although it allows for **multiple parallel SA simulations**, it **cannot** simulate quantum tunneling behavior.

Outcomes are heavily dependent on initial conditions and stochastic randomness.

Can SQA Run Without Using Virtual (Imaginary) Time?

2-2 CMOS annealing technology developed

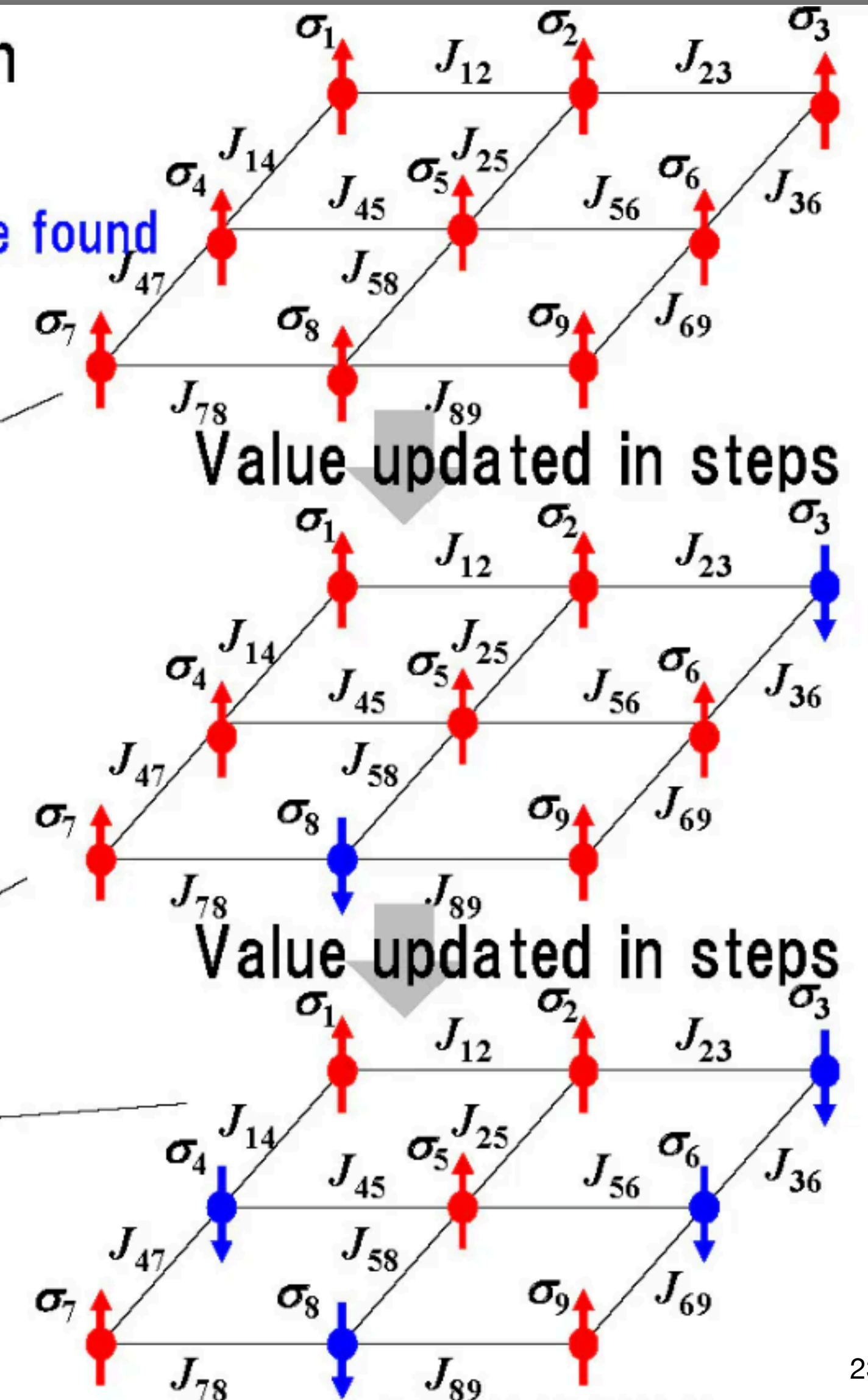
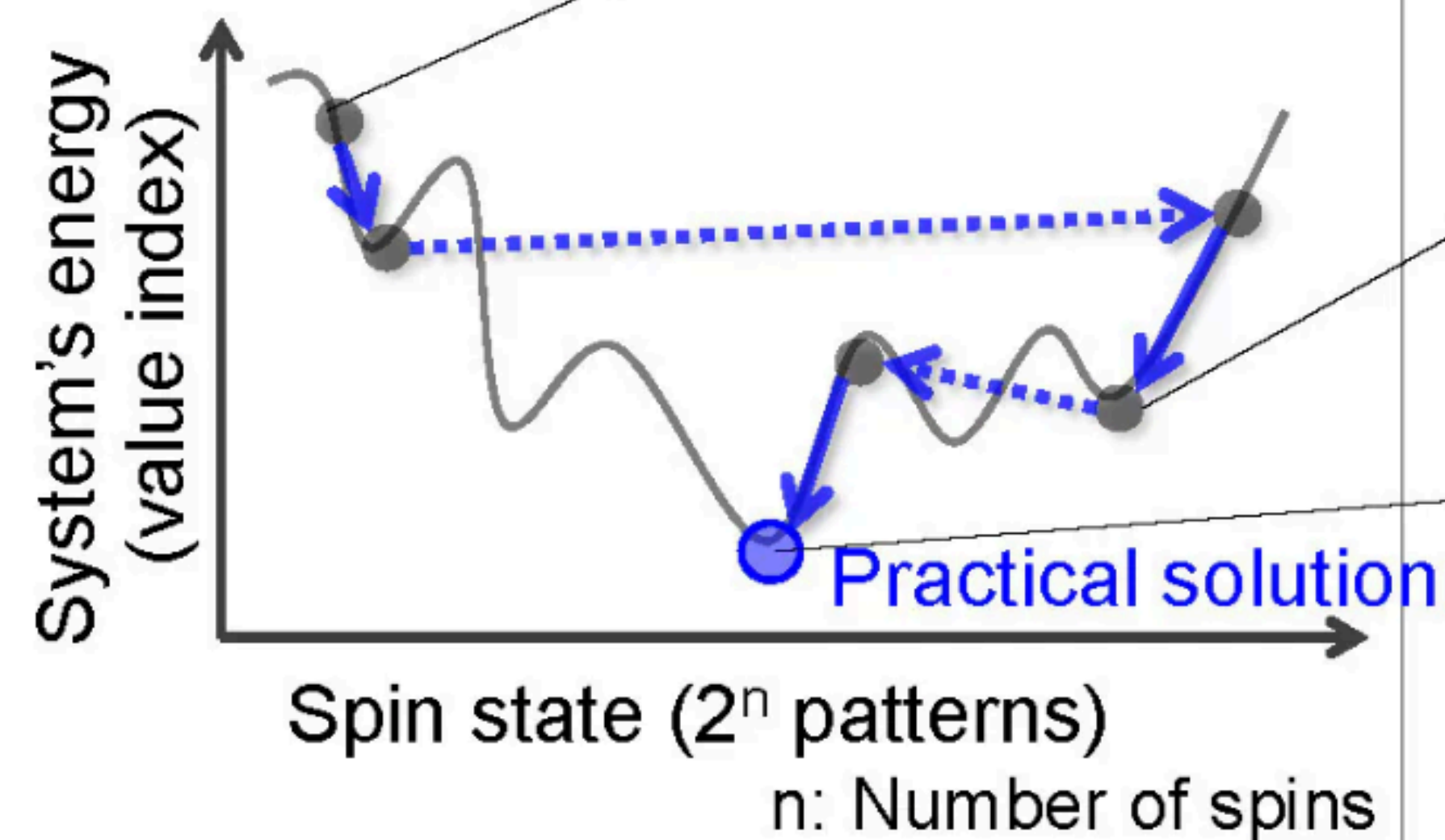
HITACHI
Inspire the Next

Spin value patterns change at every spin
(1/8 of all spins are updated each time)

- No guarantee that the optimal solution would be found

CMOS annealing

- Shifts to states of lower energy (through digital circuits) →
- Randomization to avert any trapping into local minimum points→
- Practical solution



Chapter 6

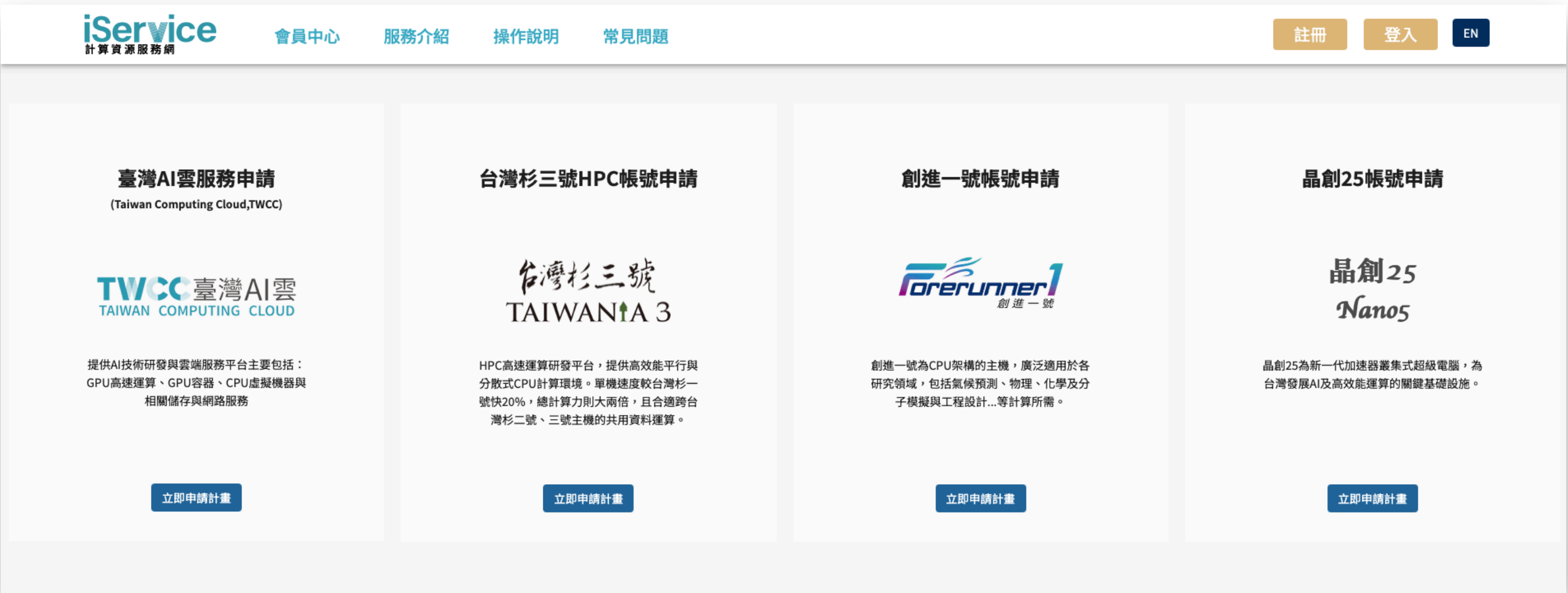
NCHC Resources (Taiwania 3)

What is Taiwania 3?

You must first register a **personal account** and a **host name** on **iService**.
After applying for various **supercomputing accounts**, link them with your personal host name to enable access.

台灣杉三號(Taiwania 3)為國內提供開放服務申請的最大CPU高速計算主機(2021年)，擁有900個計算節點。

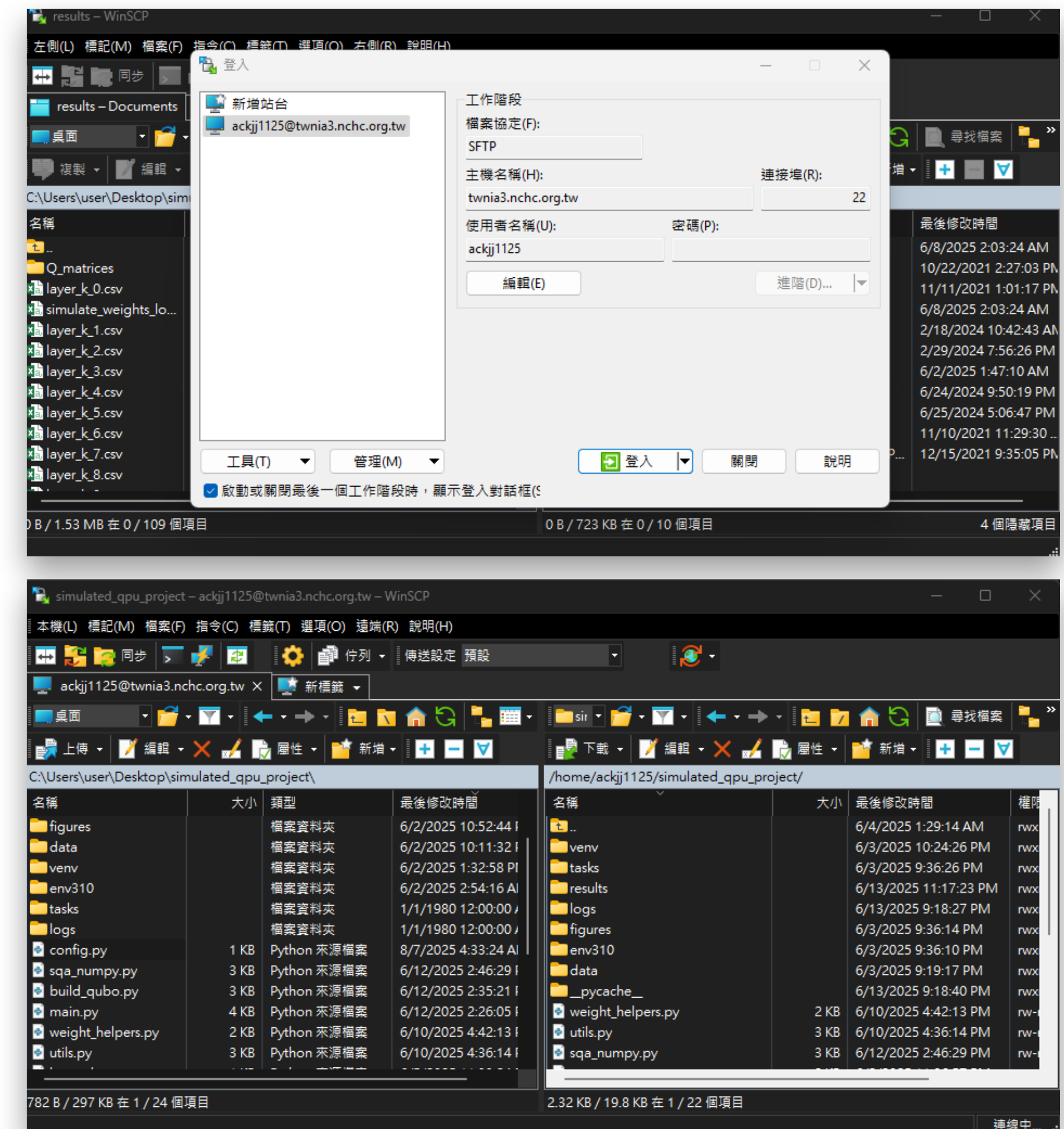
簡介	規格表
系統環境規格	
計算節點	2顆 Intel® Xeon® Platinum 8280 2.4GHz CPU (28 Cores/CPU) 與 192GB 主記憶體
網路界面	每個節點具1張 Mellanox InfiniBand HDR100 網路卡，同時提供MPI平行計算與平行檔案存取使用
作業系統	CentOS x86_64 7.8
編譯器	Intel Parallel Studio XE 2020、Nvidia HPC SDK v20.11 (PGI Compilers)
排程系統	Slurm Workload Manager
連線資訊	twnia3.nchc.org.tw



Specifically, how do we use Taiwania 3?

- Linux-based; requires familiarity with common Linux commands such as ls, cd, vim / nano, and scp.
- **Core Concepts:** Batch jobs, resource scheduling (**Slurm**), and **Environment Modules**.

1. SSH Login:
`ssh username@twnia3.nchc.org.tw`
2. Check Available Nodes and Partitions:
`sinfo`
3. Load Environment Modules:
`module avail` # List all available modules
`module load python/3.9` # e.g. : Load Python 3.9
4. Upload Packages/Files:
Recommended software: WinSCP
5. Verify Python Environment or Packages:
`python --version`
`pip list`
6. Write Slurm Batch Script
7. Submit and Monitor
8. Download Results/Packages



Specifically, how do we use Taiwania 3?

```
#!/bin/bash
#SBATCH -J sqa_main
Sets the Job Name to "sqa_main"
#SBATCH -o logs/out_%j.txt
#SBATCH -e logs/err_%j.txt
Directs standard output and standard error to separate
files in the "logs/" directory (%j represents the Job ID).
#SBATCH -p ctest
Submits to the "ctest" queue (nodes for testing). For long-
term or official production runs, change to partitions such
as cpu, gpu, main, dev, or cfast.
#SBATCH -N 21
#SBATCH --ntasks=1000
# Requests 21 nodes and 1,000 tasks (CPUs).
```

```
#SBATCH --time=02:00:00
# Limits job execution to a maximum of 2 hours; the
system automatically terminates jobs that exceed this time.
#SBATCH --account=mst113433
Specifies the project account (mandatory; otherwise, the
submission will be rejected).
source /etc/profile.d/modules.sh
module load rcec
module load rcec/stack-gnu-mpi
module load python/3.10.13
Initializes the module system and loads the Python
execution environment.
mkdir -p logs
Ensures the "logs/" directory exists.
/home/ackjj1125/.conda/envs/sqa_env/bin/python main.py
```

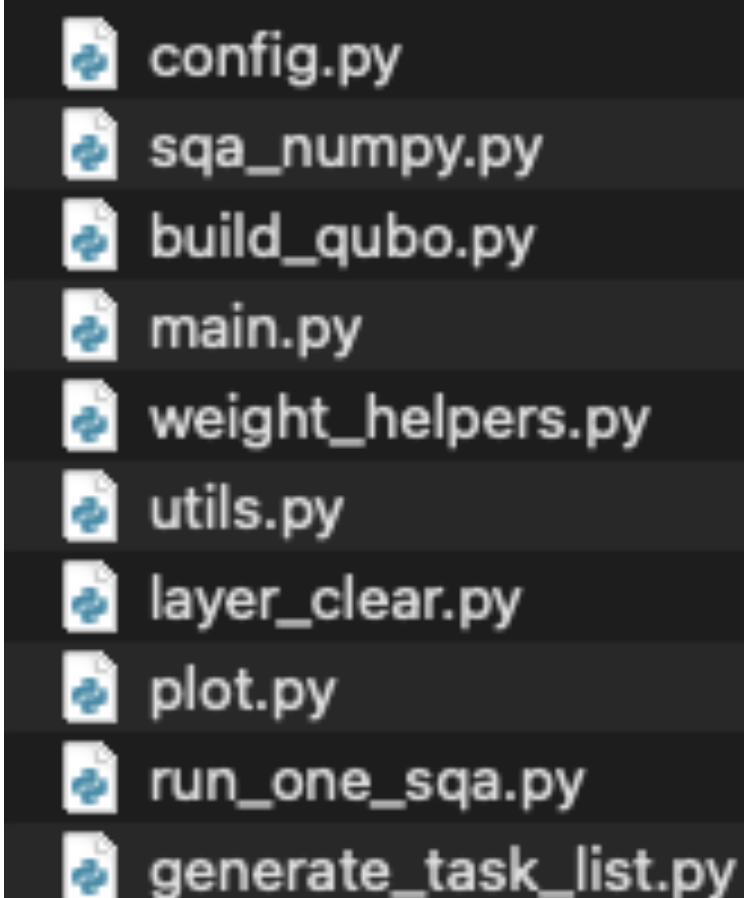
```
(base) [ackjj1125@lgn301 ~]$ cd simulated_qpu_project
(base) [ackjj1125@lgn301 simulated_qpu_project]$ sbatch run_main.slurm
sbatch: User ackjj1125 Account mst113433 wallet has 114138.26 remaining.
Submitted batch job 16116419
(base) [ackjj1125@lgn301 simulated_qpu_project]$ squeue -u ackjj1125
  JOBID PARTITION   NAME   USER  ST       TIME  NODES NODELIST(REASON)
  16116419      ctest sqa_main ackjj112  R        0:19     21 cpn[3697-3717]
(base) [ackjj1125@lgn301 simulated_qpu_project]$ squeue -u ackjj1125
  JOBID PARTITION   NAME   USER  ST       TIME  NODES NODELIST(REASON)
  16116419      ctest sqa_main ackjj112  R        0:59     21 cpn[3697-3717]
(base) [ackjj1125@lgn301 simulated_qpu_project]$ scancel 16116419
(base) [ackjj1125@lgn301 simulated_qpu_project]$ squeue -u ackjj1125
  JOBID PARTITION   NAME   USER  ST       TIME  NODES NODELIST(REASON)
(base) [ackjj1125@lgn301 simulated_qpu_project]$
```

Will My Original Programs Need Adjustments for Supercomputing?

1. The Need for Distributed Computing

Originally, all algorithms were implemented in a single `main.py` or `sqa_numpy.py`, where one execution completed the entire workflow.

On supercomputing platforms such as Taiwania-3, large-scale QUBO or SQA problems require explicitly utilizing hundreds to thousands of CPU cores or multiple GPUs to achieve reasonable runtimes.



- `config.py`
- `sqa_numpy.py`
- `build_qubo.py`
- `main.py`
- `weight_helpers.py`
- `utils.py`
- `layer_clear.py`
- `plot.py`
- `run_one_sqa.py`
- `generate_task_list.py`

2. Map-Reduce Workflow

All parameter combinations or independent QUBO instances are decomposed into a task list using `generate_task_list.py`.

Each task independently runs `run_one_sqa.py` with a specific parameter set to solve one QUBO instance.

3. Recommended Modular Program Structure

config.py

Centralized management of all hyperparameters.

build_qubo.py

Constructs QUBO matrices for problems such as Knapsack, TSP, and N-Queens.

run_one_sqa.py

Executes SA or SQA for a single parameter configuration and outputs results to JSON or CSV.

generate_task_list.py

Converts all parameter combinations into a task list.

plot.py

Visualizes and compares algorithm performance across different hyperparameter settings.

utils.py

Common utilities for I/O, logging, and Slurm job script generation.



Integrated Multi-objective Optimization and Spatial Analysis for the Site Selection of Levee Crossing Facilities

Tsun-Yuan Ku¹*, Febrian Fitryanik Susanta¹, Pei-Fun Kuo¹, Chung-Wei Shen², Ching-Jui Lai³, Nan-You Chen⁴

1. Department of Geomatics, National Cheng Kung University, Tainan, Taiwan *Email: ackjj1125@gmail.com
2. Department of Transportation and Communication Management Science, National Cheng Kung University, Tainan, Taiwan
3. Department of Mathematics, National Cheng Kung University, Tainan, Taiwan
4. National Center for High-Performance Computing, Taiwan

informa
ANNUAL
MEETING

```
def build_qubo_matrix(p_mat, s_mat, d_mat, cost_vec, lam, w1=1.0, w2=1.0, w3=1.0, w4=1.0):
```

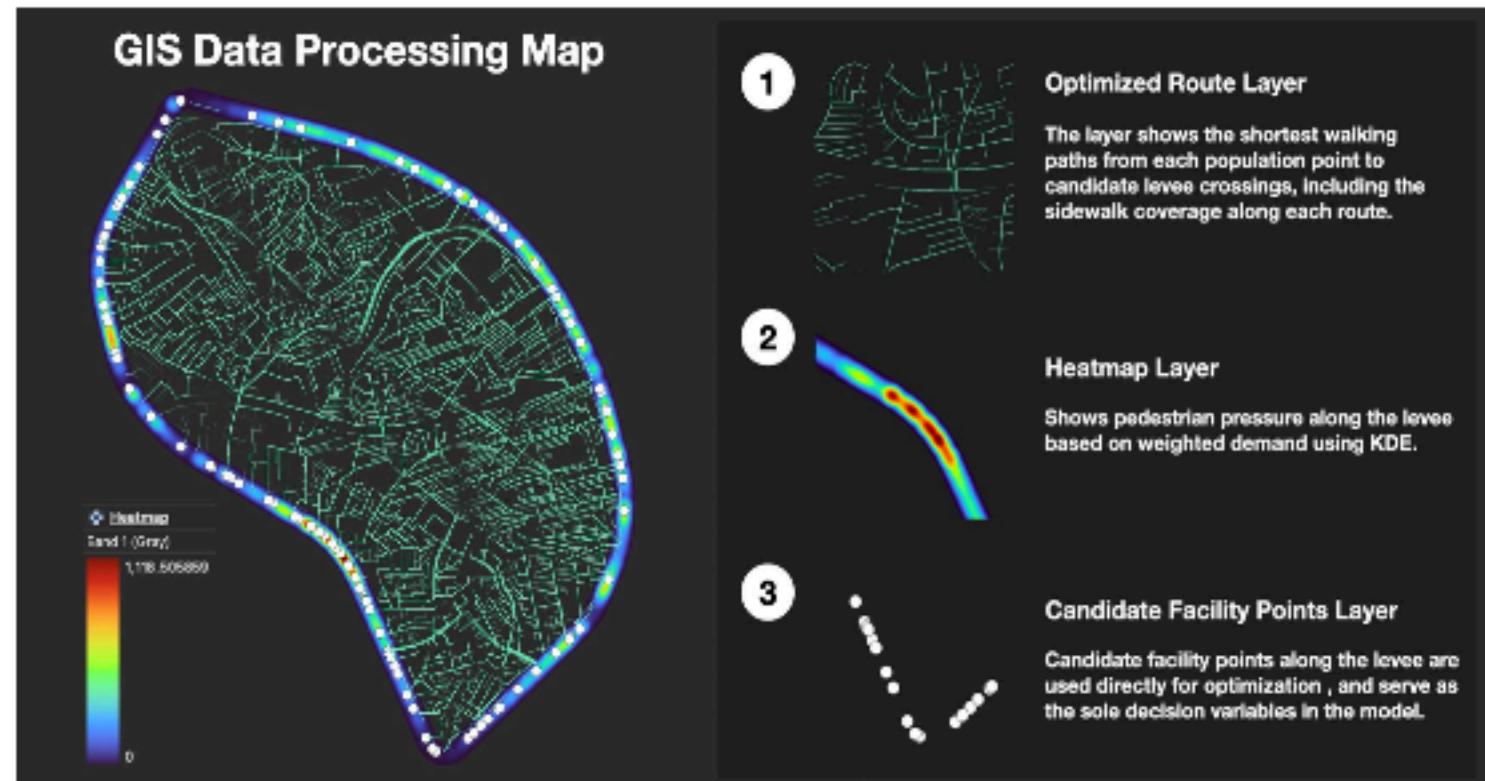
```
    N = p_mat.shape[0]
    print(f"\n[構建 QUBO] N={N} - 使用權重 w1={w1}, w2={w2}, w3={w3}, w4={w4}")
```

```
    Q1 = -np.dot(p_mat, p_mat.T)
    Q2 = np.dot(s_mat, s_mat.T)
    Q3 = np.diag(cost_vec)
    mask = ~np.eye(N, dtype=bool)
    Q4 = np.zeros((N,N))
    Q4[mask] = np.exp(-lam * d_mat[mask])
```

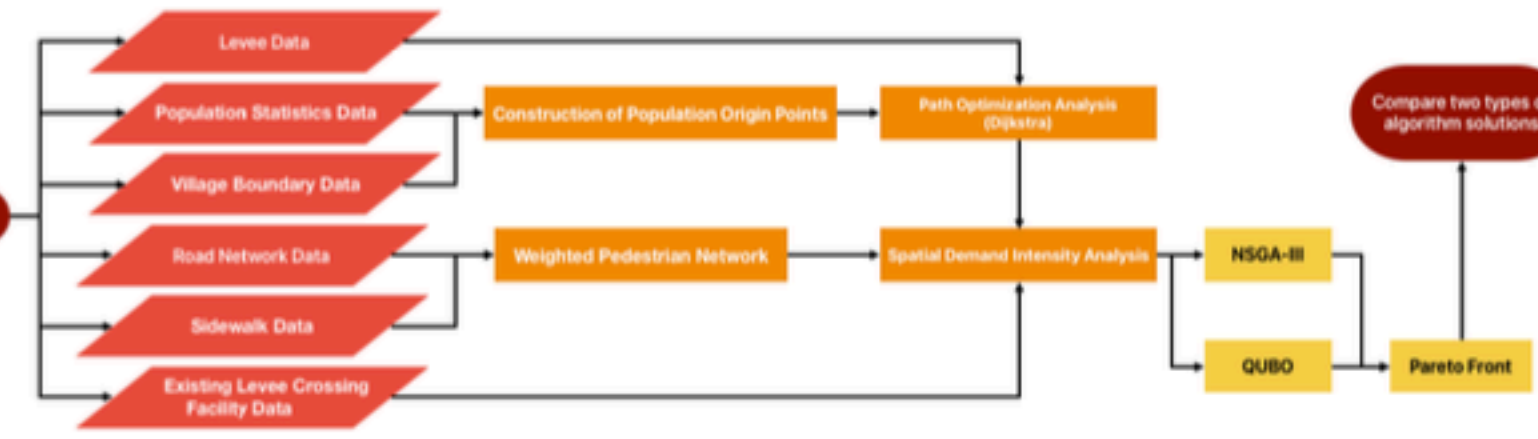
```
    print(" ||Q1||", np.linalg.norm(Q1),
          "||Q2||", np.linalg.norm(Q2),
          "||Q3||", np.linalg.norm(Q3),
          "||Q4||", np.linalg.norm(Q4))
```

```
# === f1: maximize demand (negative term) ===
Q = w1 * Q1
# === f2: sidewalk retrofit cost ===
Q += w2 * Q2
# === f3: construction cost (diagonal term) ===
Q += w3 * Q3
# === f4: distance-based spread penalty ===
Q += w4 * Q4
```

DATA



FLOWCHART



METHODOLOGY

an kernel-based unnormalized density estimation

$$p_i = \sum_{j=1}^M w_j \cdot \exp\left(-\frac{(d_{ij})^2}{2h^2}\right) \quad s_i = \sum_{j=1}^M cost_j$$

p_i : Estimated population demand at candidate point i , computed via Gaussian KDE
 s_i : Estimated sidewalk retrofit cost at candidate point i , also derived via KDE
 w_j : Population weight at entry point j
 d_{ij} : Missing sidewalk length at entry point j
 d_{ij} : Levee distance between entry point j and candidate point i
 h : Bandwidth parameter controlling the spatial decay of kernel influence

NSGA-III

$$f_1(x) = -\sum_{i \in S} p_i \cdot x_i \quad f_2(x) = \sum_{i \in S} s_i \quad f_3 = \sum_{i \in S} c_i \quad f_4 = \text{penalty}_i = \sum_{i=1}^N \sum_{j=1, j \neq i}^N e^{-\lambda \cdot d_{ij}} x_i x_j$$

$f_1(x)$: Negative population benefit
 $f_2(x)$: Sidewalk retrofit cost adjusted
 $f_3(x)$: Total construction cost of selected facilities
 penalty_i : Product of exponential decay terms for point i , reducing its influence when nearby selected points j exist
 d_{ij} : Levee network distance between points i and j
 λ : Decay rate set to enforce a minimum levee distance between any two selected points.
 x_i : A binary decision variable for candidate site i
 x_j : A binary decision variable for any other candidate site j ($i \neq j$);

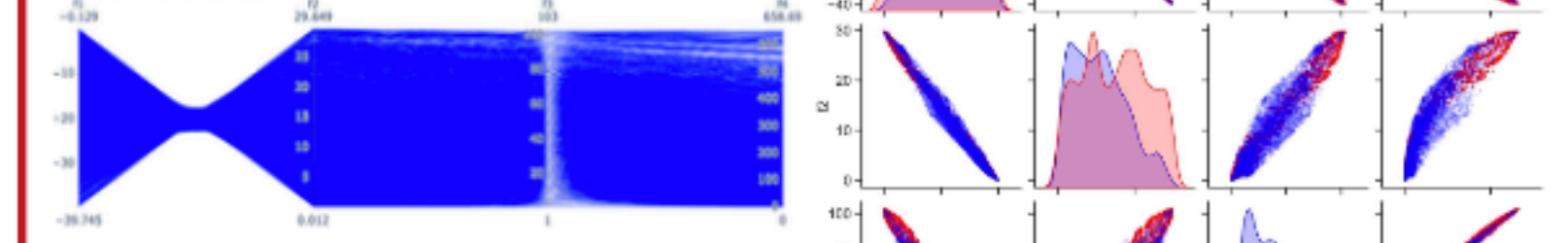
QUBO

$$\text{Hamiltonian} = w_1 f_1 + w_2 f_2 + w_3 f_3 + w_4 f_4$$

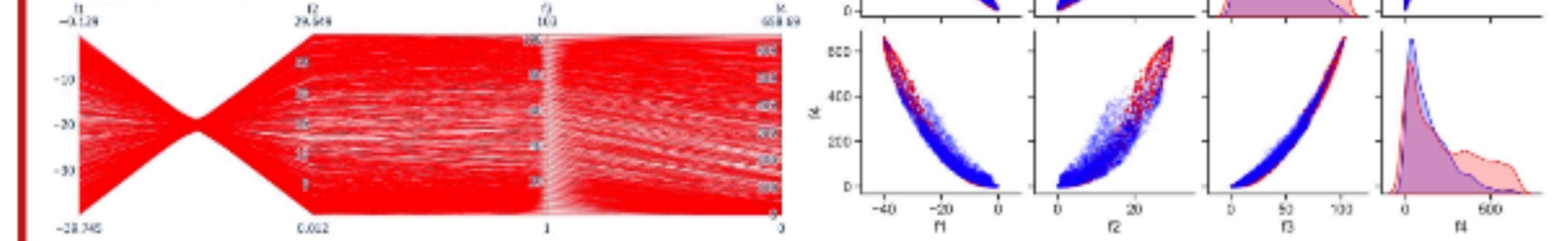
The scalar weights w_1 , w_2 , w_3 and w_4 are derived through systematic scalarization over the unit simplex, allowing the QUBO formulation to approximate the Pareto front by exploring diverse trade-offs across objectives.

RESULT

NSGA-III Parallel Coordinates



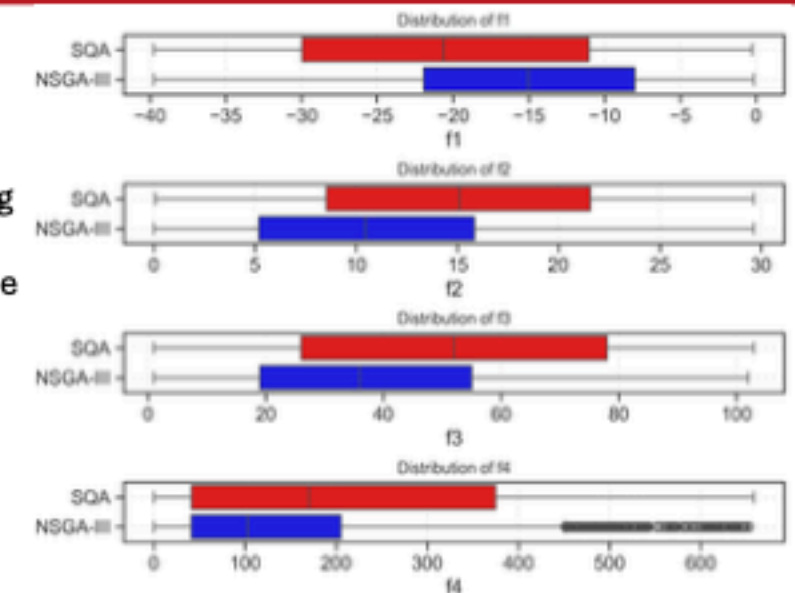
SQA Parallel Coordinates



Using simulated quantum annealing with systematic weight scanning, we obtained diverse local optima, each tied to a specific scalarization. This shows that NSGA-III-style multi-objective problems can be effectively handled via scalarized single-objective SQA.

DISCUSSIONS AND CONCLUSIONS

- Solution Quality:** NSGA-III and SQA generated 6,746 and 8,570 solutions respectively, with no method strictly dominating the other—indicating similar search depth.
- Search Strategy:** NSGA-III converges near dense reference points, while SQA uniformly scans all weights across facility counts.
- Future Work:** Explore GPU-accelerated SQA and QPU experiments to improve efficiency and performance.



REFERENCE

- [1] Scott, D. W. (2015). *Multivariate Density Estimation: Theory, Practice, and Visualization*. Wiley.
- [2] Deb, K., & Jain, H. (2014). An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, Part I: Solving problems with box constraints. *IEEE Transactions on Evolutionary Computation*, 18(4), 577–601.
- [3] Lucas, A. (2014). Ising formulations of many NP problems. *Frontiers in Physics*, 2, 5.
- [4] Glover, F., Kochenberger, G., & Du, Y. (2018). A tutorial on formulating and using QUBO models. *arXiv preprint arXiv:1811.11538*.


```
import numpy as np
from tqdm import tqdm
import matplotlib.pyplot as plt

# --- 1. 參數設定（全部繁體註解） ---
NUM_REPLICAS    = 4      # 複製體數量
TROTTER_SLICES  = 5      # 虛擬時間切片數
NUM_STEPS       = 100    # beta/gamma 演化步數
BETA_INITIAL    = 0.1    # 退火參數 β 起始值
BETA_FINAL      = 5.0    # 退火參數 β 結束值
GAMMA_INITIAL   = 1.0    # 橫場強度 γ（可改動）
SEED            = 42     # 隨機種子

class SQA_numpy:
    def __init__(self, Q: np.ndarray, init: np.ndarray):
        if init is None:
            raise ValueError("需要提供初始解 init")
        np.random.seed(SEED)
        self.Q = Q.astype(float)      # QUBO 矩陣
        N = Q.shape[0]
        self.rep    = NUM_REPLICAS    # 複製體數
        self.slices = TROTTER_SLICES  # 時間切片數
        # 建立 β 與 γ 的走線
        self.beta  = np.linspace(BETA_INITIAL, BETA_FINAL,
NUM_STEPS)
        self.gamma = np.full(NUM_STEPS, GAMMA_INITIAL)
        # state[r, s, i] : r=replica, s=slice, i=bit
        self.state = np.tile(init, (self.rep, self.slices, 1)).astype(int)

        # 全域最佳解與能量
        self.best_state = init.copy()
        self.best_energy = float(init @ (self.Q @ init))

        # 接受率計數器
        self.attempted = 0
        self.accepted  = 0

    def local(self, x: np.ndarray) -> float:
        """計算單一配置 x 的 QUBO 能量 x^T Q x"""
        return float(x @ (self.Q @ x))

    def coupling(self, a: np.ndarray, b: np.ndarray, g: float) -> float:
        """計算相鄰切片間的耦合能量 -γ Σ(a_i ≠ b_i)"""
        return -g * np.sum(a != b)

    def totalE(self, r: int, s: int, g: float) -> float:
        """計算 replica r、slice s 的總能量"""
        e_local = self.local(self.state[r, s])
        e_coup  = self.coupling(self.state[r, s],
                                self.state[r, (s + 1) % self.slices],
                                g)
        return e_local + e_coup

    def swap1(self, x: np.ndarray) -> np.ndarray:
        """隨機交換一個 1 和一個 0 以產生新解"""
        x_new = x.copy()
        ones = np.where(x_new == 1)[0]
        zeros= np.where(x_new == 0)[0]
        i = np.random.choice(ones)
        j = np.random.choice(zeros)
        x_new[i], x_new[j] = 0, 1
        return x_new

    def step(self, b: float, g: float):
        """
        一個 Monte Carlo sweep :
        對每個 replica r、每個切片 s 嘗試 swap1 並使用 Metropolis 準則
        """
        for r in range(self.rep):
            for s in range(self.slices):
                cur = self.state[r, s]
                prop= self.swap1(cur)
                e0  = self.totalE(r, s, g)
                self.attempted += 1

                # 臨時套用新解，計算新能量
                self.state[r, s] = prop
                e1 = self.totalE(r, s, g)

                # 決定接受或拒絕
                if np.random.rand() < np.exp(-b * (e1 - e0)):
                    # 接受
                    self.accepted += 1
                    # 更新全局最佳
                    if e1 < self.best_energy:
                        self.best_energy = e1
                        self.best_state = prop.copy()
                else:
                    # 拒絕，回復原本狀態
                    self.state[r, s] = cur

    def run(self, init=None):
        """主迴圈：在 beta/gamma 走線下嚮導演化"""
        print(f"[初始能量] {self.best_energy:.4f}")
        for t in tqdm(range(NUM_STEPS), desc="SQA 進度"):
            self.step(self.beta[t], self.gamma[t])
            # 顯示 Metropolis 接受率
            if self.attempted > 0:
                rate = self.accepted / self.attempted
                print(f"[接受率] {self.accepted}/{self.attempted} = {rate:.2%}")
            return self.best_state, self.best_energy

def run_sqa(Q: np.ndarray, init: np.ndarray):
    return SQA_numpy(Q, init).run()

if __name__ == "__main__":
    # --- 2. 測試用小型 QUBO 舉例（N=10） ---
    np.random.seed(SEED)
    N = 10
    # 隨機產生一個對稱 QUBO 矩陣
    Q_test = np.random.randn(N, N)
    Q_test = (Q_test + Q_test.T) / 2
    init  = np.zeros(N, dtype=int)
    init[:N//2] = 1 # 前半部設成 1、後半部設成 0

    # 3. 執行 SQA
    best_state, best_energy = run_sqa(Q_test, init)

    # 4. 列印數值結果
    print("\n=== 測試結果 ===")
    print("初始解      :", init)
    print("最佳解       :", best_state)
    print(f"最佳能量    : {best_energy:.4f}")

    # 5. 繪製進化過程中的接受率（簡單示意）
    fig, ax = plt.subplots(1,2, figsize=(10,4))
    ax[0].plot(np.arange(len(best_state)), best_state, 'o-')
    ax[0].set_title("最佳配置 (0/1)")
    ax[0].set_xlabel("變數索引 i")
    ax[0].set_ylabel("值")
    ax[1].bar(["嘗試", "接受"], [SQA_numpy(Q_test, init).attempted,
SQA_numpy(Q_test, init).accepted])
    ax[1].set_title("接受率示意")
    plt.tight_layout()
    plt.show()
```

Chapter 7

D-Wave DEMO

<https://github.com/dwave-training/nl-opt-intro>

DEMO 1: NL Solver Basics-Solution

Minimize where $y = i^2 - 4i$ is a variable that can take on integer values between -5 and 5 (inclusive).

Create Model

```
# required imports
from dwave.optimization import Model

# create blank model
model = Model()

# add in i variable
i = model.integer(lower_bound=-5,
upper_bound=5)

# add in constant of 4
c = model.constant(4)

# create $y = i^2 - 4i$
y = i**2 - c*i

# minimize y
model.minimize(y)
```

Solve Problem

```
# hint-use LeapHybridNLSampler in dwave.system
from dwave.system import LeapHybridNLSampler
sampler = LeapHybridNLSampler()
sampler.sample(model)    Importing the D-Wave Hybrid Solver
```

Check Results

```
# Check the objective and value of i
current_state = 0
with model.lock():
    print(model.objective.state(current_state))    y = -4.0
    print(i.state(current_state))                i = 2.0
```

$$H_P = y = \min_{i \in [-5, 5]} i^2 - 4i$$

DEMO 2: NL Solver-Constraints-Solution

Create an NL Model with the following objective function and constraints:

Objective function: minimize $x_1^2 - x_1x_2$

(each variable is an integer variable with lower bound of -10 and upper bound of 10)

Constraint 1: $x_1 \leq x_2$

Constraint 2: $x_1 \leq x_2^2$

Create Model

```
# required imports
from dwave.optimization import Model

# create blank model
model = Model()

# add x1 and x2 integer variables to model
x1 = model.integer(lower_bound =
-10, upper_bound = 10)
x2 = model.integer(lower_bound =
-10, upper_bound = 10)
```

Add Constraints, Objective

```
constraint1 = x1 <= x2
model.add_constraint(constraint1)

constraint1 = x1 <= x2 ** 2
model.add_constraint(model.constraint2)

model.minimize(x1 ** 2 - x1 * x2)
```

Solve Problem

```
from dwave.system import LeapHybridNLSampler
sampler = LeapHybridNLSampler()
sampler.sample(model, label = "demo model")
```


DEMO 2: NL Solver-Constraints-Solution

Create an NL Model with the following objective function and constraints:

Objective function: minimize $x_1^2 - x_1x_2$

(each variable is an integer variable with lower bound of -10 and upper bound of 10)

Constraint 1: $x_1 \leq x_2$

Constraint 2: $x_1 \leq x_2^2$

Check Results

```
state_index = 0
with model.lock():
    print(f"Objective :
{model.objective.state(state_index)}")
    print(f"Constraint 1 Fulfilled? :
{constraint1.state(state_index)}")
    print(f"Constraint 2 Fulfilled? :
{model.constraint2.state(state_index)}")
    print(f"x1: {x1.state(state_index)}")
    print(f"x2: {x2.state(state_index)}")
```

Objective : -25.0

Constraint 1 Fulfilled? : 1.0

Constraint 2 Fulfilled? : 1.0

x1: 5.0

x2: 5.0

Check Results

```
with model.lock():
    print(constraint1.state(state_index))
    print(model.constraint2.state(state_index))
```

1.0

1.0

DEMO 3: "Set" Variable-Knapsack Problem

There are four items to choose from with the following weights and values:

- Item 1: 10 kg; \$5
- Item 2: 20 kg; \$7
- Item 3: 5 kg; \$2
- Item 4: 15 kg; \$9

The knapsack has a capacity of 30 kg. Select which items to put in the knapsack to maximize the total value of included items while staying under the 30 kg weight limit.

Create Model

```
# required imports
```

```
from dwave.optimization import Model
```

```
# create blank model
```

```
model = Model()
```

```
# Add constants
```

```
weights = model.constant([10, 20, 5, 15])
```

```
values = model.constant([-5, -7, -2, -9])
```

```
capacity = model.constant(30)
```

```
# Add the decision variable
```

```
items = model.set(4)
```

```
Declare variables for four items:
```

```
items[i] = 1 selected
```

```
items[i] = 0 not selected
```

```
# add the capacity constraint
```

```
model.add_constraint(weights[items].sum() <= capacity)
```

```
# Set the objective
```

```
model.minimize(values[items].sum())
```

DEMO 3: "Set" Variable-Knapsack Problem

There are four items to choose from with the following weights and values:

- Item 1: 10 kg; \$5
- Item 2: 20 kg; \$7
- Item 3: 5 kg; \$2
- Item 4: 15 kg; \$9

The knapsack has a capacity of 30 kg. Select which items to put in the knapsack to maximize the total value of included items while staying under the 30 kg weight limit.

Solve Problem

```
from dwave.system import LeapHybridNLSampler
sampler = LeapHybridNLSampler()
sampler.sample(model)
```

Value of Selected Items: 16.0

Index of Selected Items: [3. 2. 0.] **Item 1, 2, 4**

Weights of Selected Items: [15.0, 5.0, 10.0]

Values of Selected Items: [9.0, 2.0, 5.0]

Check Results

```
with model.lock():
    print("Value of Selected Items:",
-model.objective.state(0))
    index_selected_items = items.state(0)
    print("Index of Selected Items:",index_selected_items)
```

```
selected_weights = [weights.state(0).tolist()[int(i)] for i in
index_selected_items.tolist()]
```

```
print("Weights of Selected Items:",selected_weights)
```

```
selected_values = [-values.state(0).tolist()[int(i)] for i in
index_selected_items.tolist()]
```

```
print("Values of Selected Items:",selected_values)
```


DEMO 4: "List" Variable-Basic routing Problem

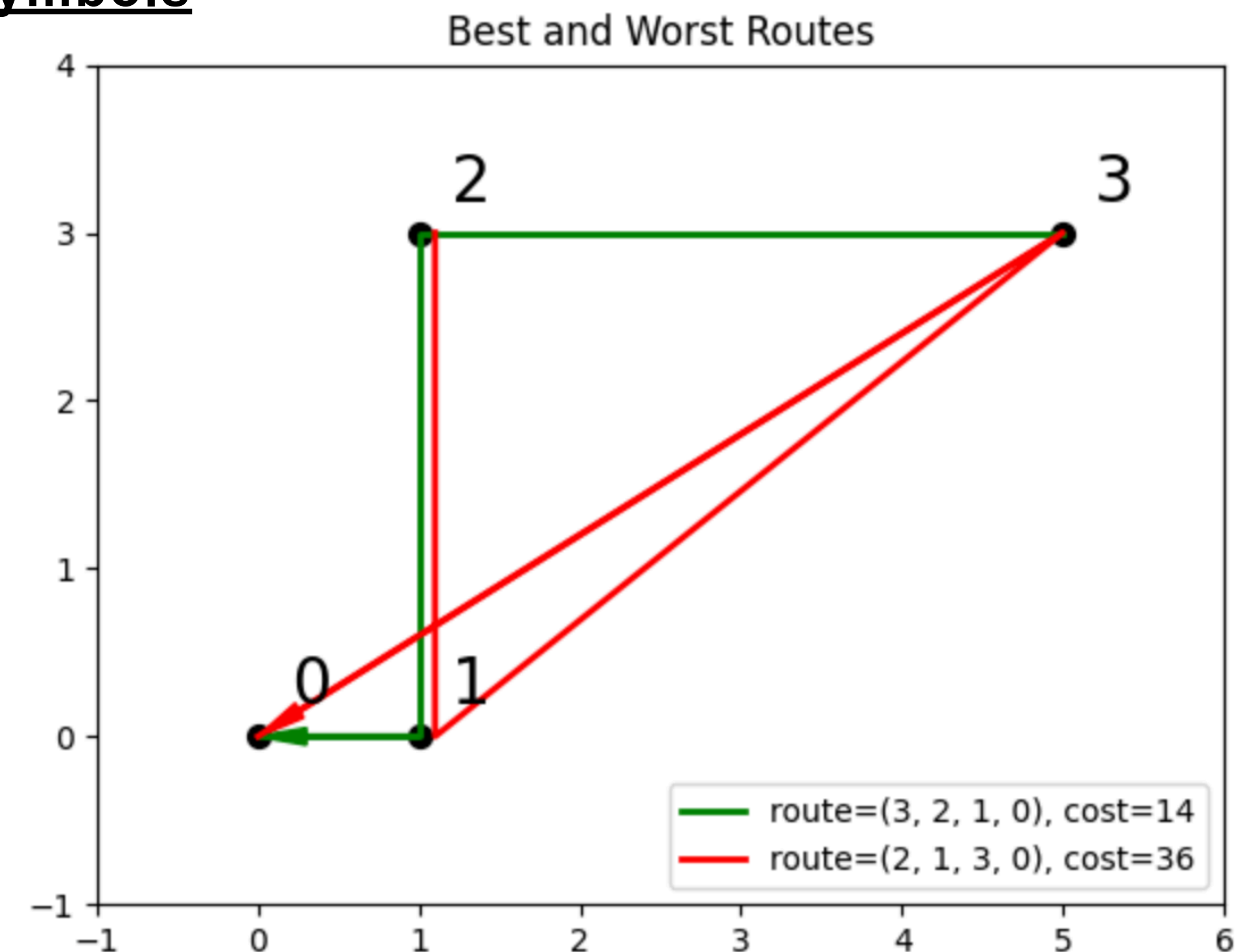
Consider a problem of selecting a route for several destinations with the cost increasing on each leg of the itinerary; for the example formulated below, one can travel through four destinations in any order, one destination per day, with the transportation cost per unit of travel doubling every subsequent day.

The figure below shows four destinations as dots labeled 0 to 3, and plots the least costly (green) and most costly (red) routes.

Reference: https://docs.dwavequantum.com/en/latest/industrial_optimization/model_construction_nl.html#example-implicitly-constrained-symbols

Import

```
from IPython.display import Image
from IPython.core.display import HTML
Image(url="https://docs.dwavequantum.com/en/latest/_images/best_worst_routes.png")
```



DEMO 4: "List" Variable-Basic routing Problem

Consider a problem of selecting a route for several destinations with the cost increasing on each leg of the itinerary; for the example formulated below, one can travel through four destinations in any order, one destination per day, with the transportation cost per unit of travel doubling every subsequent day.

Define Costs

```
import numpy as np
cost_per_day = [1, 2, 4]
distance_matrix = np.asarray([
    [0, 1, np.sqrt(10), np.sqrt(34)],
    [1, 0, 2, np.sqrt(25)],
    [np.sqrt(10), 3, 0, 4],
    [np.sqrt(34), np.sqrt(25), 4, 0]])
```

Create Model

```
from dwave.optimization import Model
model = Model()
# Add the constants
cost = model.constant(cost_per_day)
distances = model.constant(distance_matrix)
# Add the decision symbol
route = model.list(4) #[0,1,2,3]

# Optimize the objective
model.minimize((cost * distances[route[:-1],route[1:]]).sum())
```

	A	B	C	D
A	0	1	$\sqrt{10} \approx 3.16$	$\sqrt{34} \approx 5.83$
B	1	0	2	$\sqrt{25} = 5$
C	$\sqrt{10} \approx 3.16$	3	0	4
D	$\sqrt{34} \approx 5.83$	$\sqrt{25} = 5$	4	0

DEMO 4: "List" Variable-Basic routing Problem

Consider a problem of selecting a route for several destinations with the cost increasing on each leg of the itinerary; for the example formulated below, one can travel through four destinations in any order, one destination per day, with the transportation cost per unit of travel doubling every subsequent day.

Solve Problem

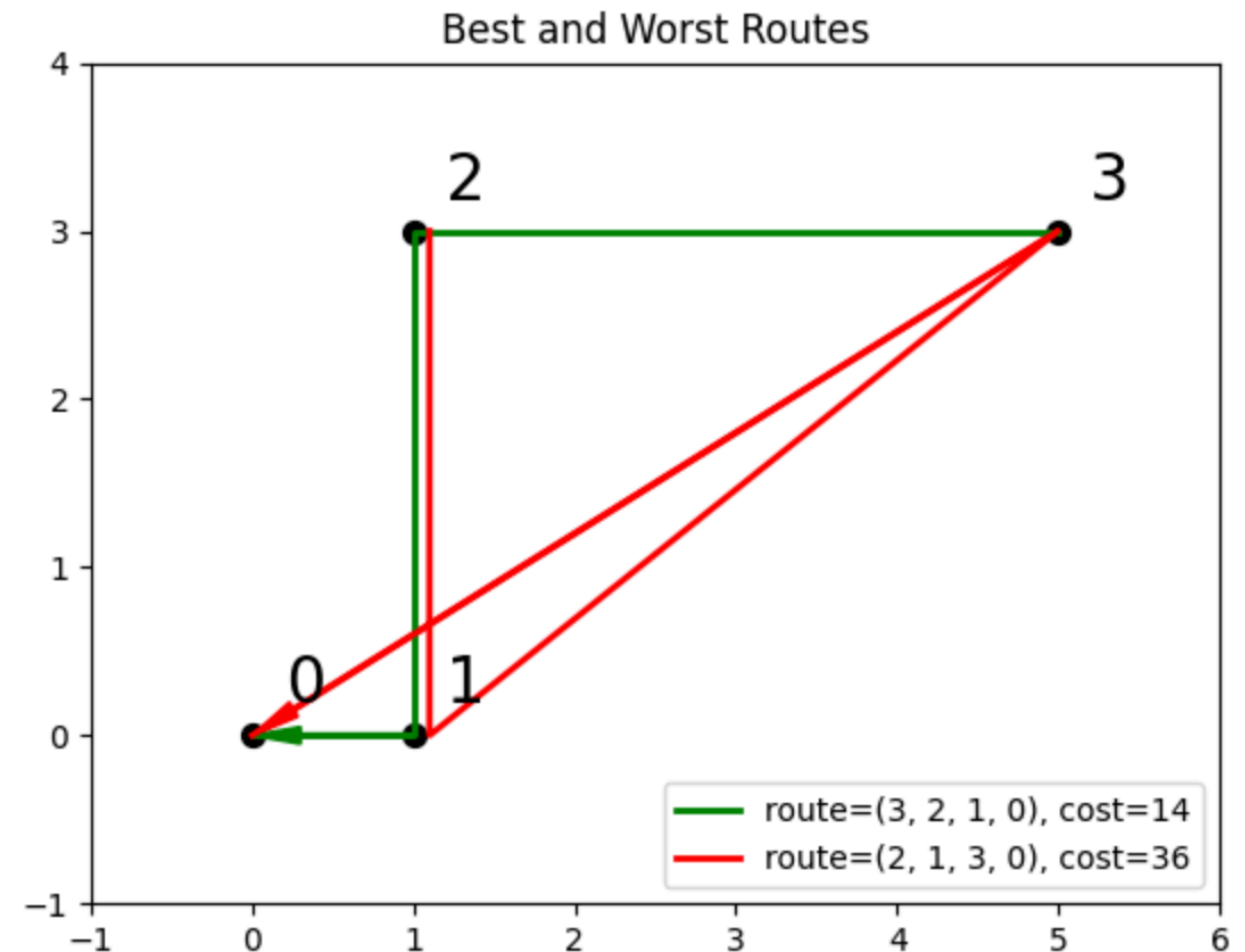
```
from dwave.system import LeapHybridNLSampler
sampler = LeapHybridNLSampler()
sampler.sample(model=model)
```

Check Results

```
with model.lock():
    print(f"Objective = {model.objective.state(0)}")
    print(route.state(0))
```

Objective = 14.0

[3. 2. 1. 0.]



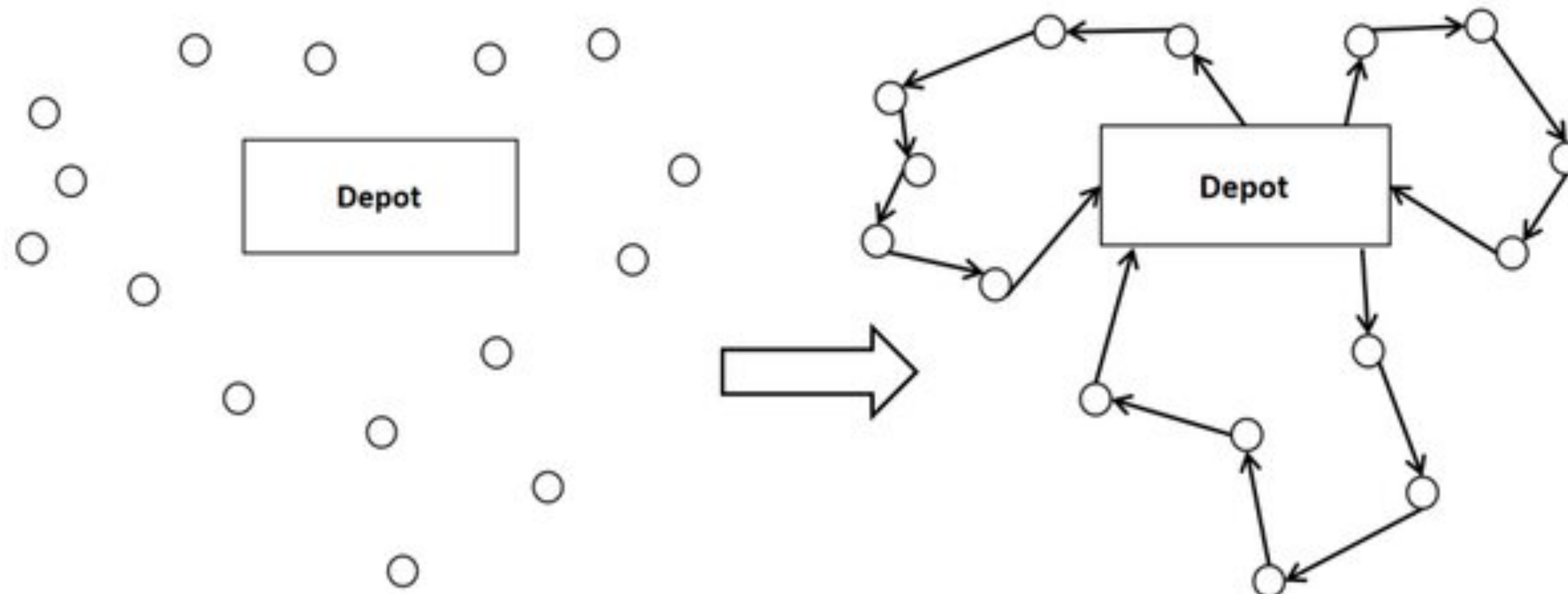
Chapter 8

Custom QUBO Application DEMO

DEMO 1: TSP to VRP

The **Vehicle Routing Problem (VRP)** is a combinatorial optimization problem that seeks the optimal visiting order and routing paths for multiple vehicles under limited resource constraints.

VRP is an extension of the **Traveling Salesman Problem (TSP)**: multiple vehicles depart from one or more depots to serve all demand points at minimum cost. It is commonly subject to practical constraints such as capacity limits, time windows, route choices, and vehicle-type restrictions.



https://www.youtube.com/watch?v=sJR_8e_Z2d4



Hirotaka Irie

DENSO CORPORATION

確認したメール アドレス: jp.denso.com - [ホームページ](#)

[mathematical physics](#) [string theory](#) [quantum computing](#)



DENSO

タイトル

引用先

年

[Quantum annealing of vehicle routing problem with time, state and capacity](#)

98

2019

H Irie, G Wongpaisarnsin, M Terabe, A Miki, S Taguchi

Quantum Technology and Optimization Problems: First International Workshop ...

[Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. \(2019\)](#)

[“Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”](#)

[Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 \(QTOP 2019, Springer\)](#)

This work proposes a **QUBO model** that incorporates schedules, capacity constraints, and vehicle states to simulate the multiple constraints commonly encountered in real-world delivery processes.

[Cattelan, M. & Yarkoni, S. \(2024\)“Modeling Routing Problems in QUBO with Application to Ride-Hailing”](#)

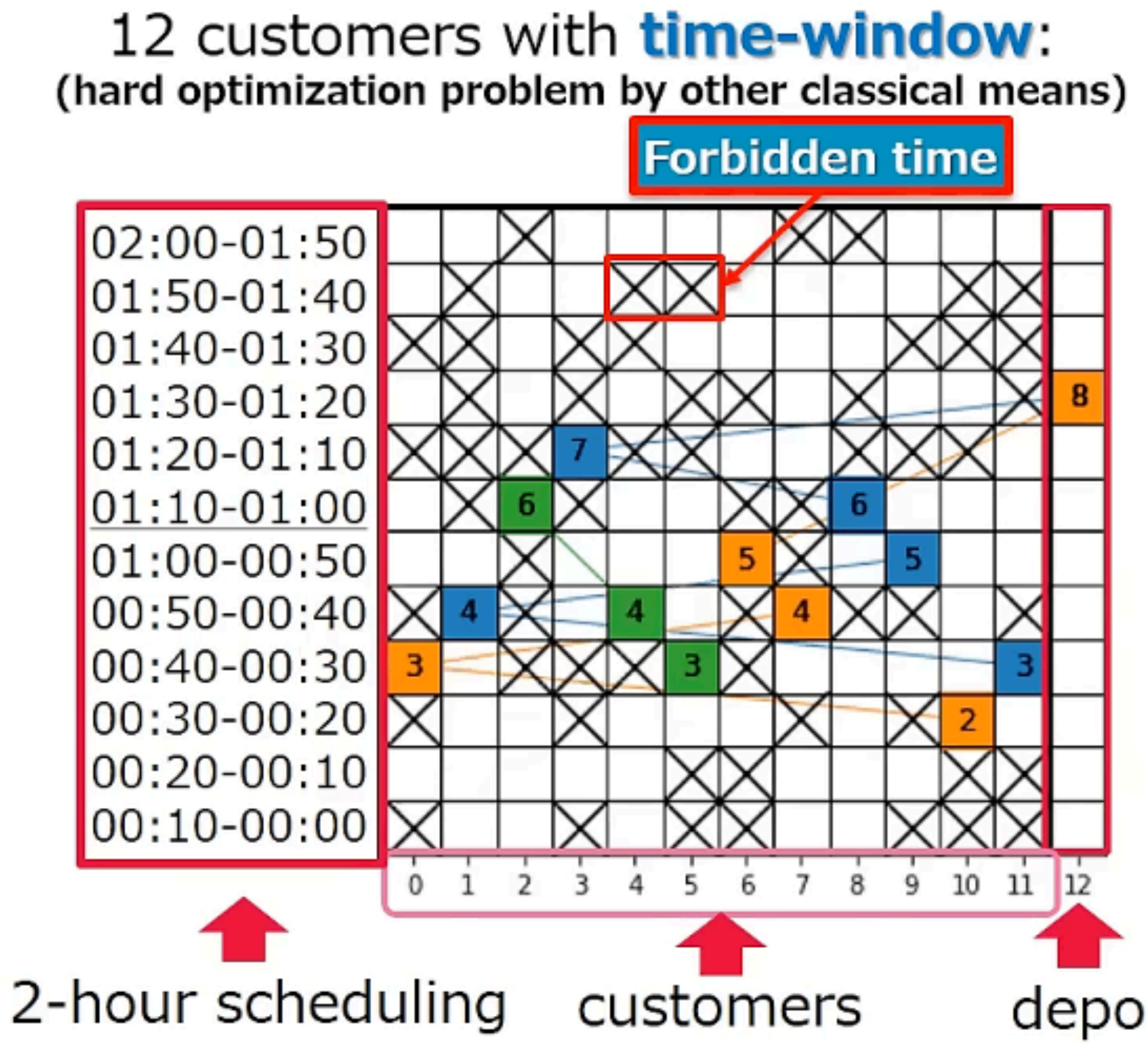
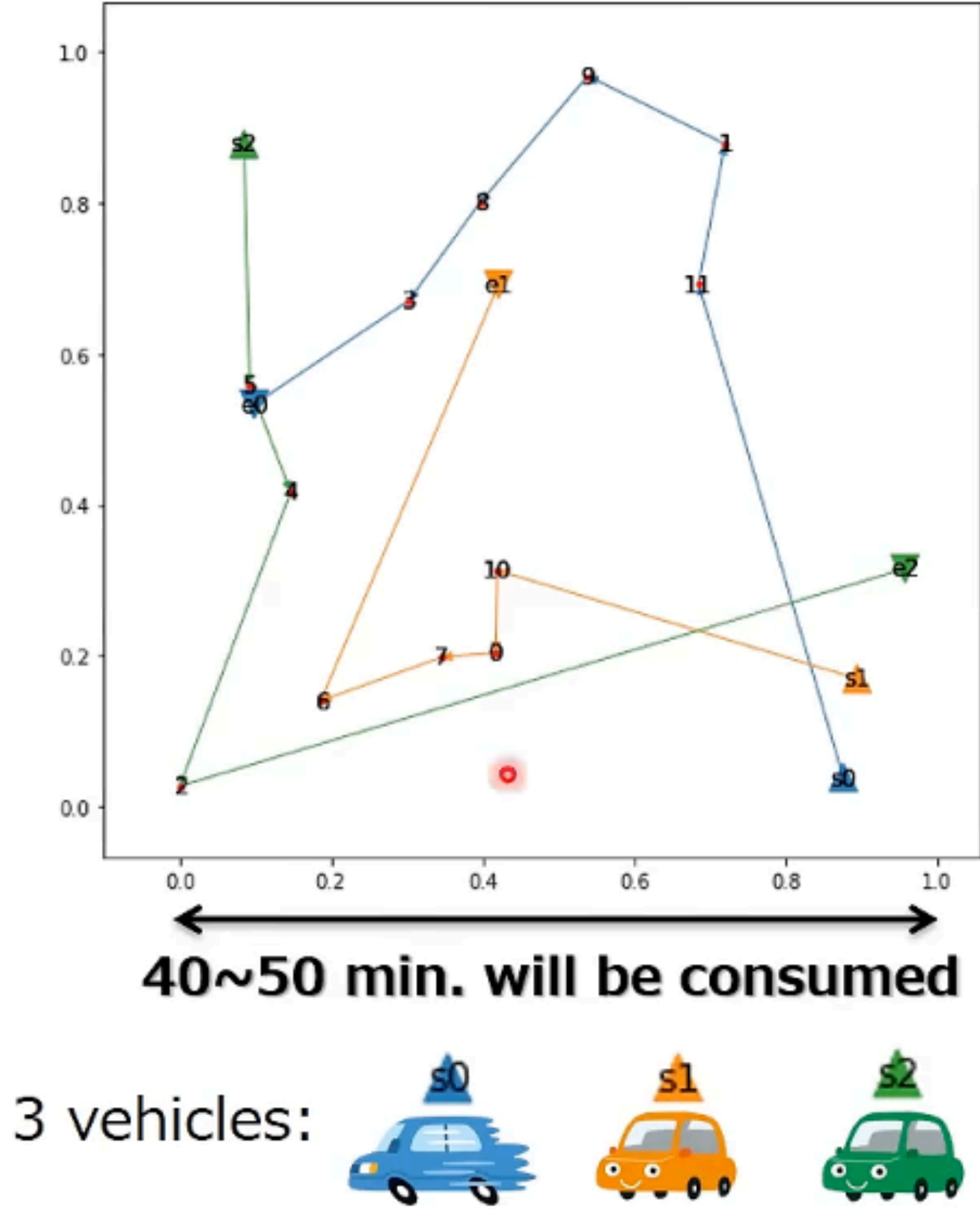
[Published in Scientific Reports, Vol. 14, Article 19768](#)

The study focuses on transforming the **Ride-Pooling problem** into a QUBO formulation and introduces a **node step-based variable design** to effectively reduce the required number of qubits and coupling density.

The goal is to improve solvability and efficiency under limited quantum resources.

DEMO 1: TSP to VRP

Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. (2019)
“Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”
Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 (QTOP 2019, Springer)

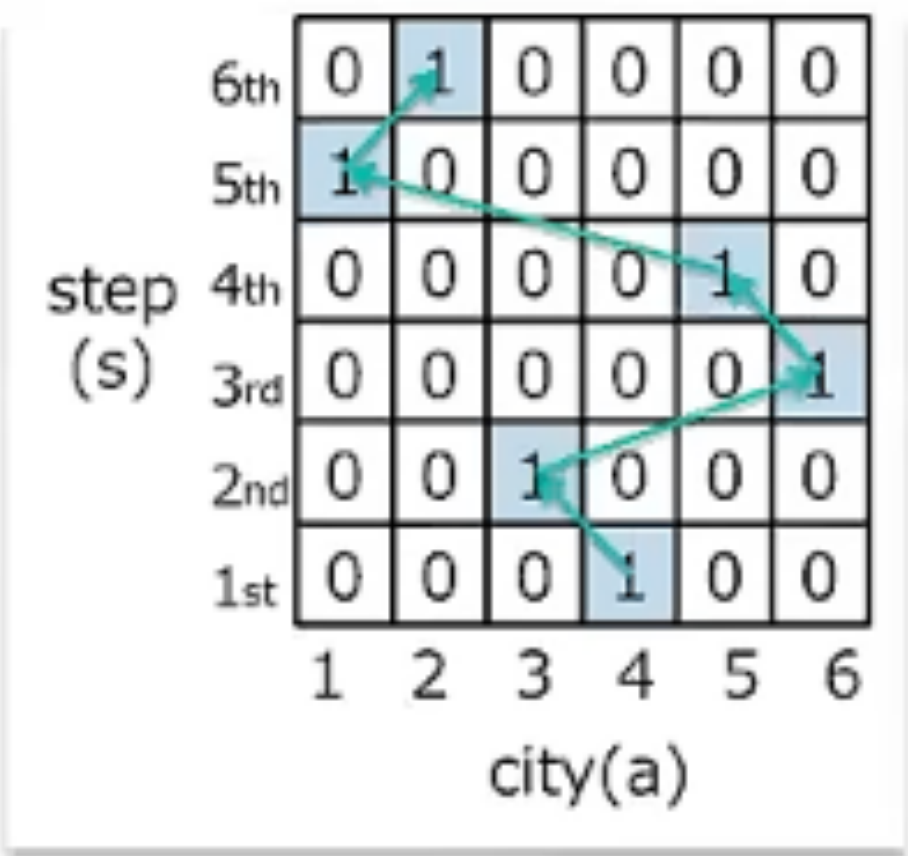


DEMO 1: TSP to VRP

Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. (2019)
 “Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”
 Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 (QTOP 2019, Springer)

TSP (Traveling Salesman Problem)
 A combinatorial optimization problem that seeks the shortest path allowing a salesman to visit each city exactly once and return to the starting point.

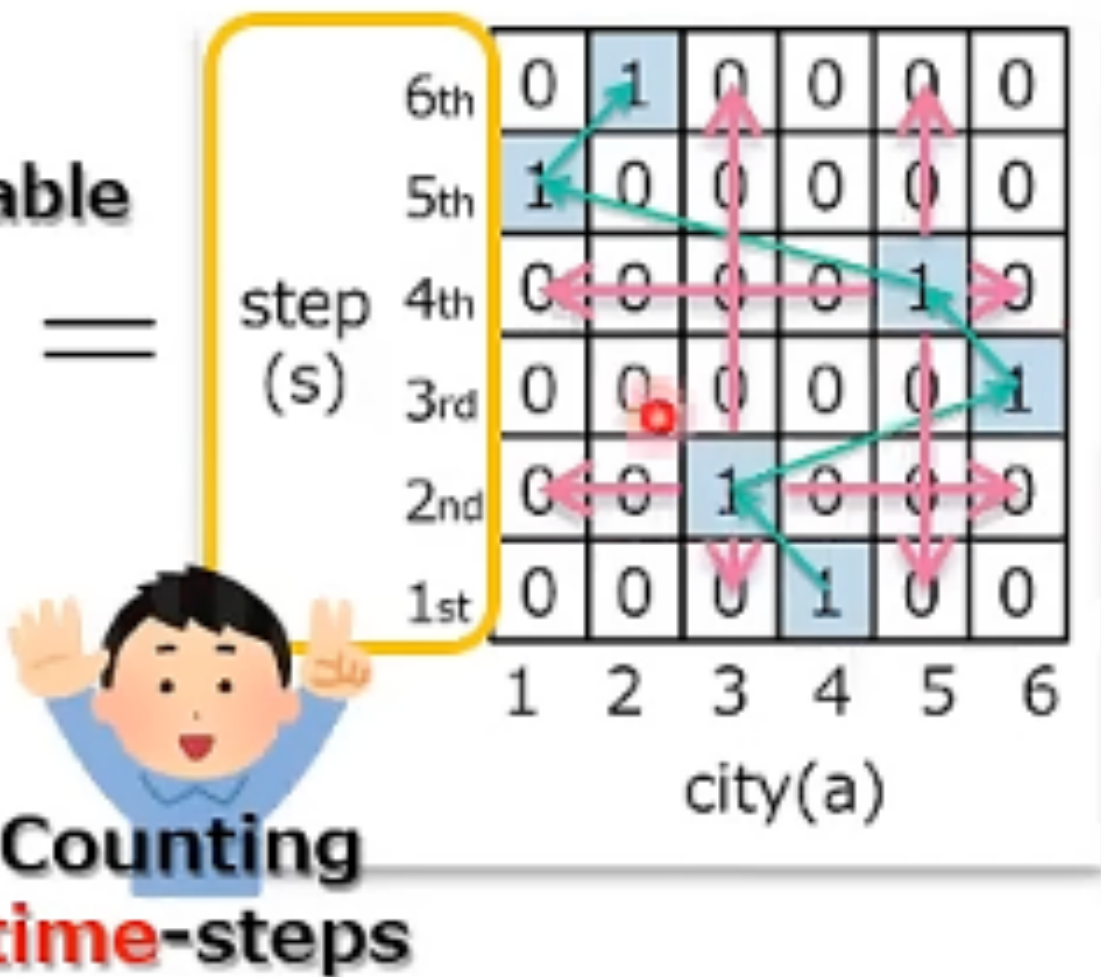
QUBO variable
 $x_{t,a} =$



Hamiltonian

$$\mathcal{H} = \sum_{\forall a \neq \forall b, \forall t} d_{ab} \times x_{t+1,a} x_{t,b} + \lambda \times \left[\sum_{\forall a} \left(\sum_{\forall t} x_{t,a} - 1 \right)^2 + \sum_{\forall t} \left(\sum_{\forall a} x_{t,a} - 1 \right)^2 \right]$$

QUBO variable
 $x_{t,a} =$



Hamiltonian

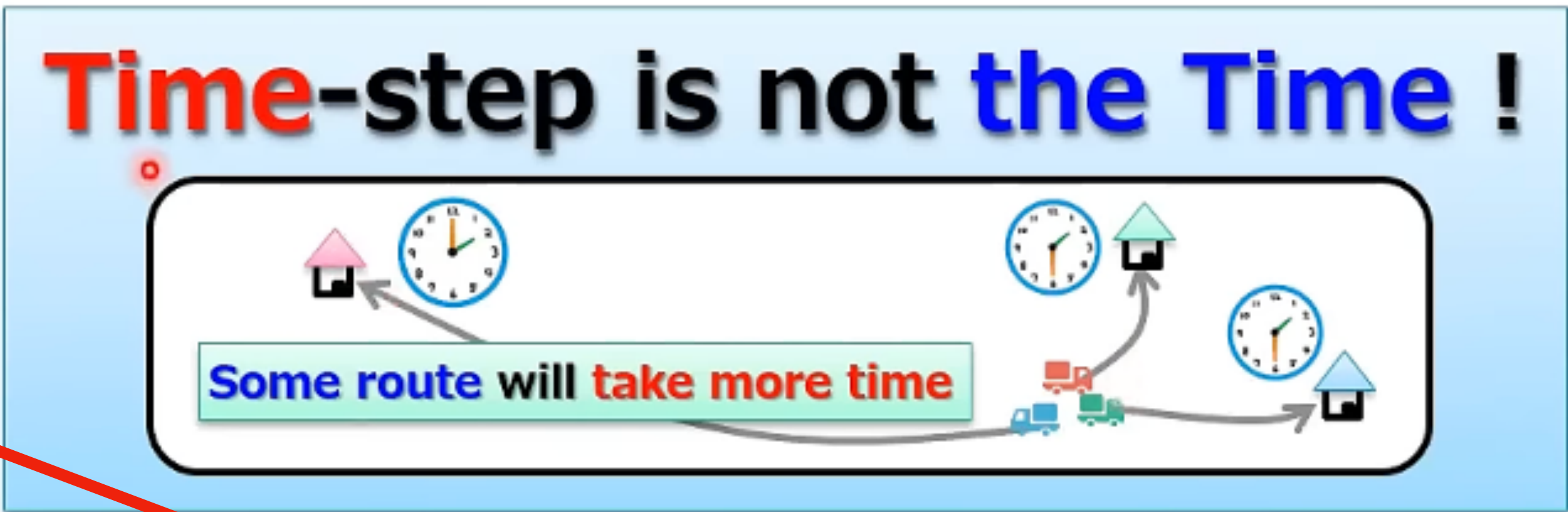
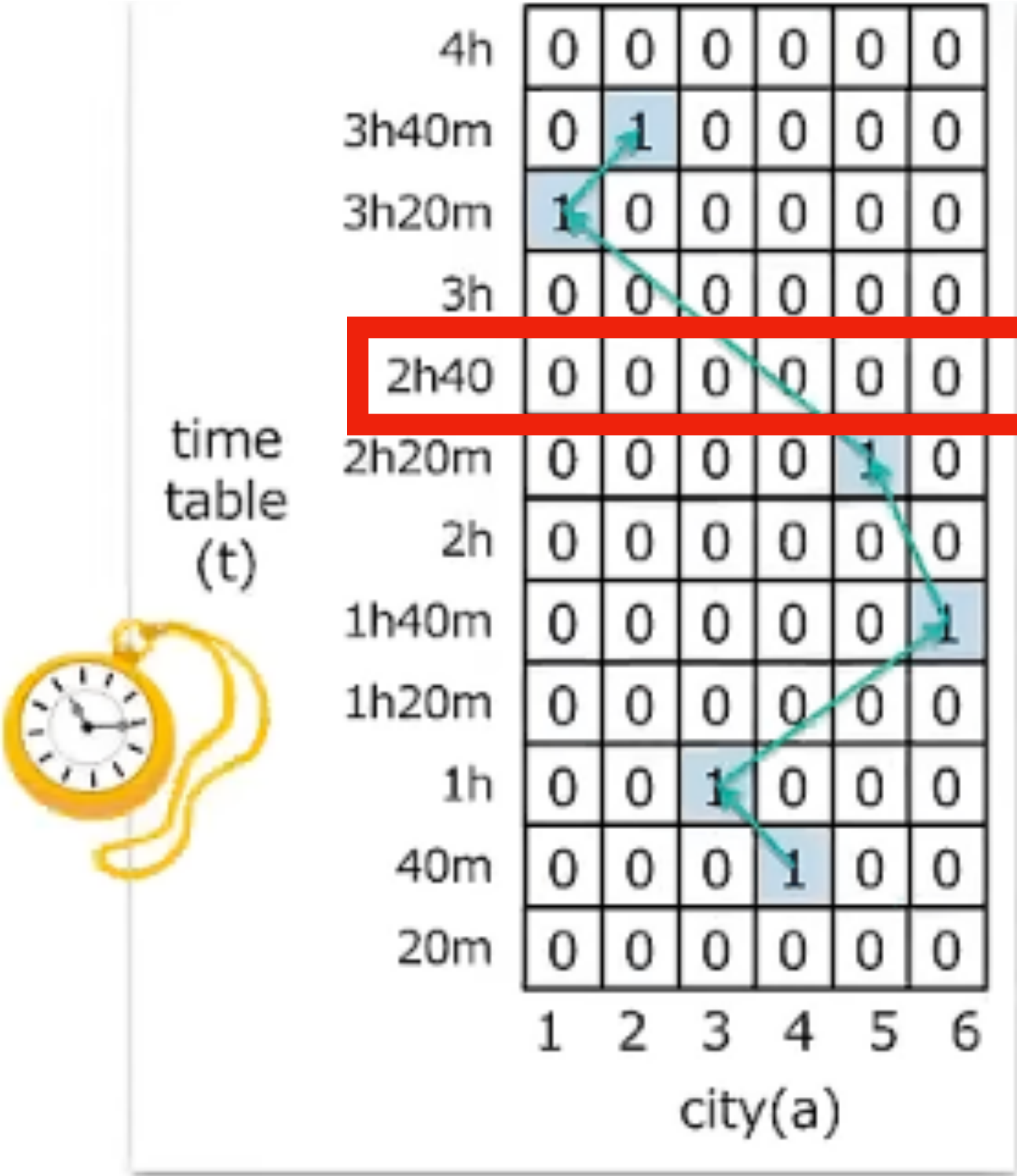
Cost of each travel step

$$\mathcal{H} = \sum_{\forall a \neq \forall b, \forall t} d_{ab} \times x_{t+1,a} x_{t,b} + \lambda \times \left[\sum_{\forall a} \left(\sum_{\forall t} x_{t,a} - 1 \right)^2 + \sum_{\forall t} \left(\sum_{\forall a} x_{t,a} - 1 \right)^2 \right]$$

Generating Hamiltonian Paths
 (N-luke configuration)

DEMO 1: TSP to VRP

Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. (2019)
“Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”
Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 (QTOP 2019, Springer)



Hamiltonian

Cost of each travel step

$$\mathcal{H} = \sum_{\forall a \neq \forall b, \forall t} d_{ab} \times x_{t+1,a} x_{t,b} +$$
$$+ \lambda \times \left[\sum_{\forall a} \left(\sum_{\forall t} x_{t,a} - 1 \right)^2 + \sum_{\forall t} \left(\sum_{\forall a} x_{t,a} - 1 \right)^2 \right]$$

NOT Satisfied !!!

Generating Hamiltonian Paths
(N-luke configuration)

DEMO 1: TSP to VRP

Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. (2019)
 “Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”
 Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 (QTOP 2019, Springer)

1. Replace the

$$\sum_{\forall a} \left(\sum_{\forall t} x_{t,a} - 1 \right)^2 + \sum_{\forall t} \left(\sum_{\forall a} x_{t,a} - 1 \right)^2$$

Replace the $\sum_{\forall a} \left(\sum_{\forall t} x_{t,a} - 1 \right)^2 + \sum_{\forall t} \left(\sum_{\forall a} x_{t,a} - 1 \right)^2$

$$= 2 \left(\sum_{\substack{\forall t < \forall t' \\ \forall a}} \boxed{x_{t,a} x_{t',a}} + \sum_{\substack{\forall a < \forall b \\ \forall t}} \boxed{x_{t,a} x_{t,b}} - \sum_{\forall t, \forall a} x_{t,a} \right)$$

Discard $x_{t,a}$ tends to be “1”

N-luke penalty term **Generate pop-up tendency**

Cost: $\sum_{\substack{\forall a \neq \forall b \\ \forall t}} d_{ab} x_{t+1,a} x_{t,b} \rightsquigarrow \sum_{\substack{\forall a \neq \forall b \\ \forall t}} (d_{ab} \boxed{-\mu}) x_{t+1,a} x_{t,b}$

Negative-shift of cost

Negative-shift formulation of TSP

$$H(x) = \sum_{\substack{\forall a \neq \forall b \\ \forall t}} \underbrace{(d_{ab} - \mu)}_{\equiv \epsilon_{ab}} x_{t+1,a} x_{t,b} + \lambda \left(\sum_{\substack{\forall t < \forall t' \\ \forall a}} x_{t,a} x_{t',a} + \sum_{\substack{\forall a < \forall b \\ \forall t}} x_{t,a} x_{t,b} \right)$$

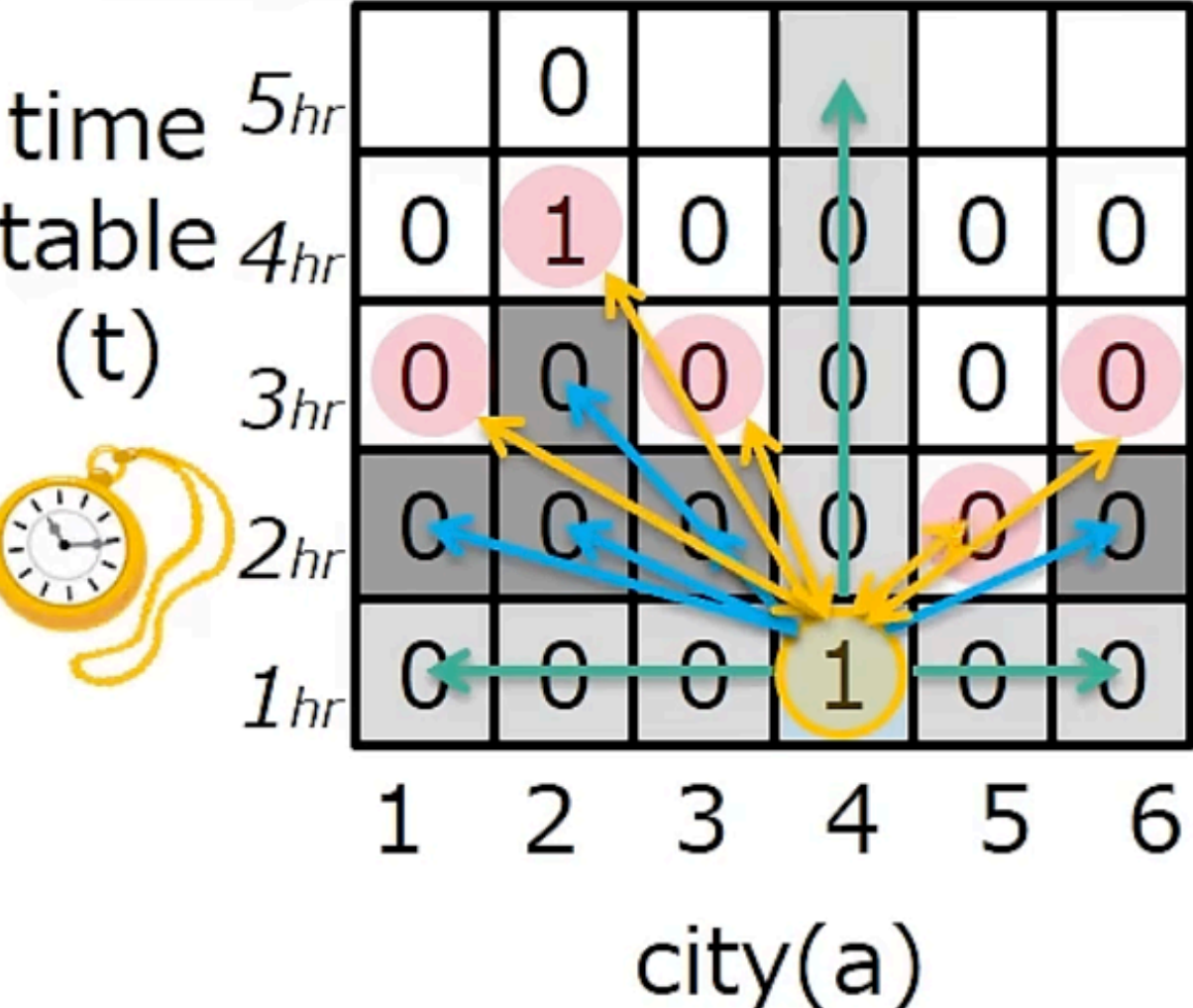
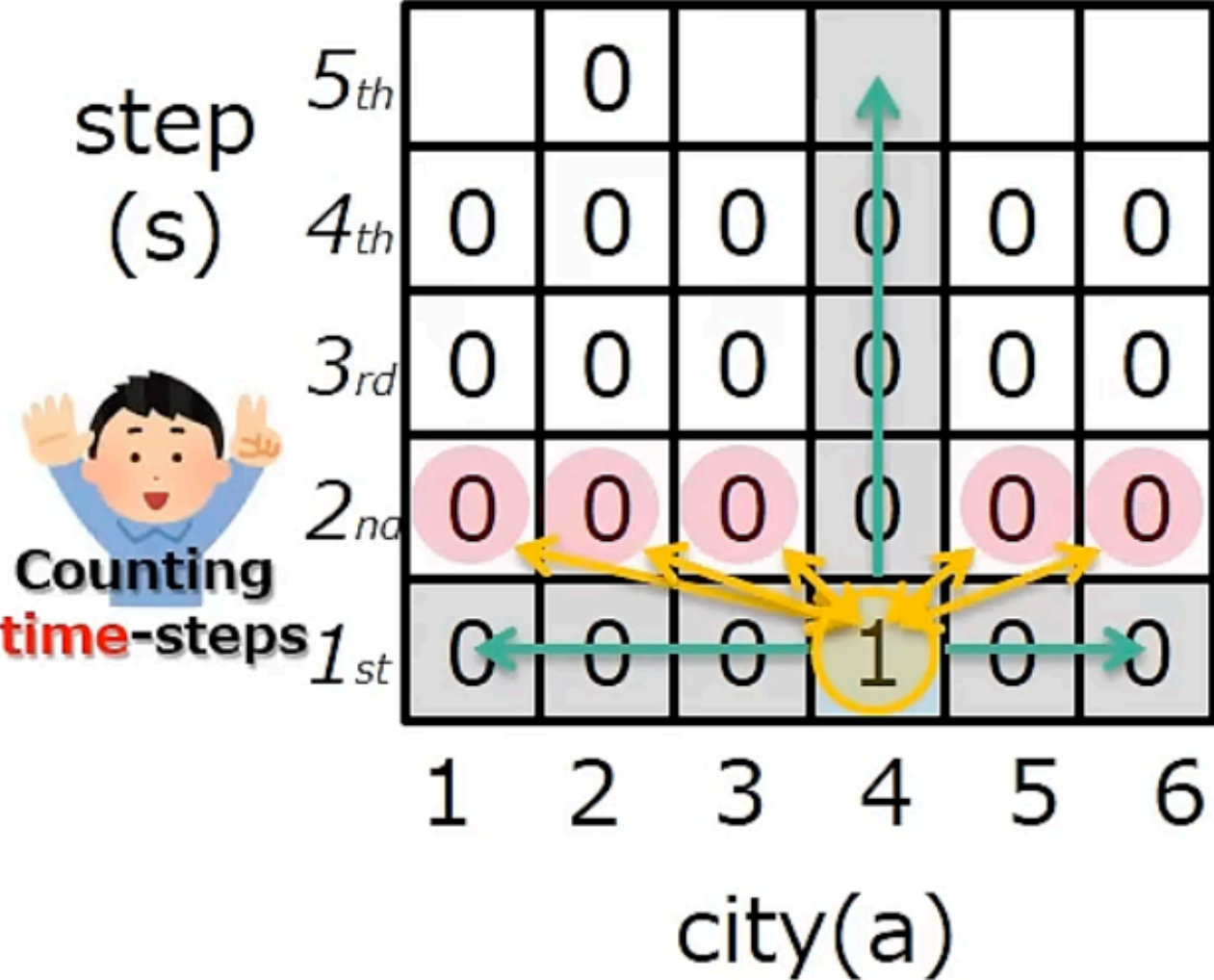
Configurations with penalty should be forbidden $\rightarrow$ relatively positive energy

$$\epsilon_{ab} + \lambda > 0 \iff -\lambda < \epsilon_{ab} = d_{ab} - \mu$$

DEMO 1: TSP to VRP

Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. (2019)
“Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”
Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 (QTOP 2019, Springer)

2. Introducing Jump



Cost:

$$\sum_{\substack{\forall a \neq \forall b \\ \forall t}} (d_{ab} - \mu) x_{t+1,a} x_{t,b}$$

One step forward
N-Luke Config.

Cost Is assigned to the **on-time** travel

$$\sum_{\substack{\forall a \neq \forall b \\ \forall t}} \left(((d_{ab} - \mu) \cdot x_{t+n_{ab},a} \cdot x_{t,b} \right) +$$

We should describe **time-duration** of travel

time-duration matrix n_{ab}

New dynamic penalty term

$$\lambda \sum_{\delta t=1}^{n_{ab}-1} x_{t+\delta t,a} x_{t,b} \right)$$

DEMO 1: TSP to VRP

Irie, H., Wongpaisarnsin, G., Terabe, M., Miki, A., & Taguchi, S. (2019)
 “Quantum Annealing of Vehicle Routing Problem with Time, State and Capacity”
 Preprint on arXiv:1903.06322; Later published in LNCS vol. 11413 (QTOP 2019, Springer)

Time-scheduled TSP

$$H = \sum_{\substack{\forall a \neq \forall b \\ \forall t}} \left((d_{ab} - \mu) \cdot x_{t+n_{ab},a} \cdot x_{t,b} + \lambda \sum_{\delta t=1}^{n_{ab}-1} x_{t+\delta t,a} x_{t,b} \right) + \lambda \left(\sum_{\substack{\forall t < \forall t' \\ \forall a}} x_{t,a} x_{t',a} + \sum_{\substack{\forall a < \forall b \\ \forall t}} x_{t,a} x_{t,b} \right)$$

One city per time step, and one time step per city.

Time-scheduled VRP

$$H = \sum_{\substack{\forall a \neq \forall b \\ \forall t, \forall i}} \left((d_{ab} - \mu) \cdot x_{t+n_{ab},a}^{(i)} \cdot x_{t,b}^{(i)} + \sum_{\delta t=1}^{n_{ab}-1} \lambda \cdot x_{t+\delta t,a}^{(i)} \cdot x_{t,b}^{(i)} \right) + \lambda \cdot \left(\sum_{\substack{\forall a < \forall b \\ \forall t, \forall i}} x_{t,a}^{(i)} \cdot x_{t,b}^{(i)} + \sum_{\substack{\forall a, \forall t \neq \forall t' \\ \forall i \leq j}} x_{t',a}^{(i)} \cdot x_{t,a}^{(j)} + \sum_{\substack{\forall a, \forall t \\ \forall i < \forall j}} x_{t,a}^{(i)} \cdot x_{t,a}^{(j)} \right)$$

With vehicle index i , each variable $x_{t,a}^{(i)}$ represents
 “vehicle i arrives at city a at time t .”

Each city may not be visited by multiple vehicles or revisited at different times.

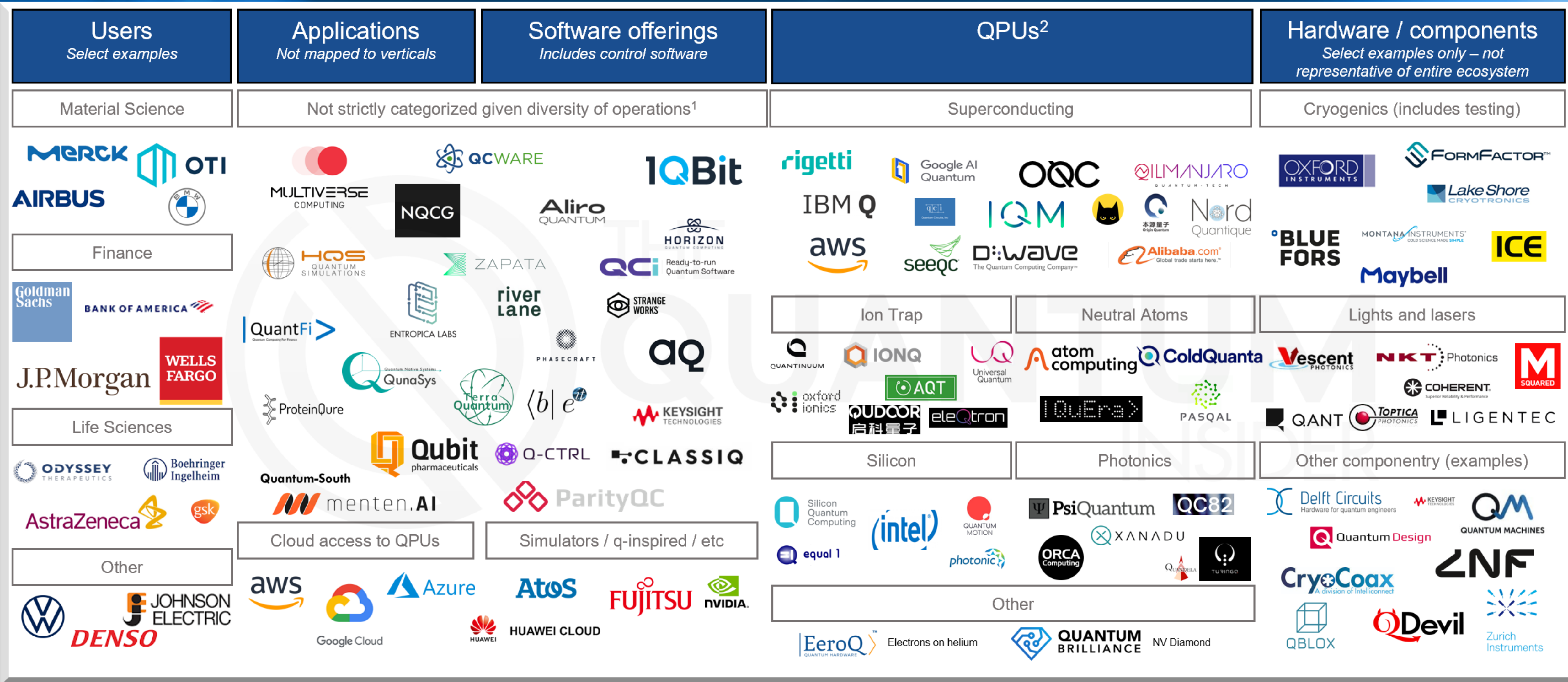
Chapter 9



Real World DEMO

Quantum Computing Market Map

Non exhaustive and in no particular order. Excludes details on control systems, assembly languages, circuit design, etc.



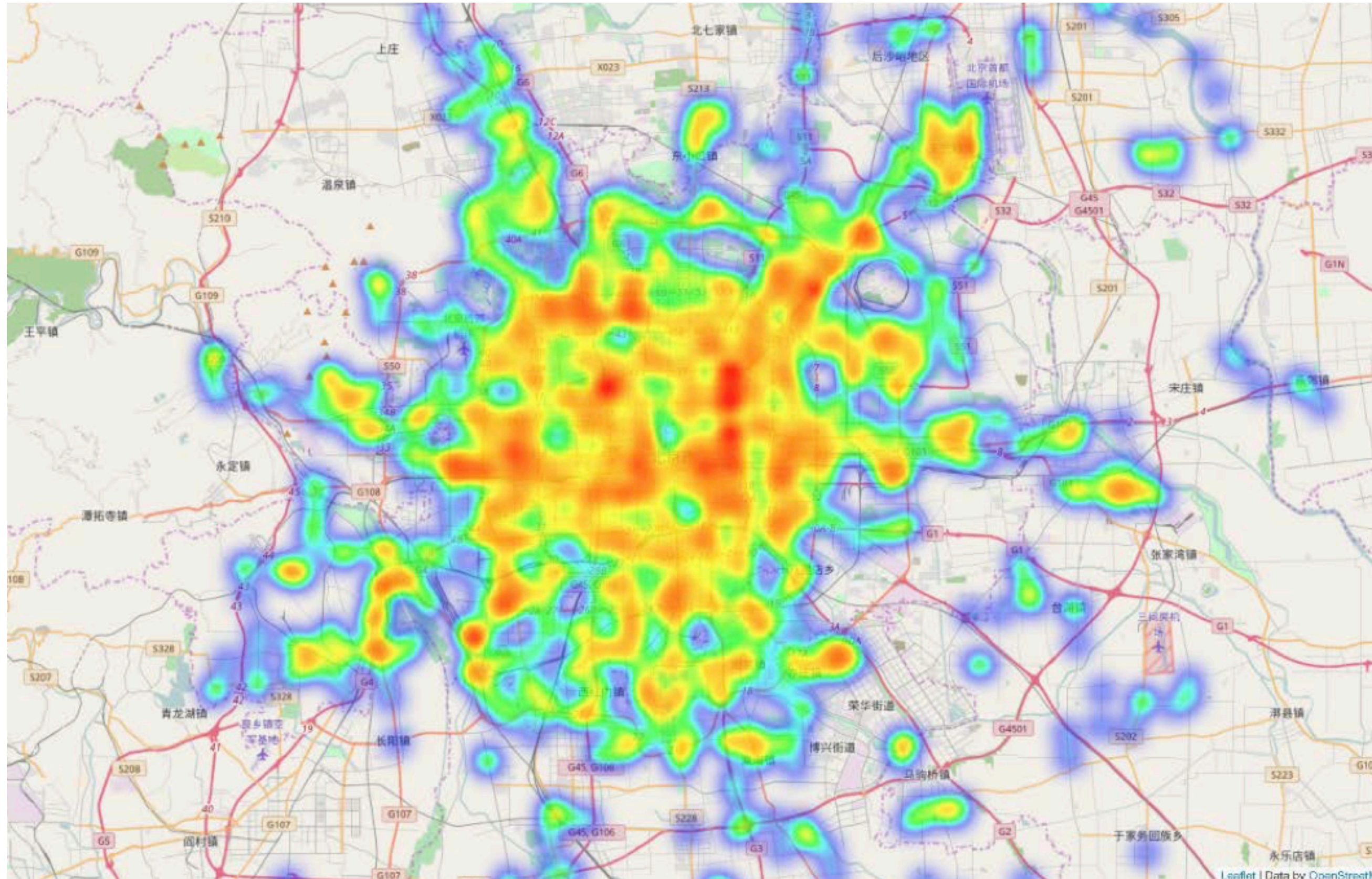
¹ Software offerings can be further classified into SDKs, firmware / enablers, algorithms / applications, simulators etc. but many companies are offering a mixture across the stack

² Many QPU providers are offering full stack services (e.g. Pasqal acquired Qu&Co, Quantinuum was originally CQC prior to merger with HQS, etc.)

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

Traffic in the city 10,000 cars



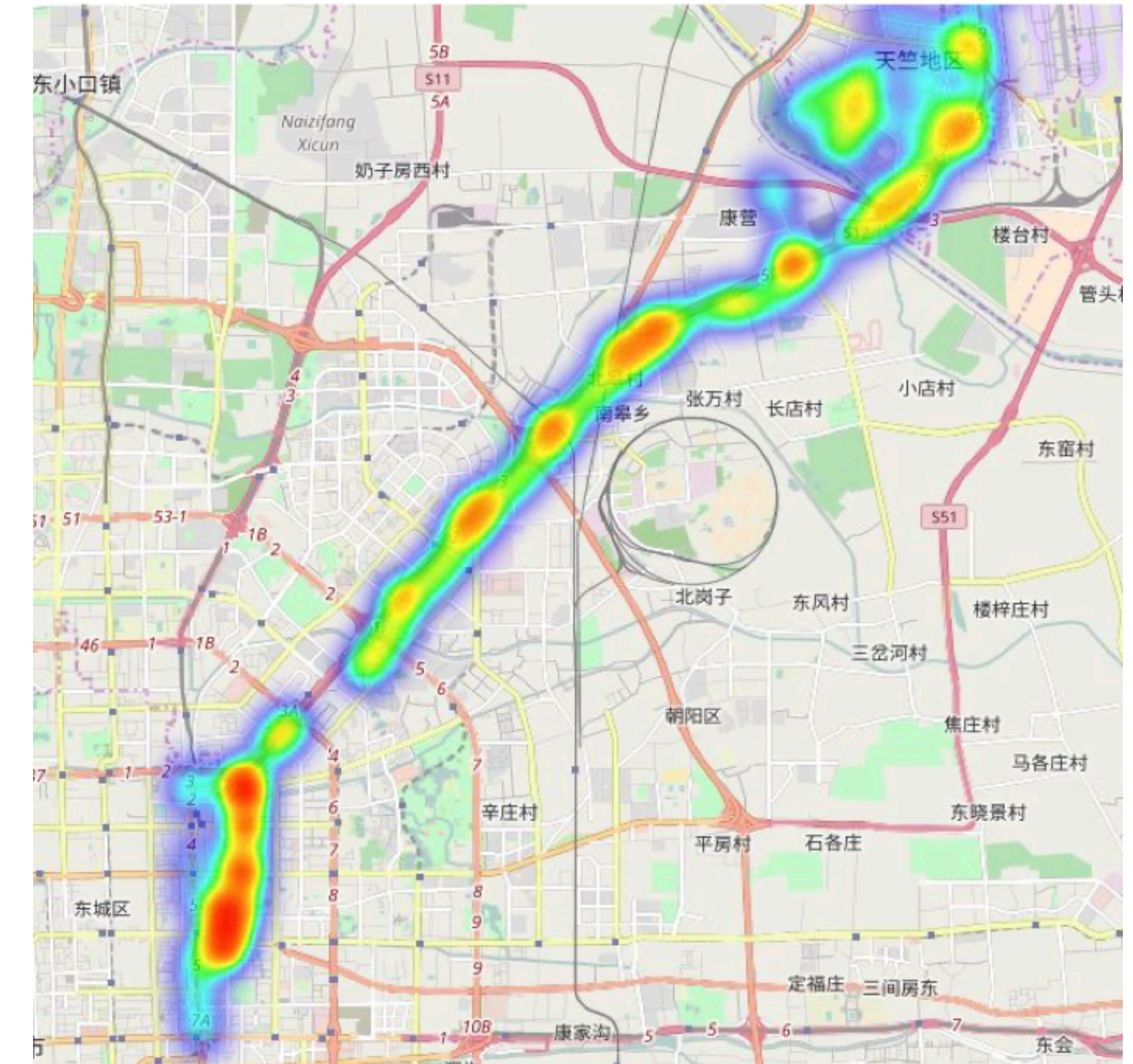
Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella

Route to the Airport 418 cars



- We assigned each of the 418 cars 3 possible routes to reach the airport
- Size of the problem space: 3^{418}

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

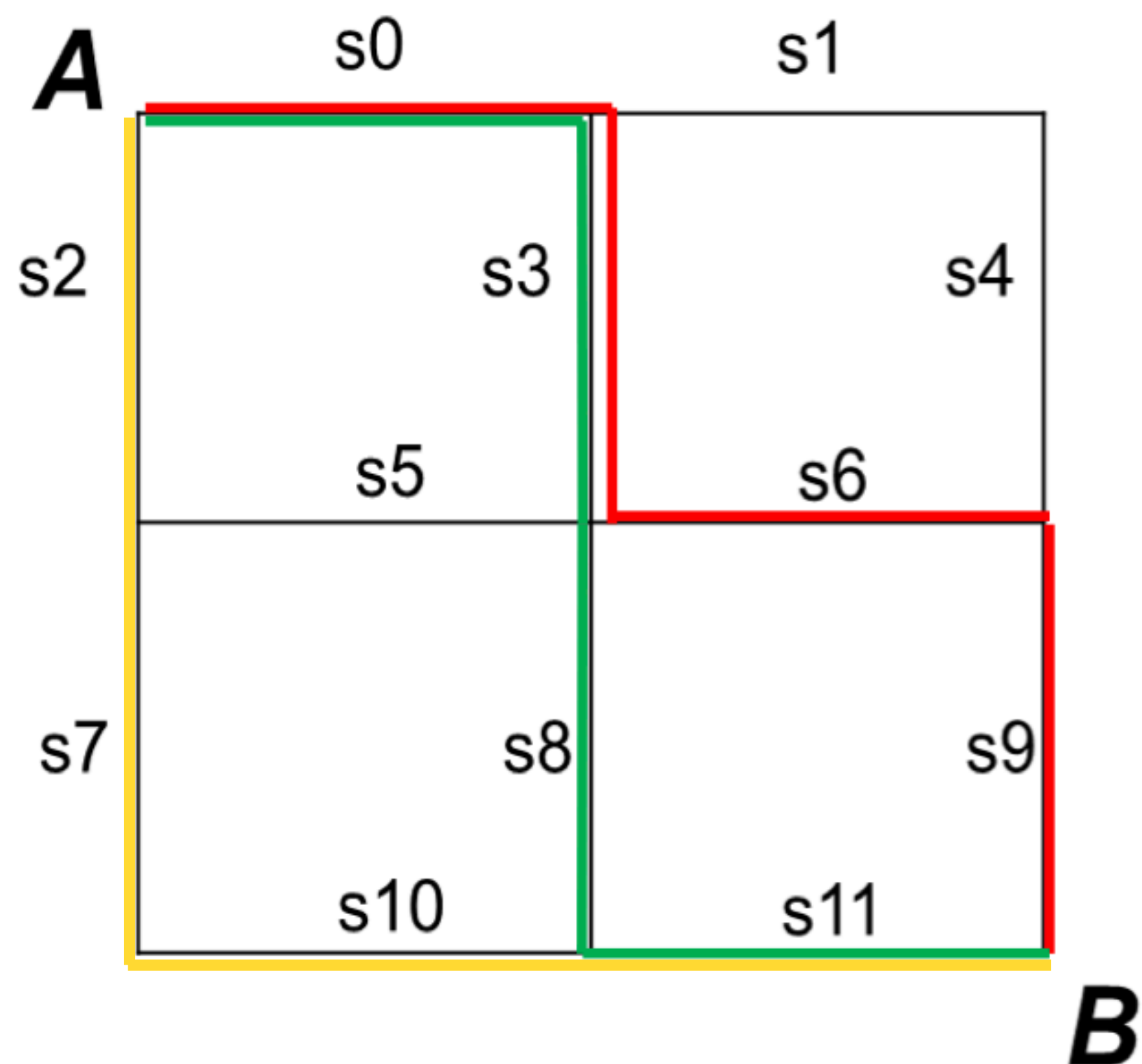
D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella

Example:

Simplified graph structure representing a route grid.

2 cars with 3 route options on a 2 x 2 grid.



Car	Route	Binary variable
Car 1	#1: s0,s3,s6,s9	Q_{11}
Car 1	#2: s0,s3,s8,s11	Q_{12}
Car 1	#3: s2,s7,s10,s11	Q_{13}
Car 2	#1: s0,s3,s6,s9	Q_{21}
Car 2	#2: s0,s3,s8,s11	Q_{22}
Car 2	#3: s2,s7,s10,s11	Q_{23}

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

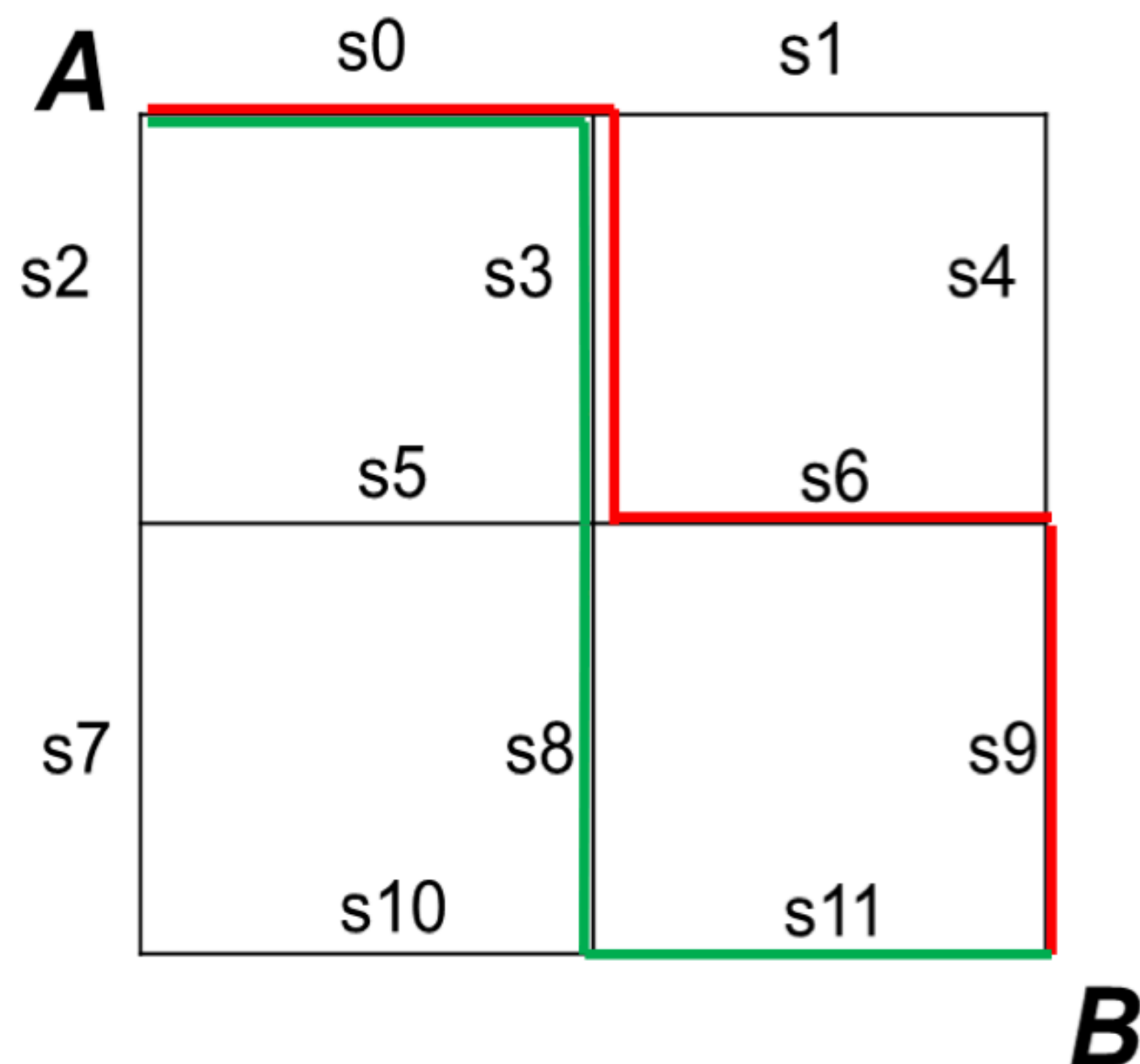
Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella

More cars on one street lead to a higher costs



Street segment	Associated cost function	Value
s0	$(Q_{11} + Q_{12} + Q_{21} + Q_{22})^2$	4
s3	$(Q_{11} + Q_{12} + Q_{21} + Q_{22})^2$	4
s6	$(Q_{11} + Q_{21})^2$	1
s9	$(Q_{11} + Q_{21})^2$	1
s8	$(Q_{12} + Q_{22})^2$	1
s11	$(Q_{12} + Q_{22} + Q_{13} + Q_{23})^2$	1
s2	$(Q_{13} + Q_{23})^2$	0
s7	$(Q_{13} + Q_{23})^2$	0
s10	$(Q_{13} + Q_{23})^2$	0

Example:

$$(Q_{11} + Q_{12} + Q_{21} + Q_{22})^2 + (Q_{11} + Q_{12} + Q_{21} + Q_{22})^2 + (Q_{11} + Q_{21})^2 + (Q_{11} + Q_{21})^2 + (Q_{12} + Q_{22})^2 + (Q_{12} + Q_{22} + Q_{13} + Q_{23})^2 + (Q_{13} + Q_{23})^2 + (Q_{13} + Q_{23})^2 + (Q_{13} + Q_{23})^2 = 12$$

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

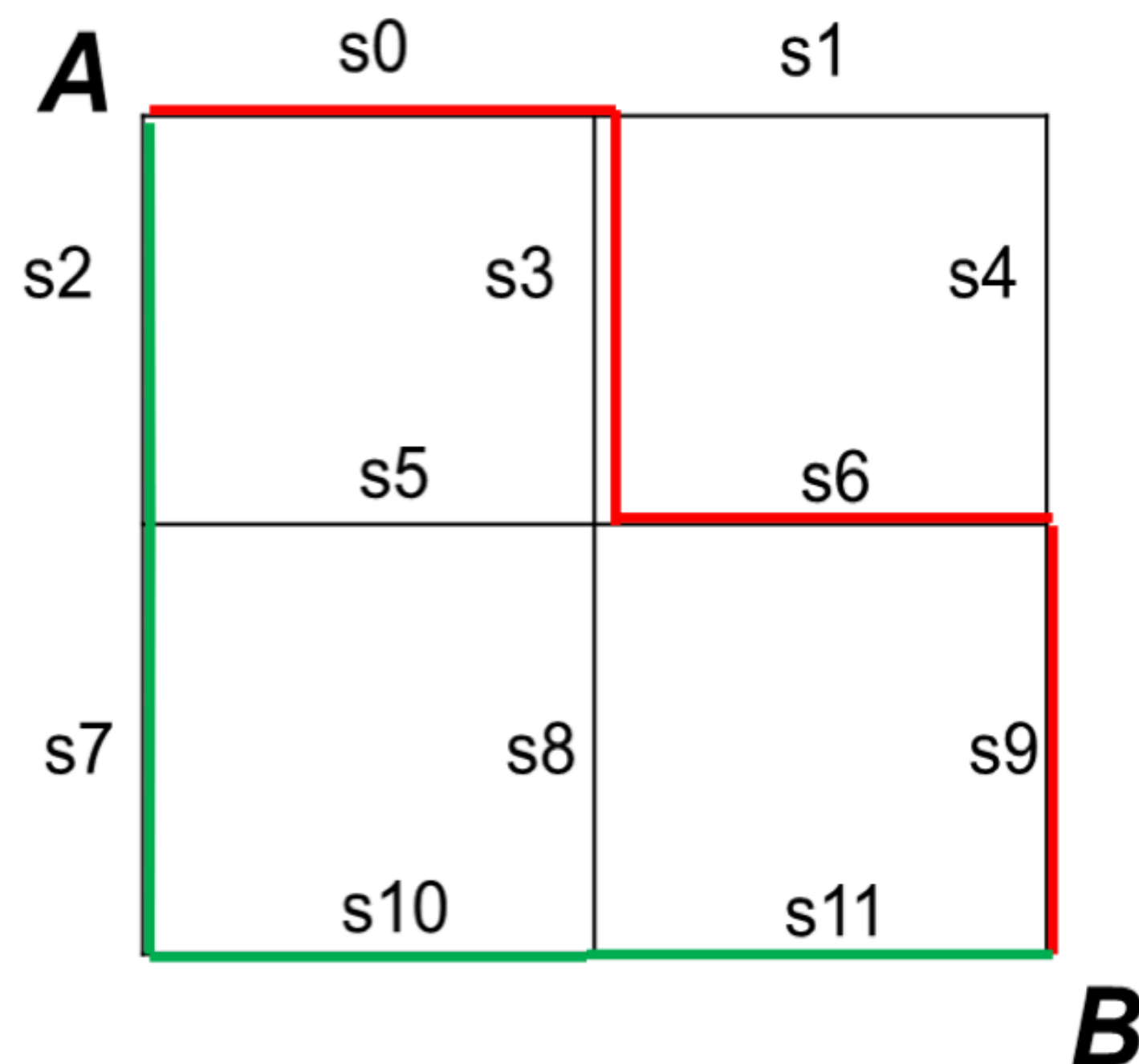
Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella

More cars on one street lead to a higher costs



Street segment	Associated cost function	Value
s0	$(Q_{11} + Q_{12} + Q_{21} + Q_{22})^2$	1
s3	$(Q_{11} + Q_{12} + Q_{21} + Q_{22})^2$	1
s6	$(Q_{11} + Q_{21})^2$	1
s9	$(Q_{11} + Q_{21})^2$	1
s8	$(Q_{12} + Q_{22})^2$	0
s11	$(Q_{12} + Q_{22} + Q_{13} + Q_{23})^2$	1
s2	$(Q_{13} + Q_{23})^2$	1
s7	$(Q_{13} + Q_{23})^2$	1
s10	$(Q_{13} + Q_{23})^2$	1

Example:

$$(Q_{11} + Q_{12} + Q_{21} + Q_{22})^2 + (Q_{11} + Q_{12} + Q_{21} + Q_{22})^2 + (Q_{11} + Q_{21})^2 + (Q_{11} + Q_{21})^2 + (Q_{12} + Q_{22})^2 + (Q_{12} + Q_{22} + Q_{13} + Q_{23})^2 + (Q_{13} + Q_{23})^2 + (Q_{13} + Q_{23})^2 + (Q_{13} + Q_{23})^2 = 8$$

Goal: minimize the overall costs => distribute cars to different streets

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella

Penalty term

We want to make sure that the solution will contain only one route for each car.

Only one binary variable per car can be 1. (otherwise the car would be on multiple routes simultaneously)

$$K(x_{11} + x_{12} + x_{13} - 1)^2$$

$$K(x_{21} + x_{22} + x_{23} - 1)^2$$

$$\begin{aligned} &K \cdot (Q_{11} + Q_{12} + Q_{13} - 1)^2 \\ &= K \cdot (-Q_{11} - Q_{12} - Q_{13} + 1 + 2Q_{11}Q_{12} + 2Q_{11}Q_{13} + 2Q_{12}Q_{13}) \end{aligned}$$

條件類型	QUBO 懲罰表示形式
選一個 : $\sum x_i = 1$	$P \cdot (\sum x_i - 1)^2$
至多一個 : $\sum x_i \leq 1$	$P \cdot \sum_{i < j} x_i x_j$
相等條件 : $x_i = x_j$	$P \cdot (x_i - x_j)^2$
相反條件 : $x_i = 1 - x_j$	$P \cdot (x_i + x_j - 1)^2$
容量限制 : $\sum w_i x_i \leq C$	$P \cdot (\max(0, \sum w_i x_i - C))^2$

$$F = \begin{pmatrix} Q_{11} & Q_{12} & Q_{13} & Q_{21} & Q_{22} & Q_{23} \end{pmatrix} \begin{pmatrix} 1-K & 2+2K & 2K & 2 & 0 & 0 \\ 1-K & 2K & 2 & 0 & 0 & 0 \\ -K & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1-K & 2+2K & 2K \\ 0 & 0 & 0 & 1-K & 2K & 2K \\ 0 & 0 & 0 & 0 & 0 & -K \end{pmatrix} \begin{pmatrix} Q_{11} \\ Q_{12} \\ Q_{13} \\ Q_{21} \\ Q_{22} \\ Q_{23} \end{pmatrix}$$

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

Quantum Computing at Volkswagen:

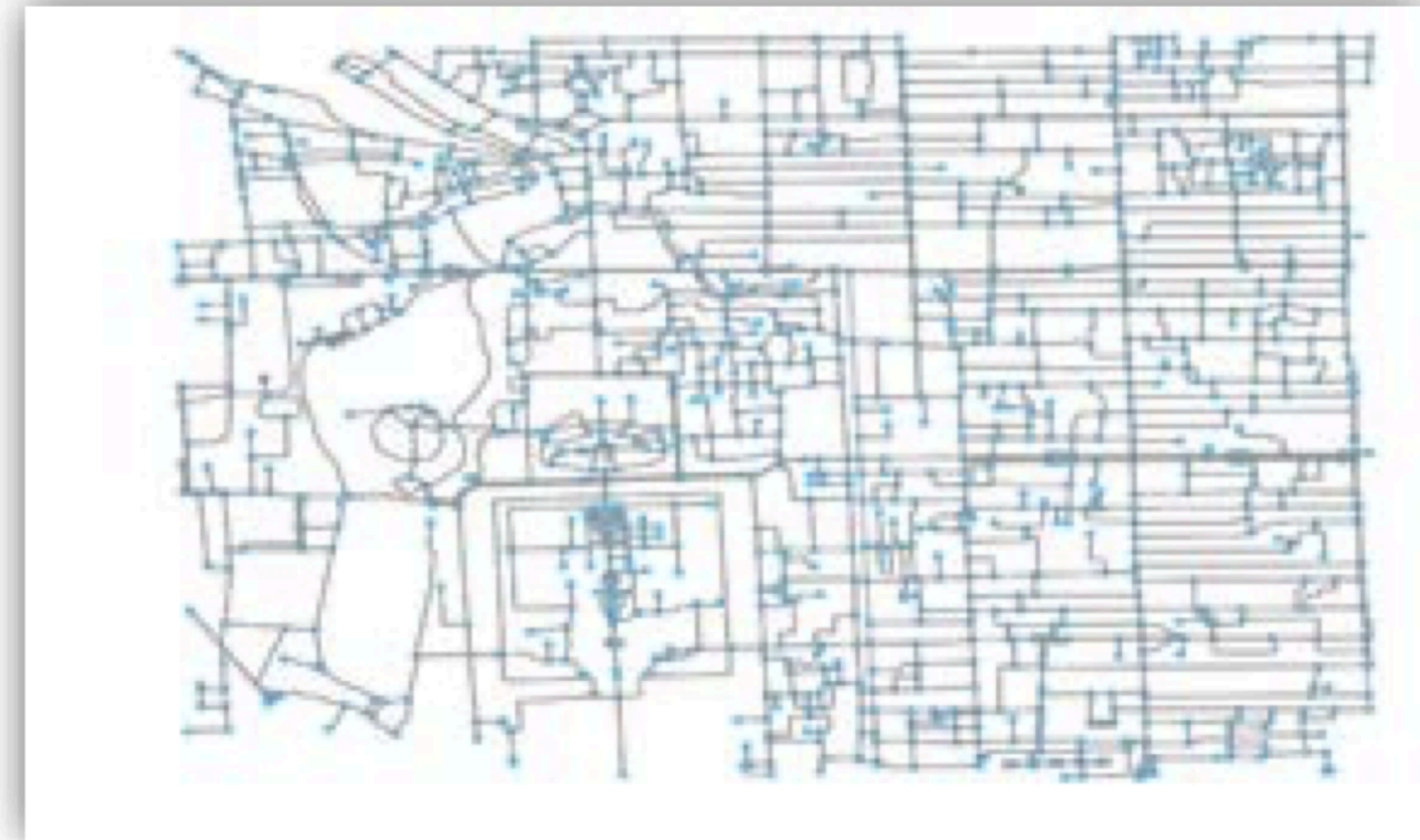
Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella

Data preprocessing

- Identification of street segments in the map by turning it into a graph using OSMnx, a Python package for street networks
- Get *valid* alternative routes for every car that needs to be rerouted
- Determine overlapping segments in the possible routes and define the cost function for the problem



Graph representation of the street map

DEMO 1: Traffic Flow Optimization

<https://arxiv.org/abs/1708.01625>

```
from dwave_sapi2.core import solve_ising, solve_qubo, async_solve_ising,
from dwave_sapi2.util import ising_to_qubo, qubo_to_ising, get_hardware_
from dwave_sapi2.embedding import find_embedding, embed_problem, unembed
from dwave_sapi2.remote import RemoteConnection
```

```
rc = RemoteConnection(url, token)
solver = rc.get_solver(solver name)
```

```
# # construct Ising problem
h, J, _ = qubo_to_ising({(i, j): qubo[i][j]
                        for i in range(nCars*nVars) for j in range(nCars*nVars)})
```

```
## Embedding onto hardware-like solvers:
embedding = find_embedding(J, adj)
```

```
if embedding is successful, continue
    if len(embedding):
        # print 'Total number of physical qubits:'
        # print sum(map(len, embedding))
```

```
hemb, jemb, jchain, embedding = embed_problem(h, J, embedding, adj)
high = max(map(abs, jemb.values()))
chain_strength = 5
for (i, j) in jchain.keys():
    jemb[(i, j)] = -high * chain_strength
    params = { "answer_mode": "histogram", "num_reads": 1000}
    num_iterations = 2
    # solve Ising problem with parameters
    answer = solve_ising(solver, hemb, jemb, **params)
```

Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

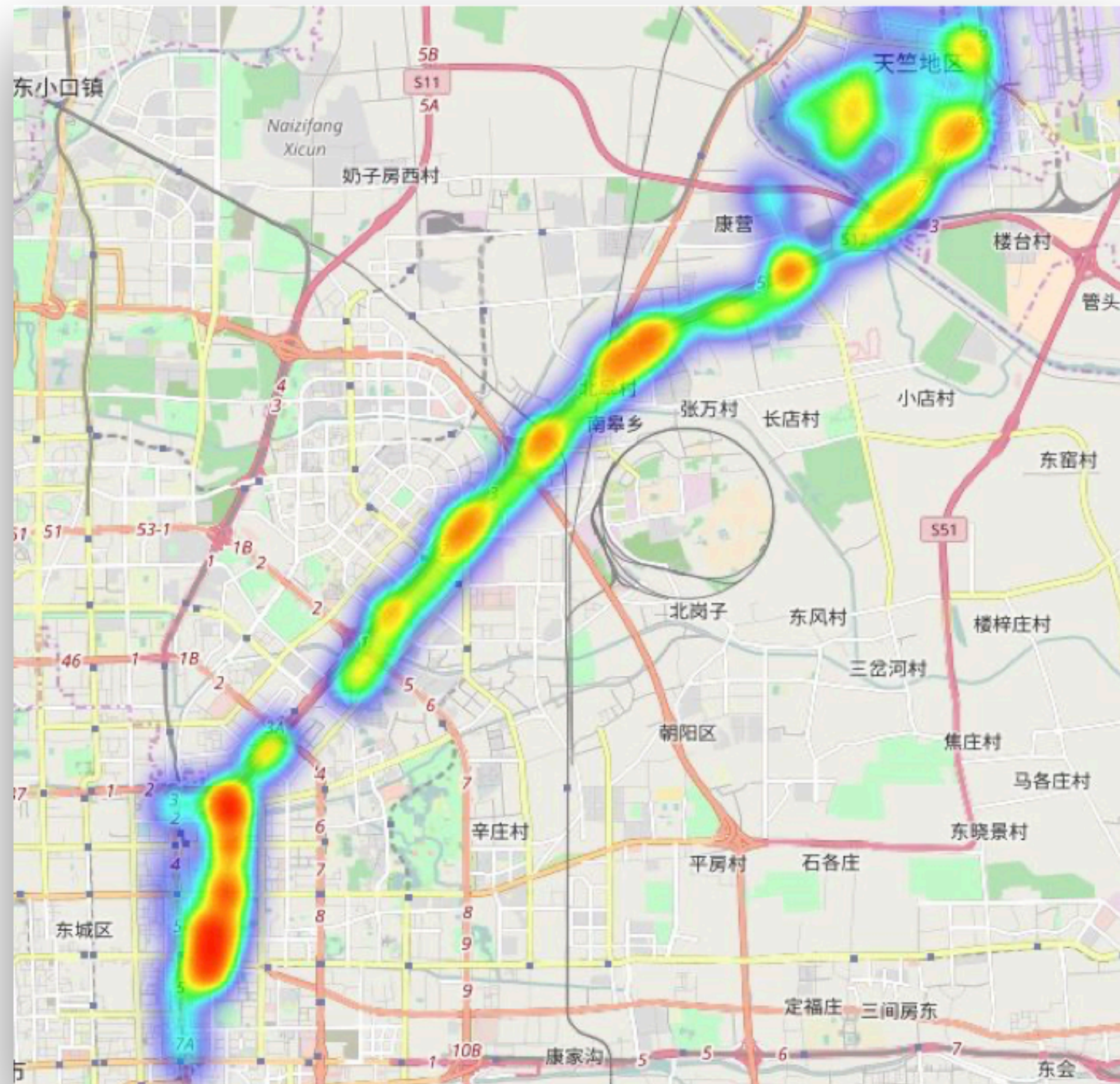
27.09.2017 – Dr. Gabriele Compostella

1. Import required modules
2. Establish a connection to D-Wave
3. Convert the QUBO model to an Ising formulation
4. Attempt to embed the Ising problem onto the physical QPU topology
5. If embedding succeeds, set chain parameters and submit the problem
6. Retrieve and store the best solution

[illegible]

<https://arxiv.org/abs/1708.01625>

RESULT: Unoptimized vs Optimized traffic

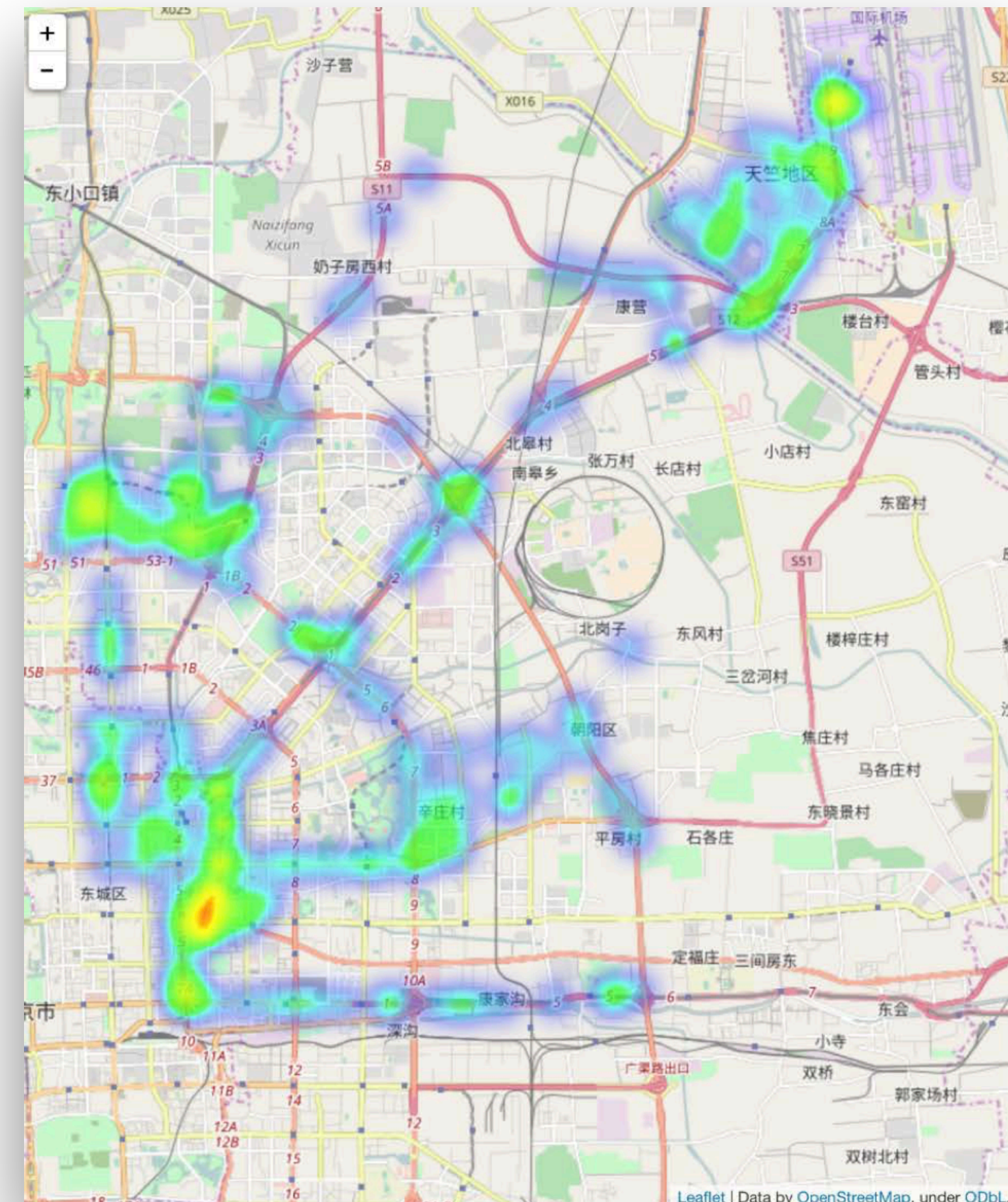


Quantum Computing at Volkswagen:

Traffic Flow Optimization using the D-Wave Quantum Annealer

D-Wave Users Group Meeting - National Harbour, MD

27.09.2017 – Dr. Gabriele Compostella



QA & References

Is the quantum annealing algorithm really better?

- <https://ycc.idv.tw/algorithm-complexity-theory.html>
- <https://medium.com/data-science/an-introduction-to-a-powerful-optimization-technique-simulated-annealing-87fd1e3676dd>
- <https://zh.wikipedia.org/zh-tw/模拟退火>
- <https://cosinewithsqrtminusone.blogspot.com/2014/08/infinite-square-well.html>
- <https://stats.stackexchange.com/questions/453309/what-is-the-relationship-between-metropolis-hastings-and-simulated-annealing>
- <https://zh.wikipedia.org/zh-tw/退火>
- <https://machining-quote.com/bolg/what-is-annealing-in-metal/>
- <https://www.intechopen.com/chapters/38520>
- <https://john1851.tumblr.com/post/104336520455/amp>
- <https://medium.com/@chs.li.work/quantum-annealing-qubo-914a3e47a9df>
- <https://kaiwu-sdk-docs.qboson.com/zh/source/tsp.html>
- <https://www.dwavequantum.com/media/erzjluvs/qubits-day-2-morning-5-vw.pdf>